

Convolutional Neural Networks

(CNNs, or ConvNet, DCN)

Sources:

“CS 790: Introduction to Machine Learning” at U. Wisconsin
by Jia-Bin Huang, U. Maryland
Ch. 9, “Deep Learning” textbook
by Goodfellow et al.

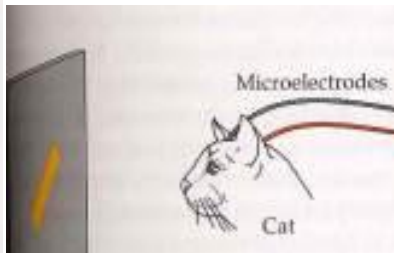
Receptive Fields of Lateral Geniculate and Primary Visual Cortex



David Hubel



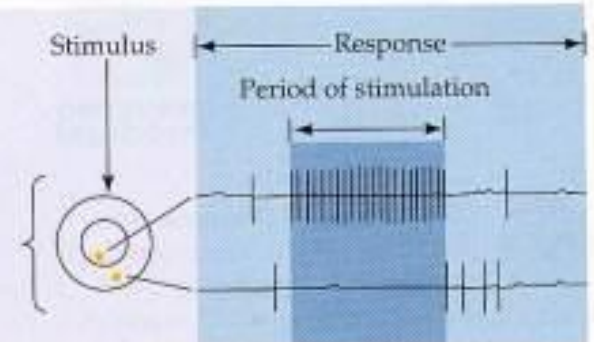
Torsten Wiesel



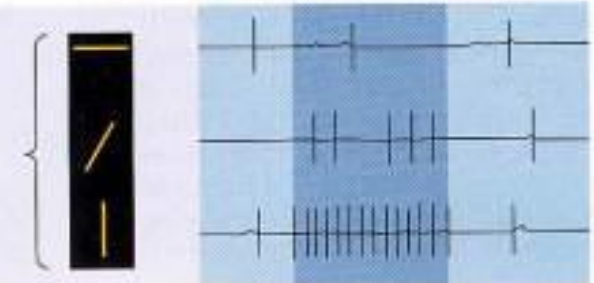
Examples of receptive fields of brain cells:

(a) Lateral geniculate cell with concentric field; on-center/off-surround.

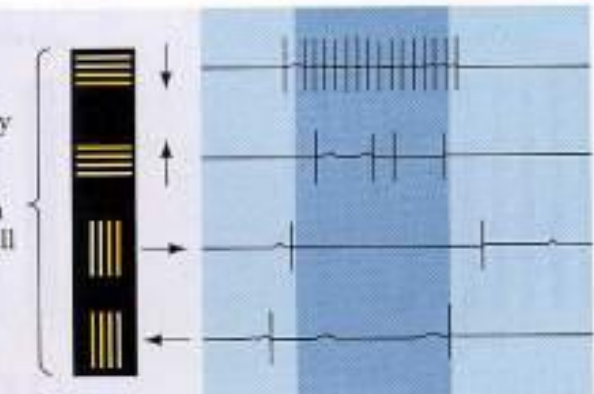
1. Response to light in center of cell's field
2. Response to light in periphery of cell's field



(b) Cortical cell sensitive to orientation. This cell responds strongly only when the stimulus is a vertical stripe.

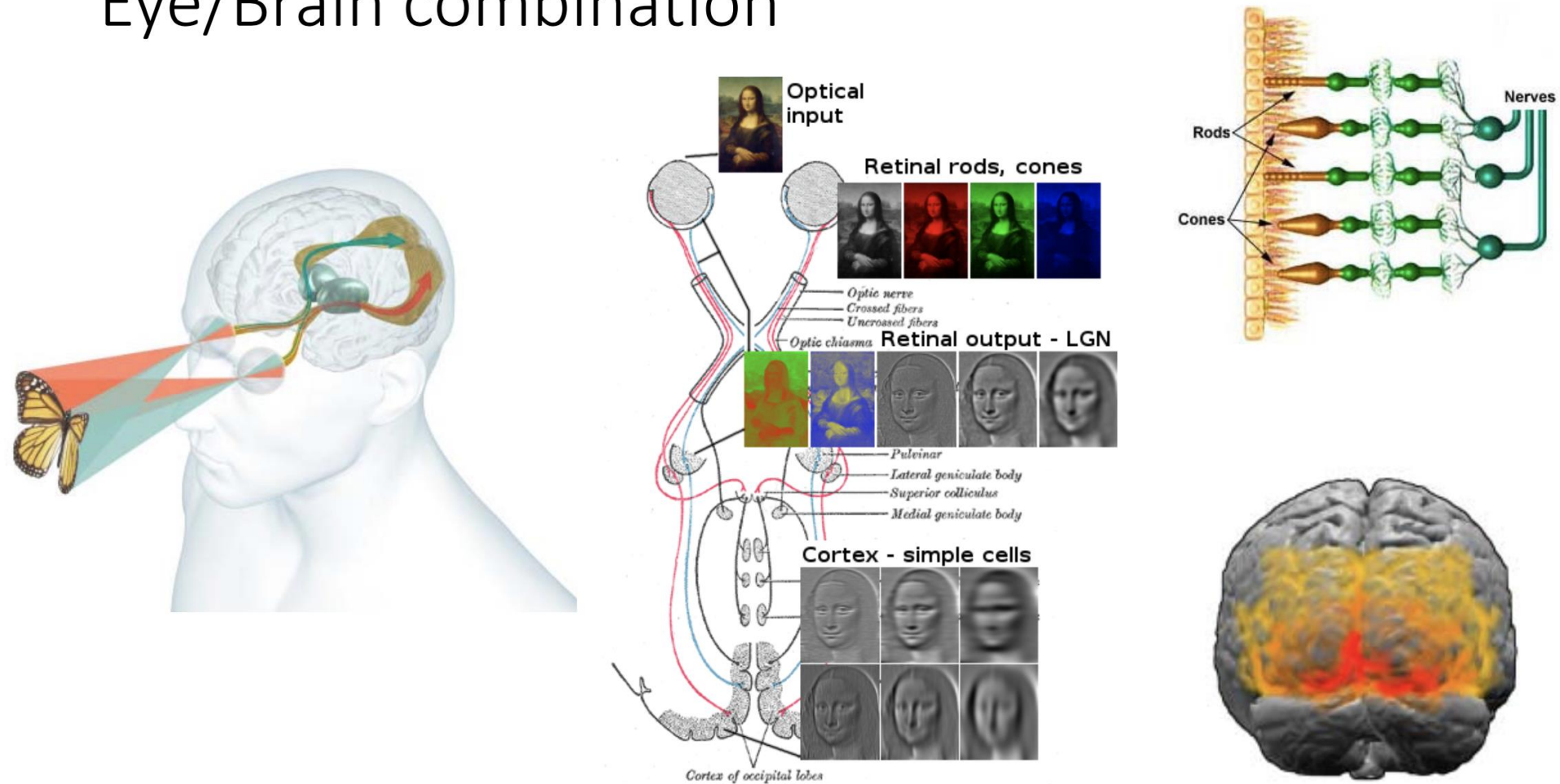


(c) Cortical cell sensitive to the direction of motion. This cell responds strongly only when the stimulus moves down. It responds weakly to upward motion and does not respond at all to sideways motion.

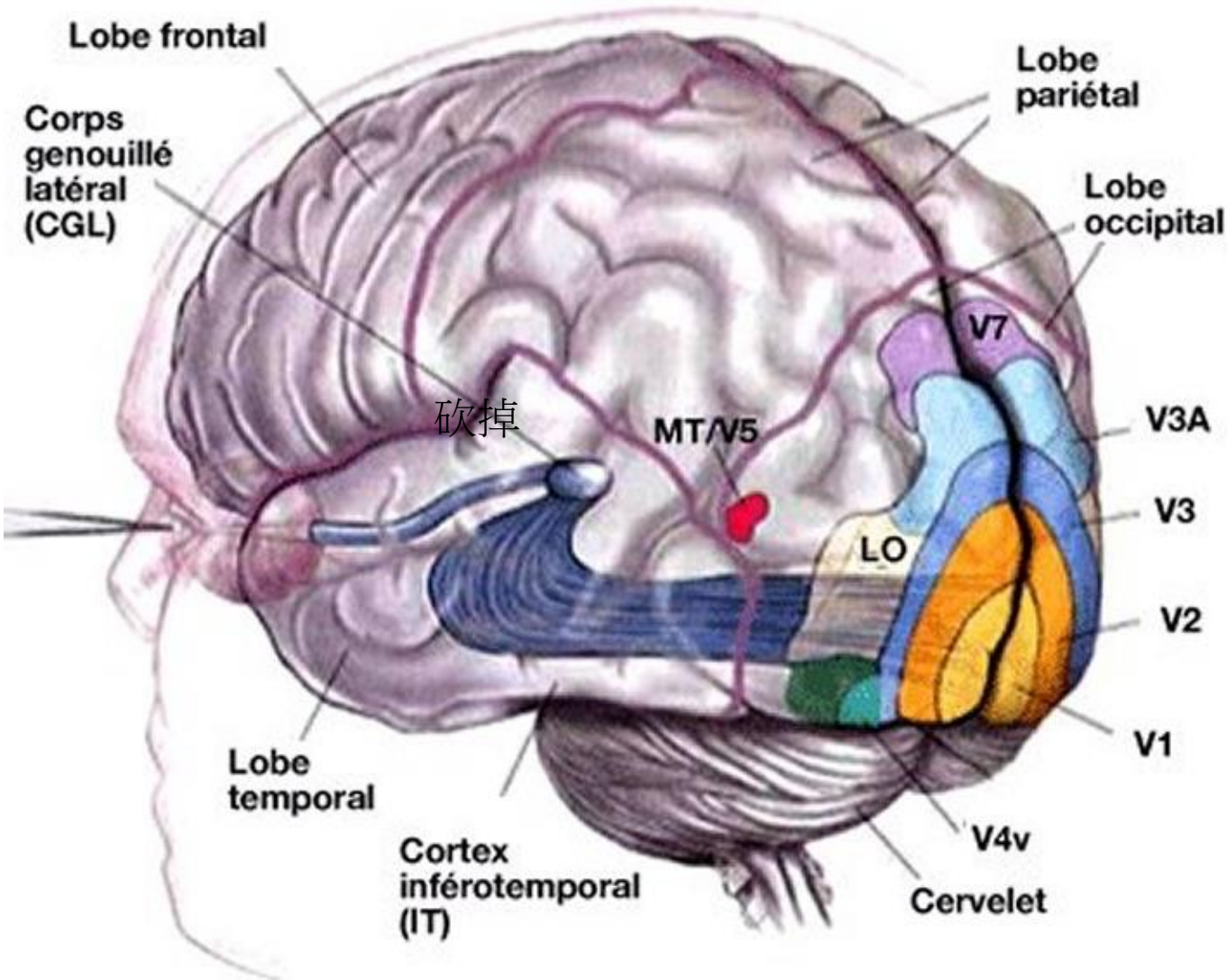


From Components to Systems

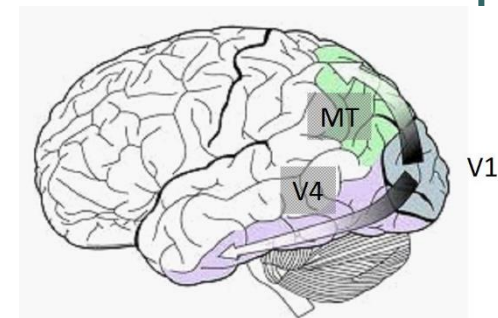
Eye/Brain combination



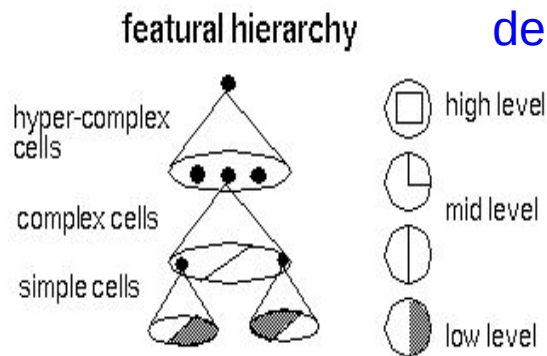
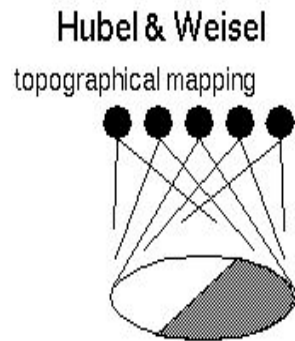
Human Cortical Visual Regions: V1, V2, V3, V4, V5 (MT)



- V1: orientation
- V2: corner
- V4: color
- IT: object recognition

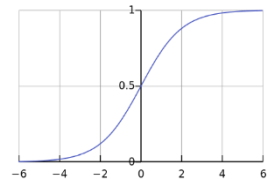
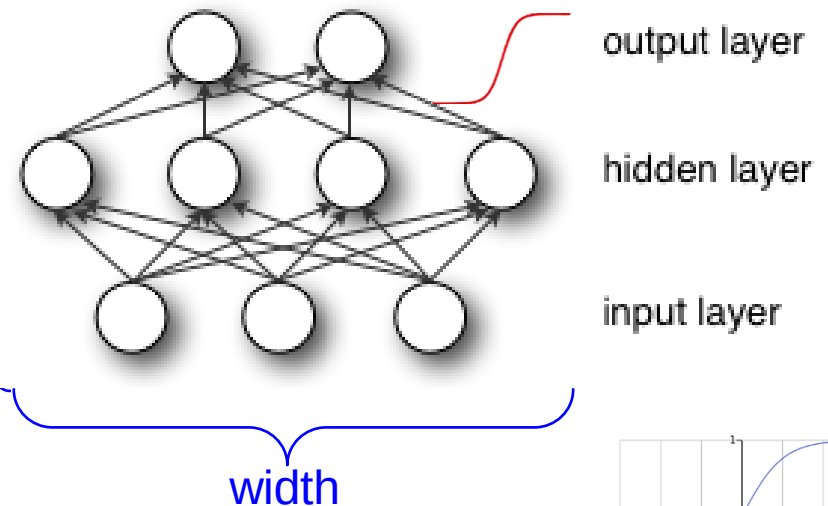


Hubel/Wiesel Architecture and Multi-layer Perceptrons



depth

Hubel and Wiesel's architecture



Multi-layer perceptrons

- A *non-linear classifier* with
sigmoid activation function

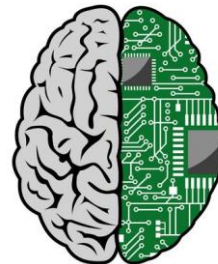


Image Categorization

Training Images



Training

Training Labels

Image Features

Classifier Training

Trained Classifier

Testing

Image Features

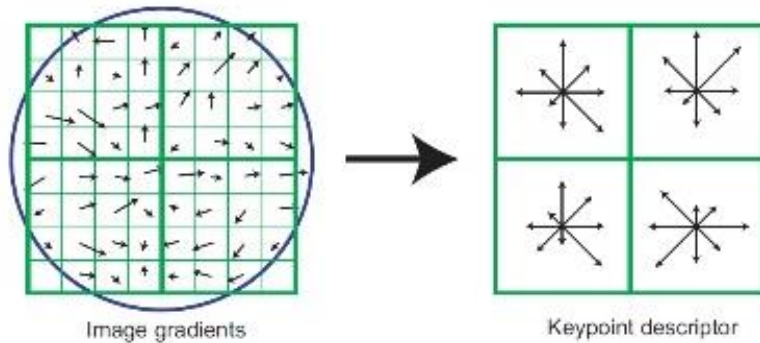
Trained Classifier

Prediction
Outdoor

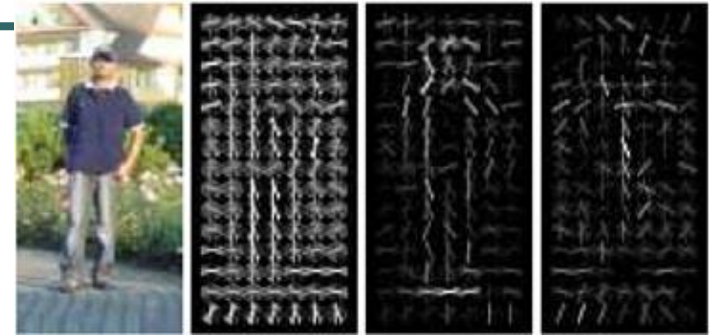
Test Image



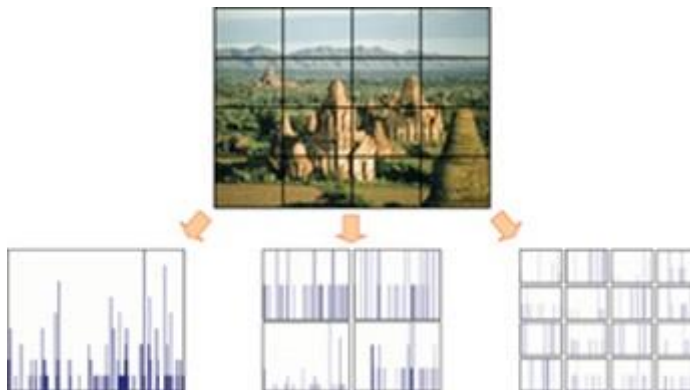
Features are the Keys



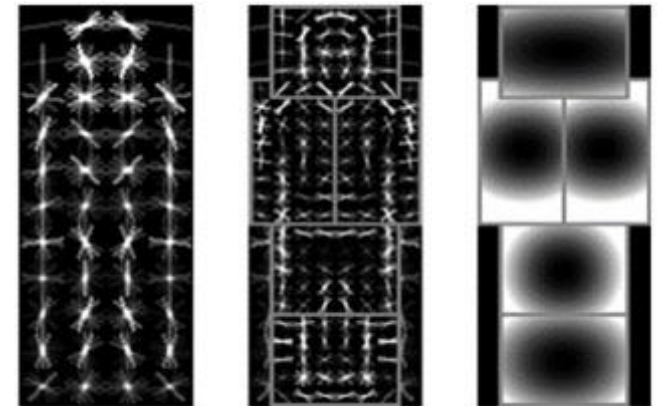
SIFT [[Lowe IJCV 04](#)]



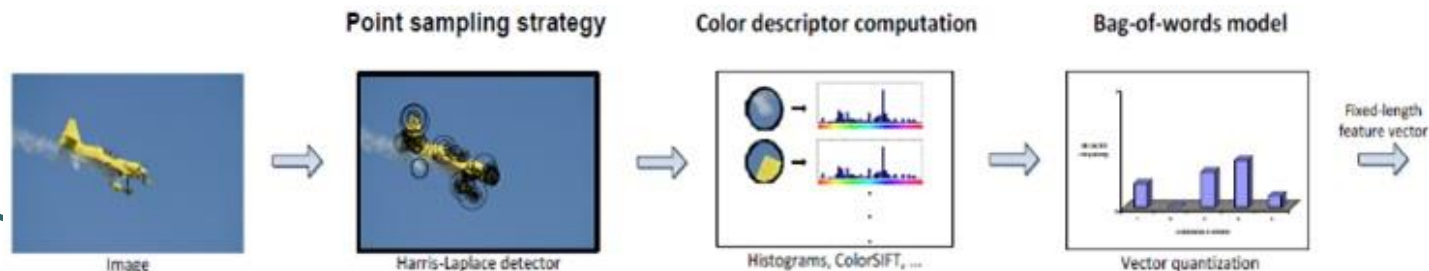
HOG [[Dalal and Triggs CVPR 05](#)]



SPM [[Lazebnik et al. CVPR 06](#)]



DPM [[Felzenszwalb et al. PAMI 10](#)]



Color Descriptor [[Van De Sande et al. PAMI 10](#)]

Learning a Hierarchy of Feature Extractors

- Each layer of hierarchy **extracts features** from output of previous layer
- All the way **from pixels → classifier**
- Layers have the (nearly) **same** structure

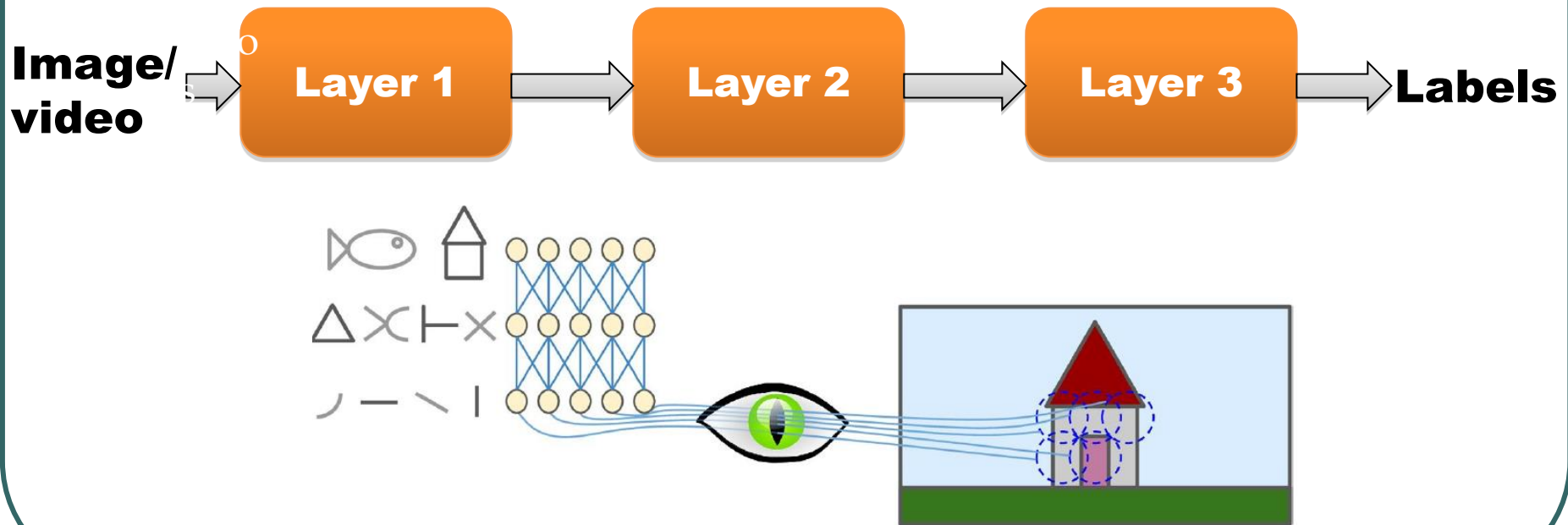


Figure 13-1. Local receptive fields in the visual cortex

Convolutional Layer

- The most important building block of a CNN is the convolutional layer.

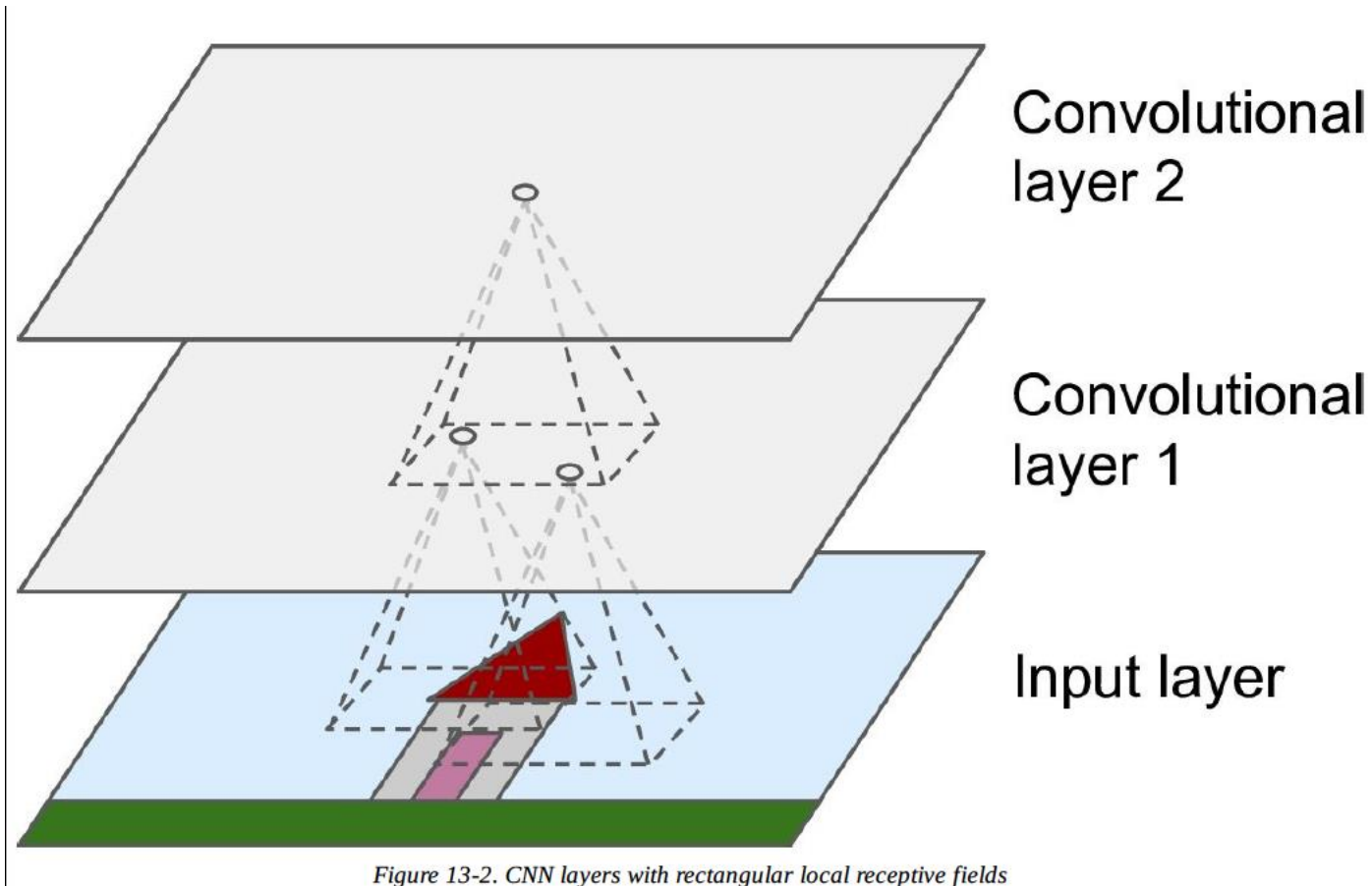
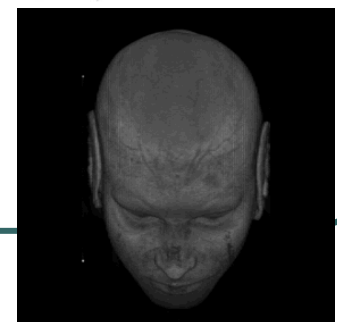
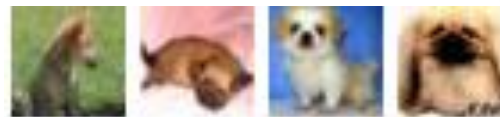
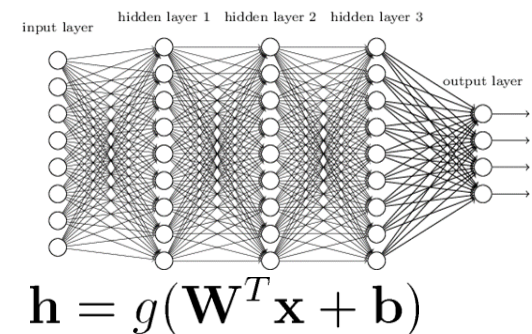


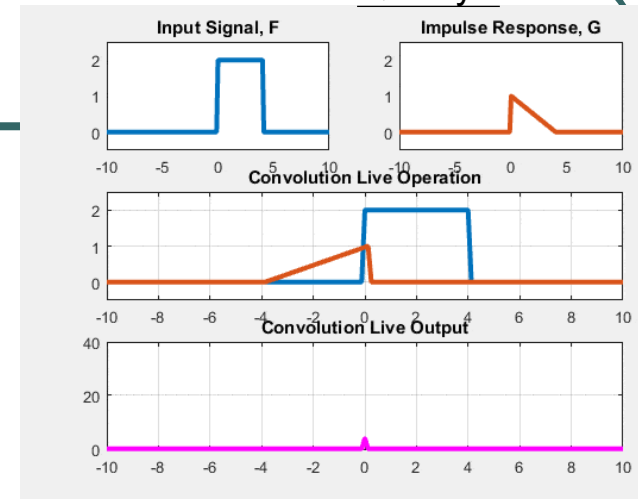
Figure 13-2. CNN layers with rectangular local receptive fields

Convolutional Neural Networks

- Convolutional neural networks (CNNs) use **convolution** in place of **matrix multiplication** in at least one of the layers of neural networks.
 - Everything else stays the same
 - Maximum likelihood
 - Negative log-likelihood
 - Back-propagation
 - CNNs are used to process data that has grid-like topology
 - 1-D: regularly sampled time-series data
 - 2-D: images
 - 3-D: volume data, video



Convolution Operation



- Convolution of two 1-D signals:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

- Convolution of a 2-D image with a 2-D kernel:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n)K(i - m, j - n), \text{ or}$$

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i - m, j - n)K(m, n)$$

- Convolution (actually cross-correlation) of a 2-D image with a 2-D kernel:

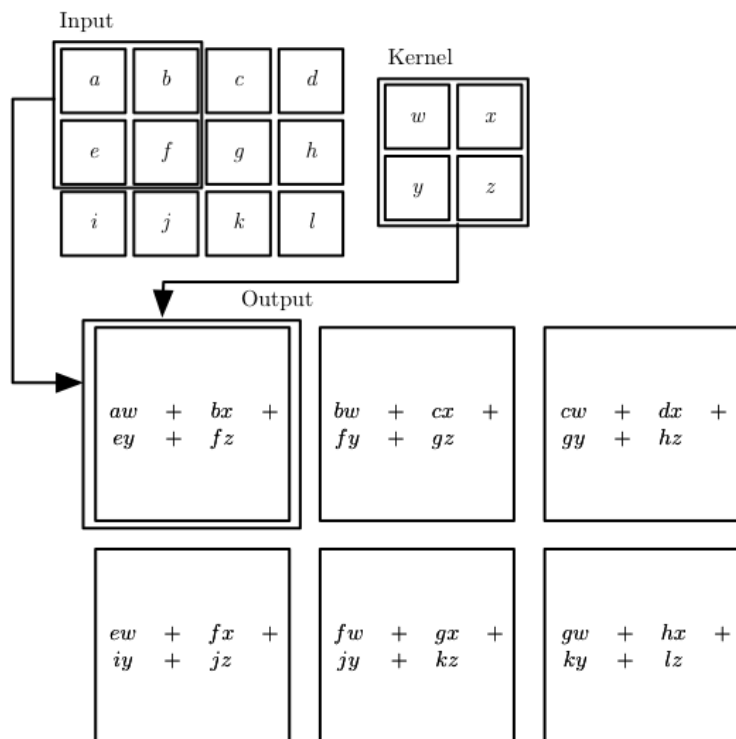
$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n)K(m, n)$$

Convolution Operation

- Convolution of a 2-D image with a 2-D kernel:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

feature map = input * kernel



1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

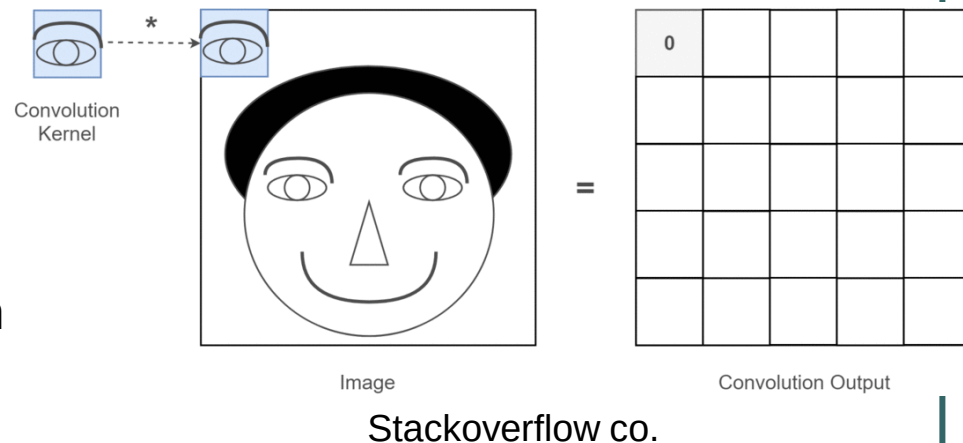
Image

4		

Convolved Feature

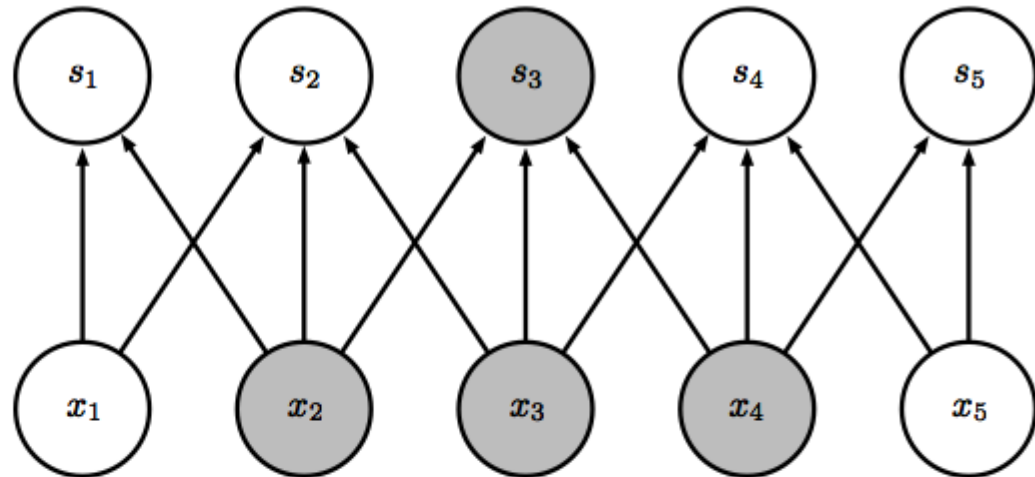
Motivation of CNN

- Scale up neural networks to process very large signals / images / video sequences
- Three essential ideas:
 - Sparse and local connectivity
 - Kernel is usually much smaller than input
 - Less memory and computation required
 - Parameter (weight) sharing
 - Use the same set of parameters for more than one function in a model
 - Equivalent representation
 - Automatically generalize across spatial translations of inputs

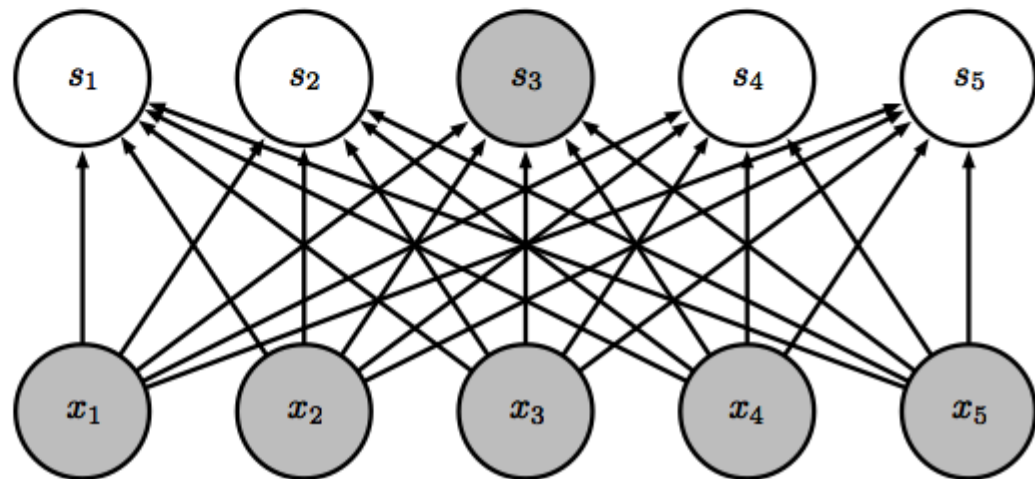


CNN: Local Connectivity

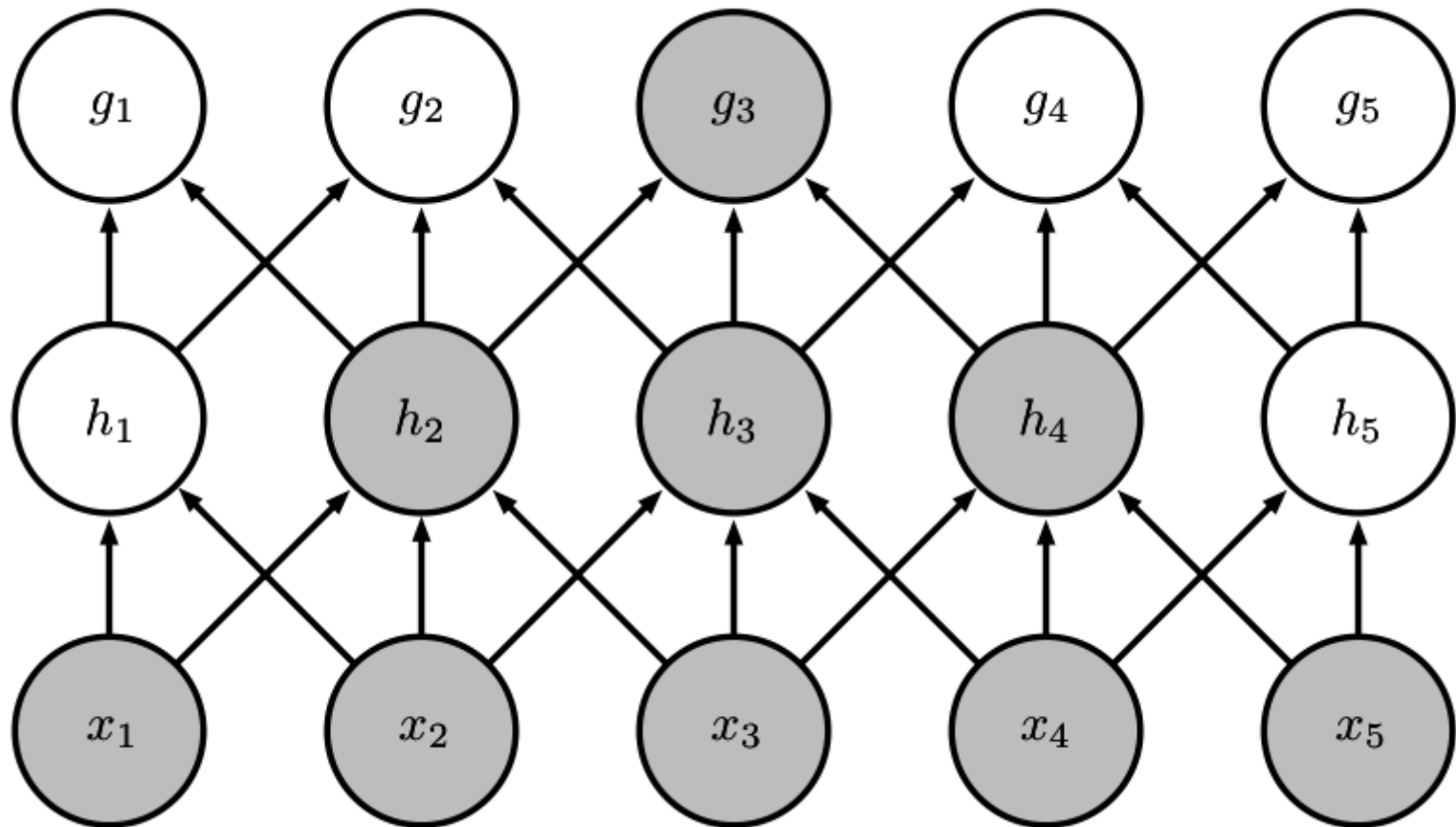
Sparse
connections
due to small
convolution
kernel

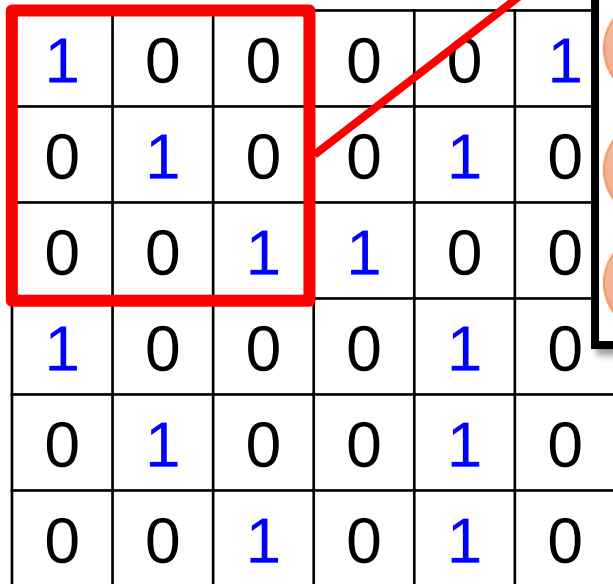
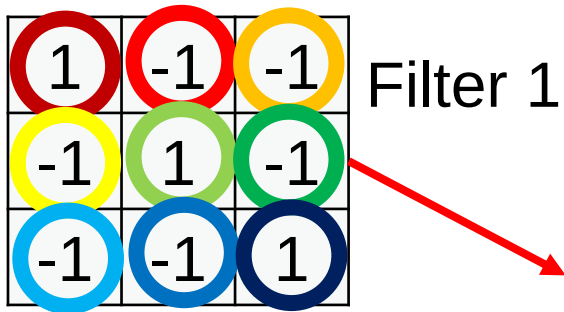


Dense
connections



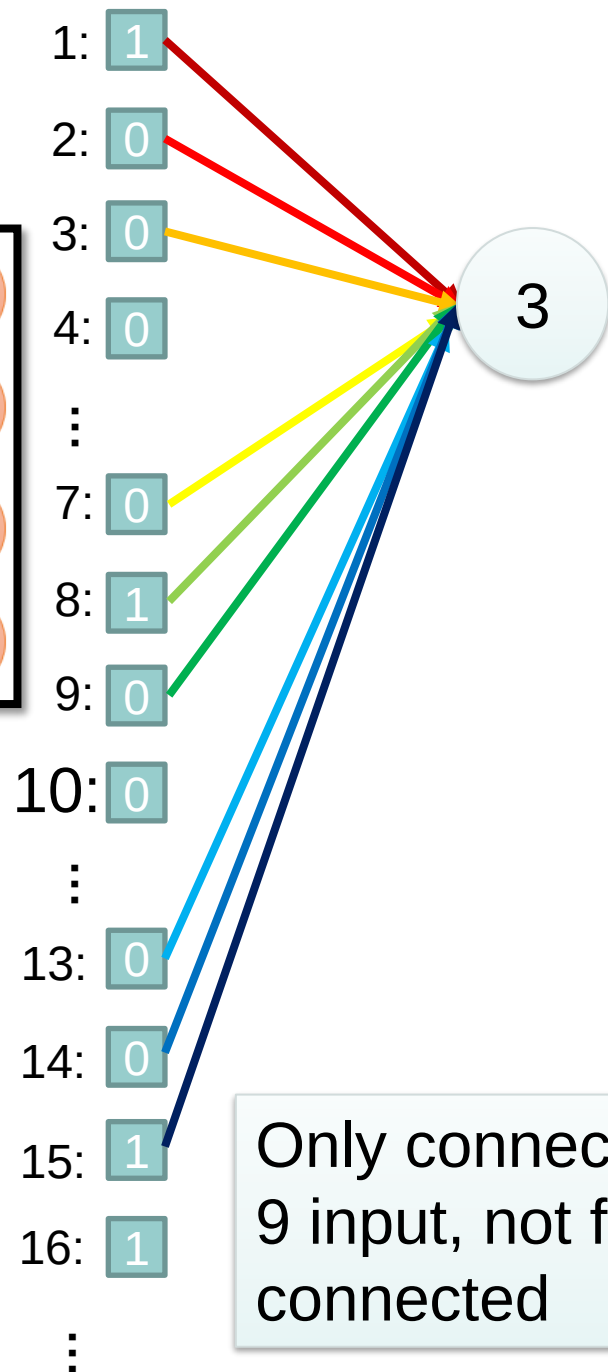
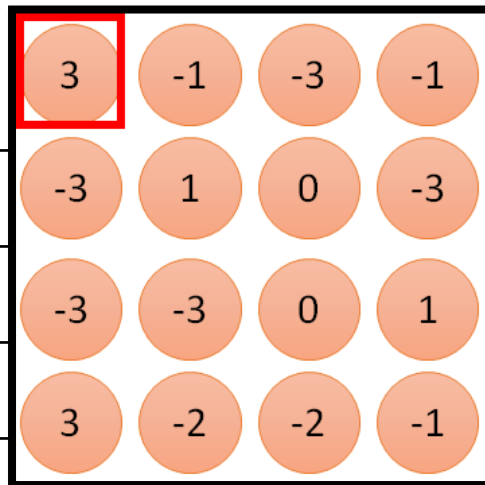
CNN: Growing Receptive Fields



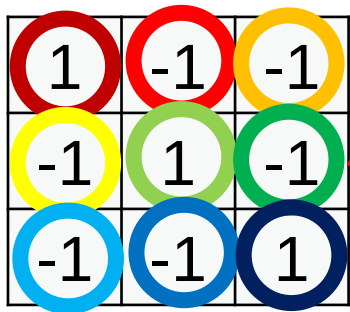


6 x 6 image

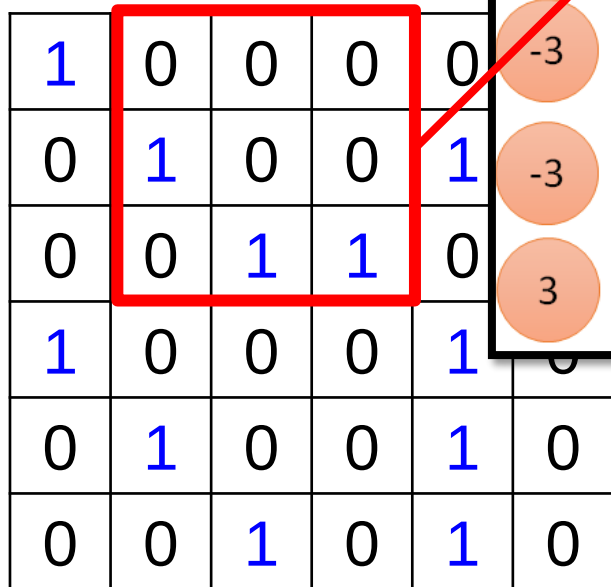
Less parameters!



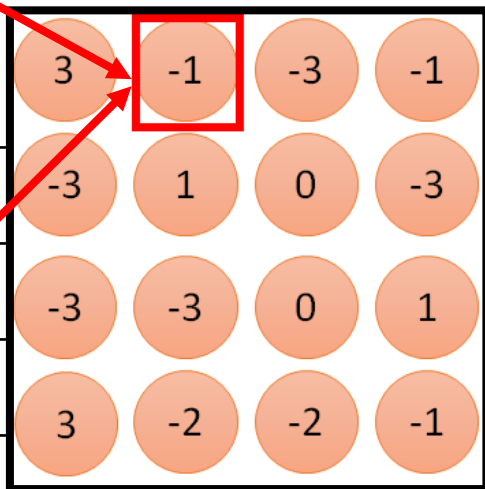
Only connect to 9 input, not fully connected



Filter 1

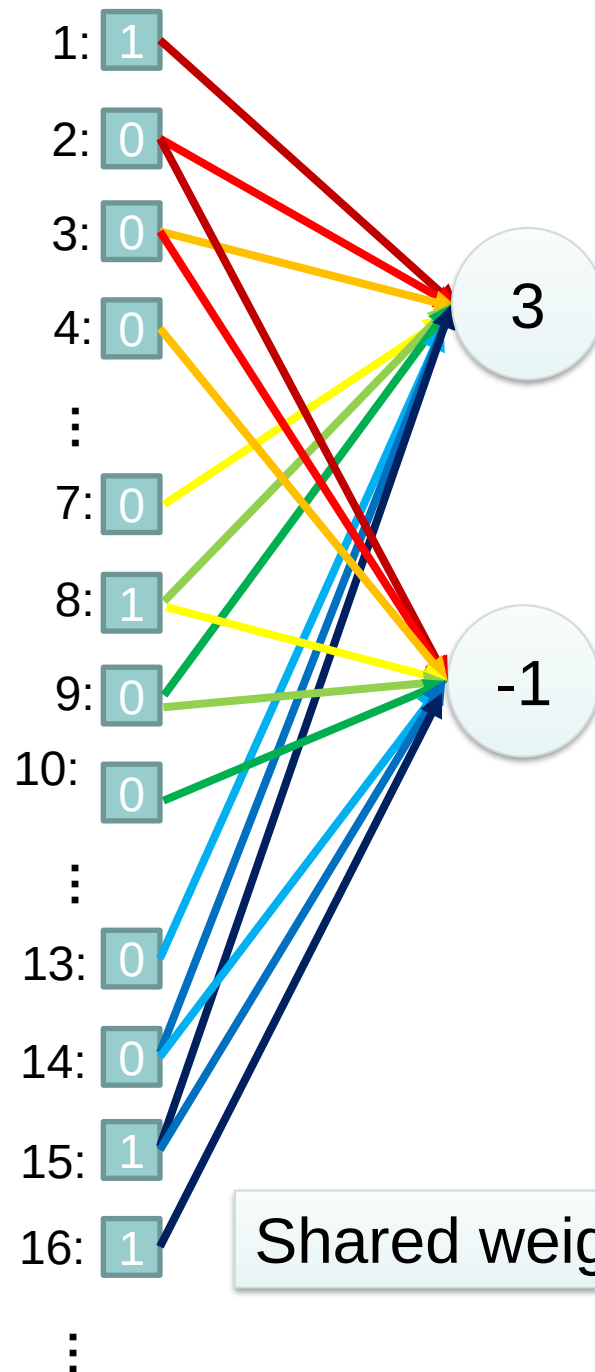


6 x 6 image

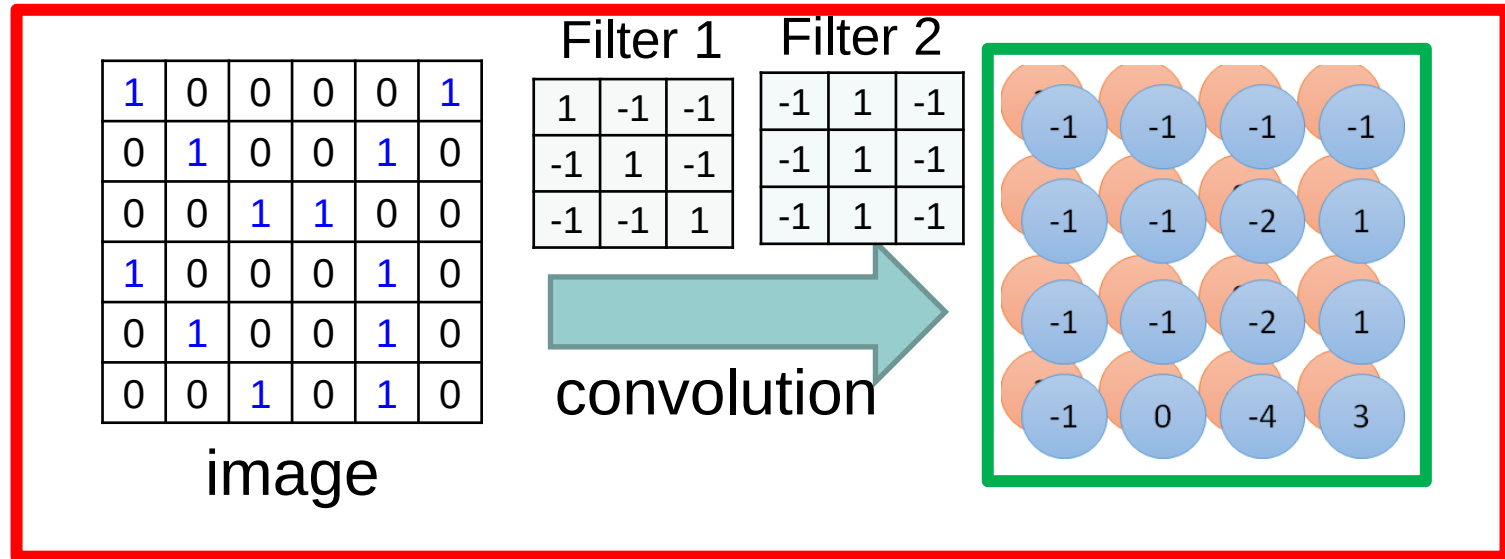


Less parameters!

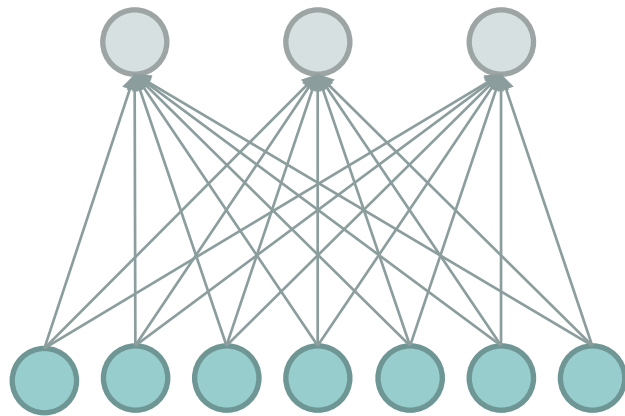
Even less parameters!



Feature map of each filter



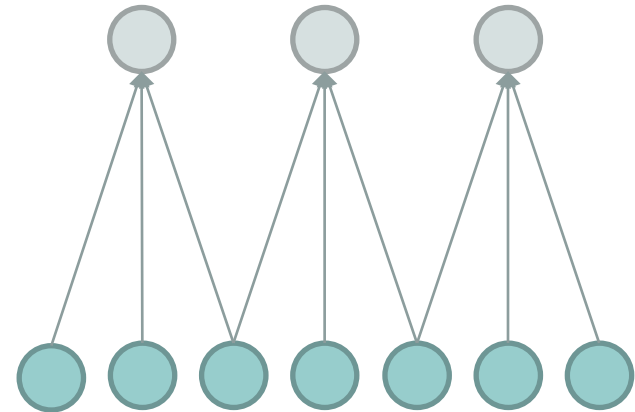
CNN: Local Connectivity



Global connectivity

Hidden layer

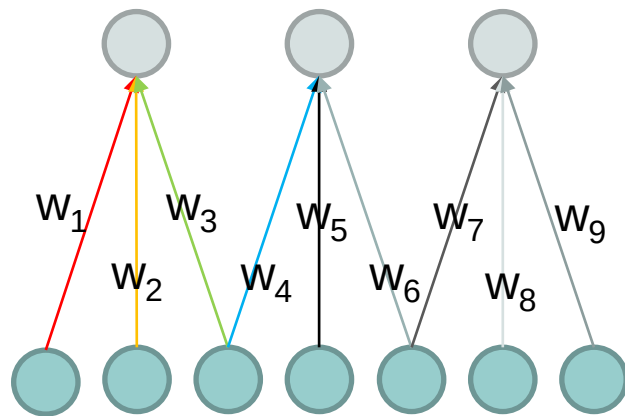
Input layer



Local connectivity

- # input units (neurons): 7
- # hidden units: 3
- Number of parameters
 - Global connectivity: $3 \times 7 = 21$
 - Local connectivity: $3 \times 3 = 9$

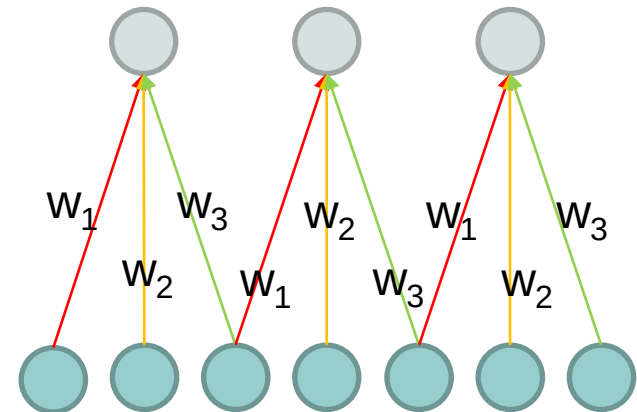
CNN: Weight Sharing



Without weight sharing

Hidden layer

Input layer



With weight sharing

- # input units (neurons): 7
- # hidden units: 3
- Number of parameters
 - Without weight sharing: $3 \times 3 = 9$
 - With weight sharing : $3 \times 1 = 3$

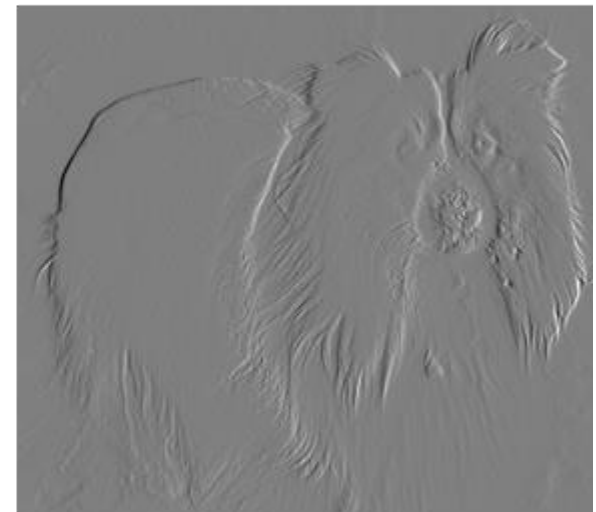
Edge Detection by Convolution



Input

-1	+1
----	----

Kernel



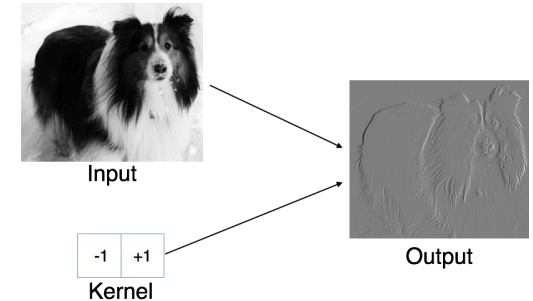
Output

Efficiency of Convolution

Input size: 320 by 280

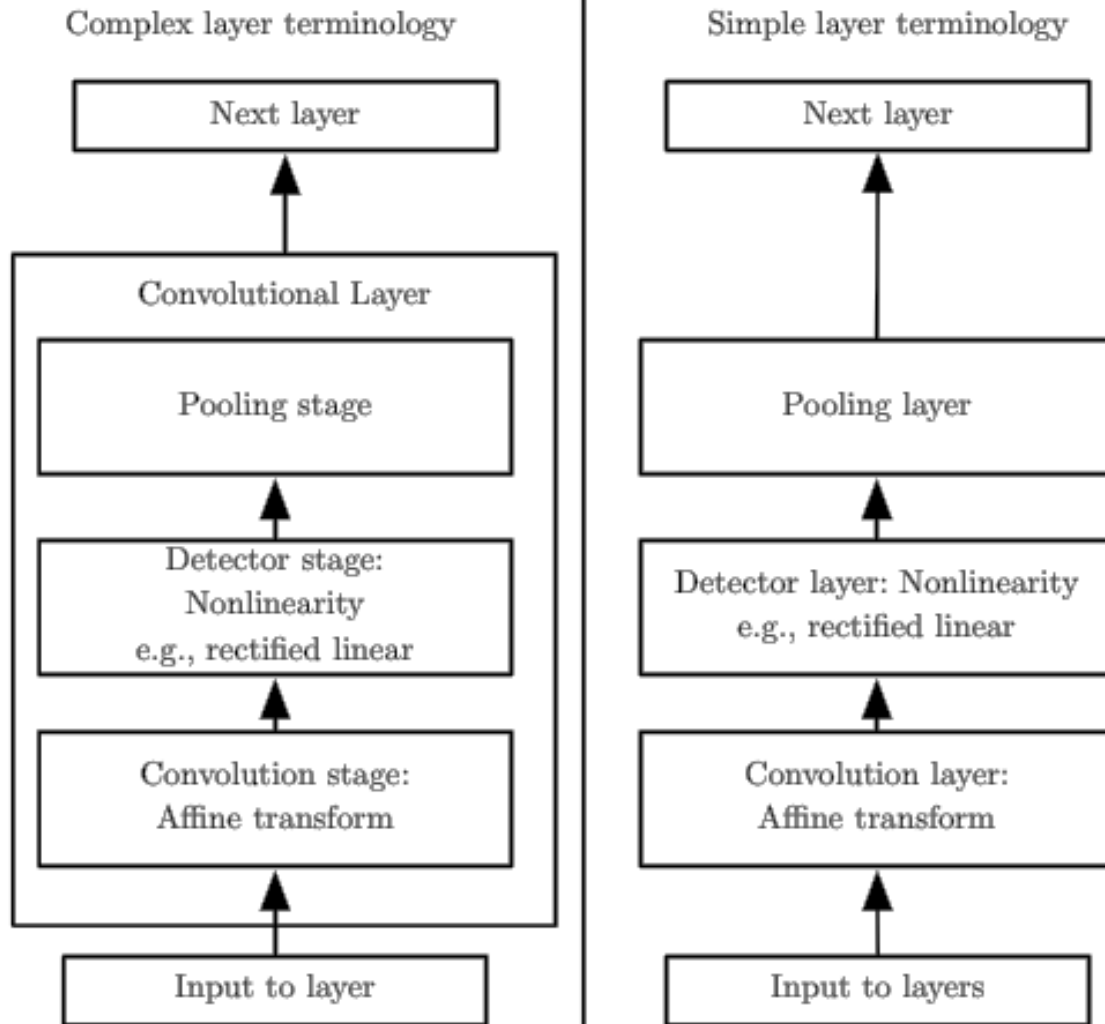
Kernel size: 2 by 1

Output size: 319 by 280



	Convolution	Dense matrix	Sparse matrix
Stored floats	2	$319 \times 280 \times 320 \times 280$ $> 8e9$	$2 \times 319 \times 280 =$ 178,640
Float muls or adds	$319 \times 280 \times 3 =$ 267,960	$> 16e9$	Same as convolution (267,960)

Components of CNN



Pooling

- A pooling function replaces the output of the net with a **summary statistic** of the **nearby outputs**.
- Pooling helps to make the representation become approximately **invariant** to small **translation**, **rotation**, and **size**

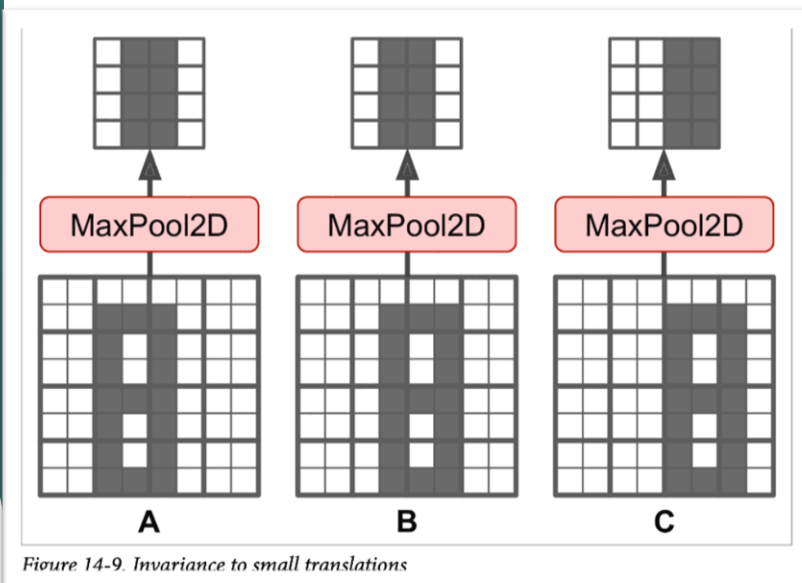
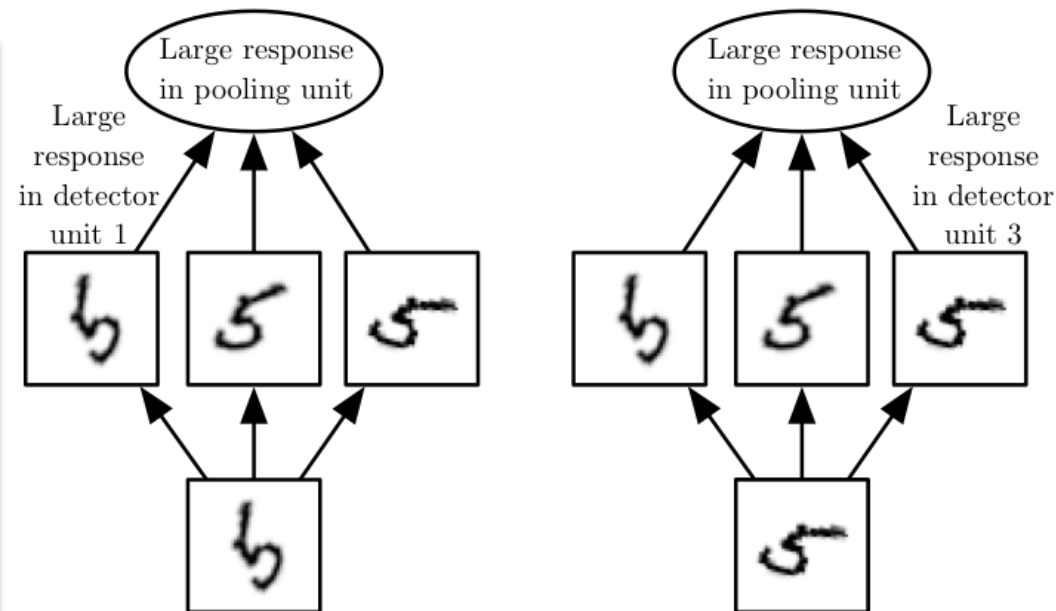


Figure 14-9. Invariance to small translations

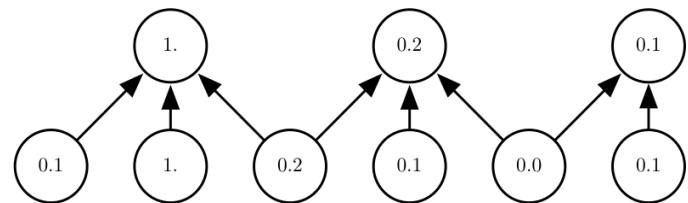


Pooling

- Examples of pooling functions:
 - Maximum within a rectangular neighborhood
 - Average of a rectangular neighborhood
 - L^2 norm of a rectangular neighborhood
 - Weighted average based on the distance from the central pixel
 - LSE (Log-Sum-Exp) Pooling $y = \log \sum_{i=1}^n e^{x_i}$

- Can use fewer pooling units than detector units (pooling with downsampling)

- E.g., pooling width 3, stride 2



- Can help to handle inputs of varying size

Pooling layer

- Need to define its size, the stride, and the padding type
- Aggregate the inputs using an aggregation function such as the **max** or **mean**

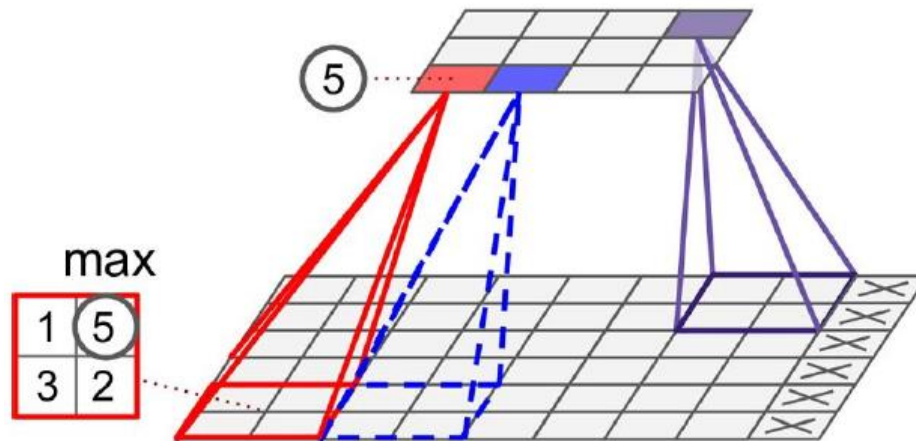
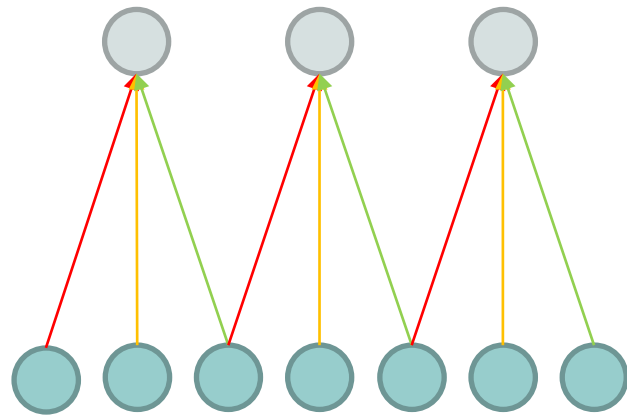


Figure 13-8. Max pooling layer (2×2 pooling kernel, stride 2, no padding)

CNN with Multiple Input Channels

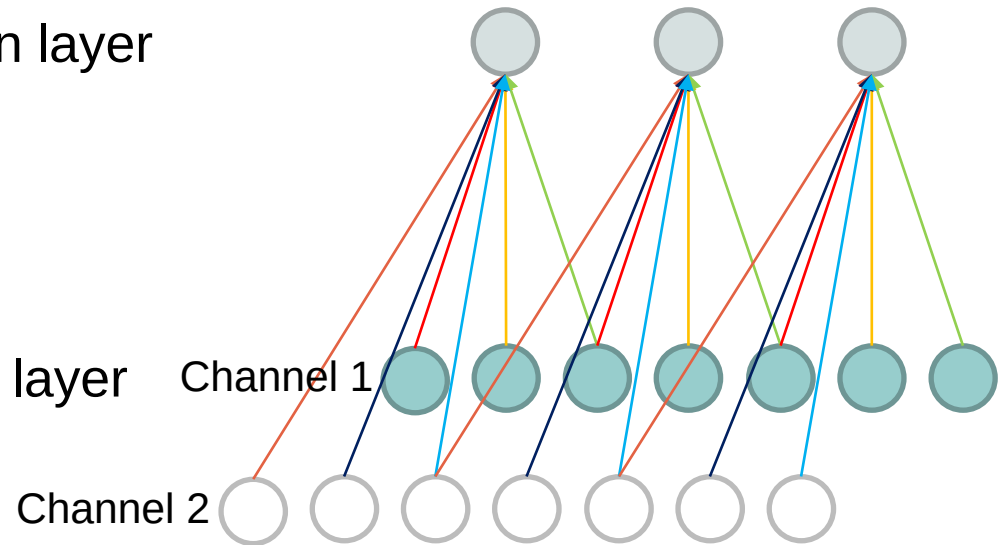


Single input channel

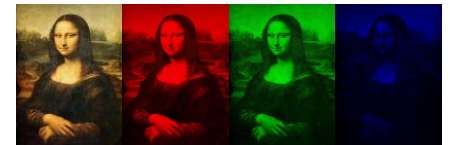
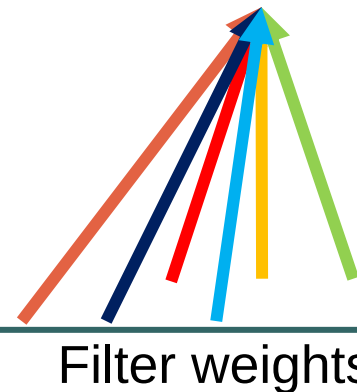


Hidden layer

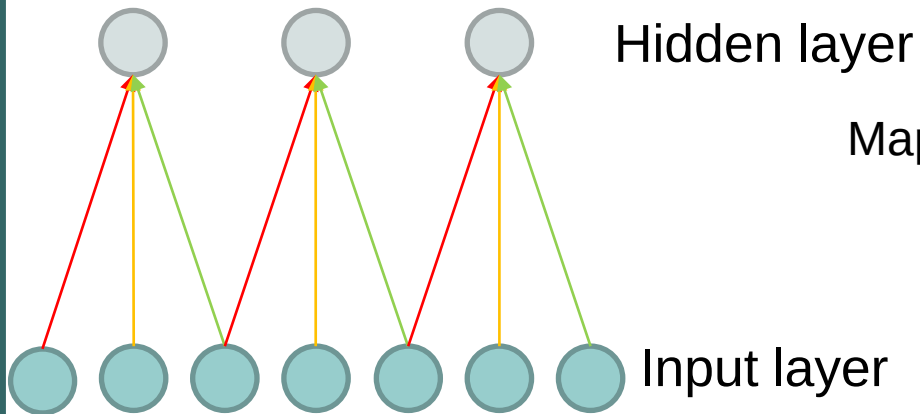
Input layer



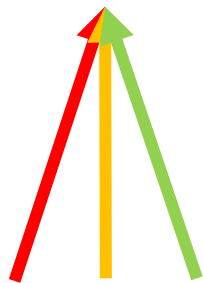
Multiple input channels



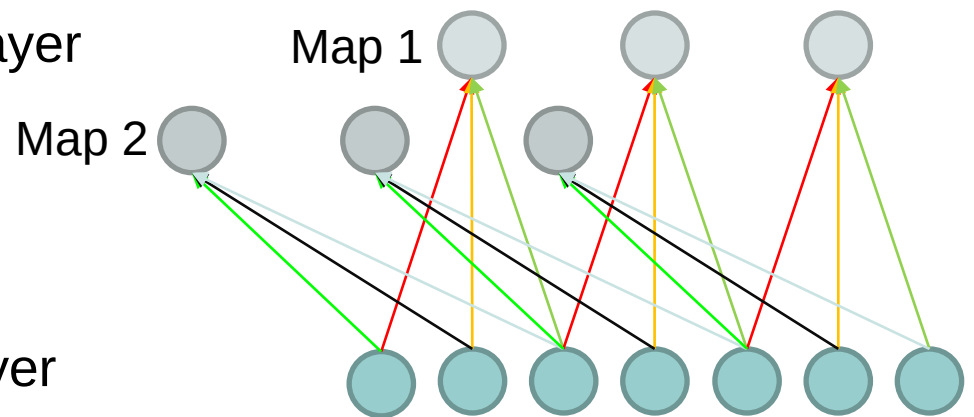
CNN with Multiple Output Maps



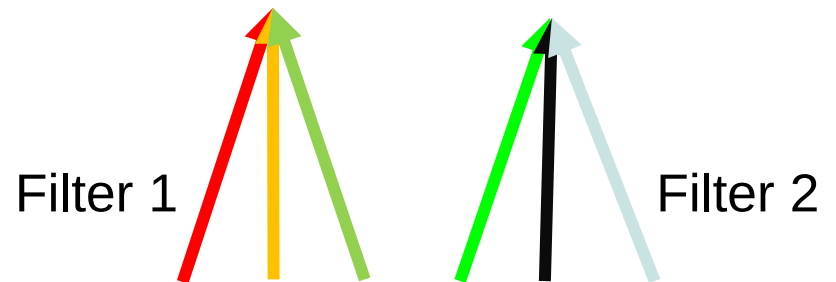
Single output map



Filter weights



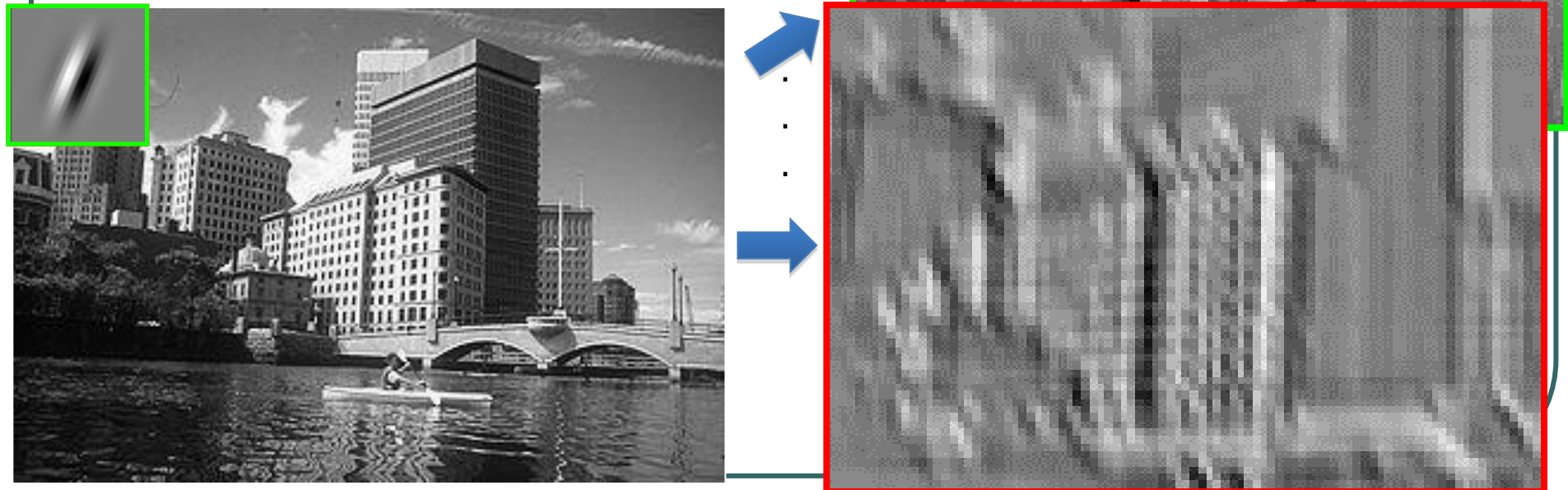
Multiple output maps



Filter weights

Multiple Feature Maps

- Feature activation map for each filter:
 - Weighted moving sum
 - E.g. applying two different filters to get two feature maps



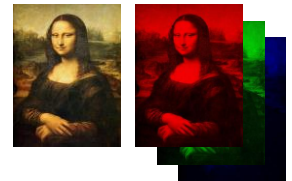
Input

Feature Activation Map

slide credit: S. Lazebnik

Multi-channel Convolution

- For color images, input/output of convolution are 3-D tensors and kernels are 4-D tensors
- Input **V**: $V_{l,j,k}$, channel l at row j , column k
- Output **Z**: $Z_{i,j,k}$, channel i at row j , column k
- Kernel **K**: $K_{i,l,m,n}$, channel i (output), channel l (input)



$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n}$$

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

Stacking Multiple Feature Maps

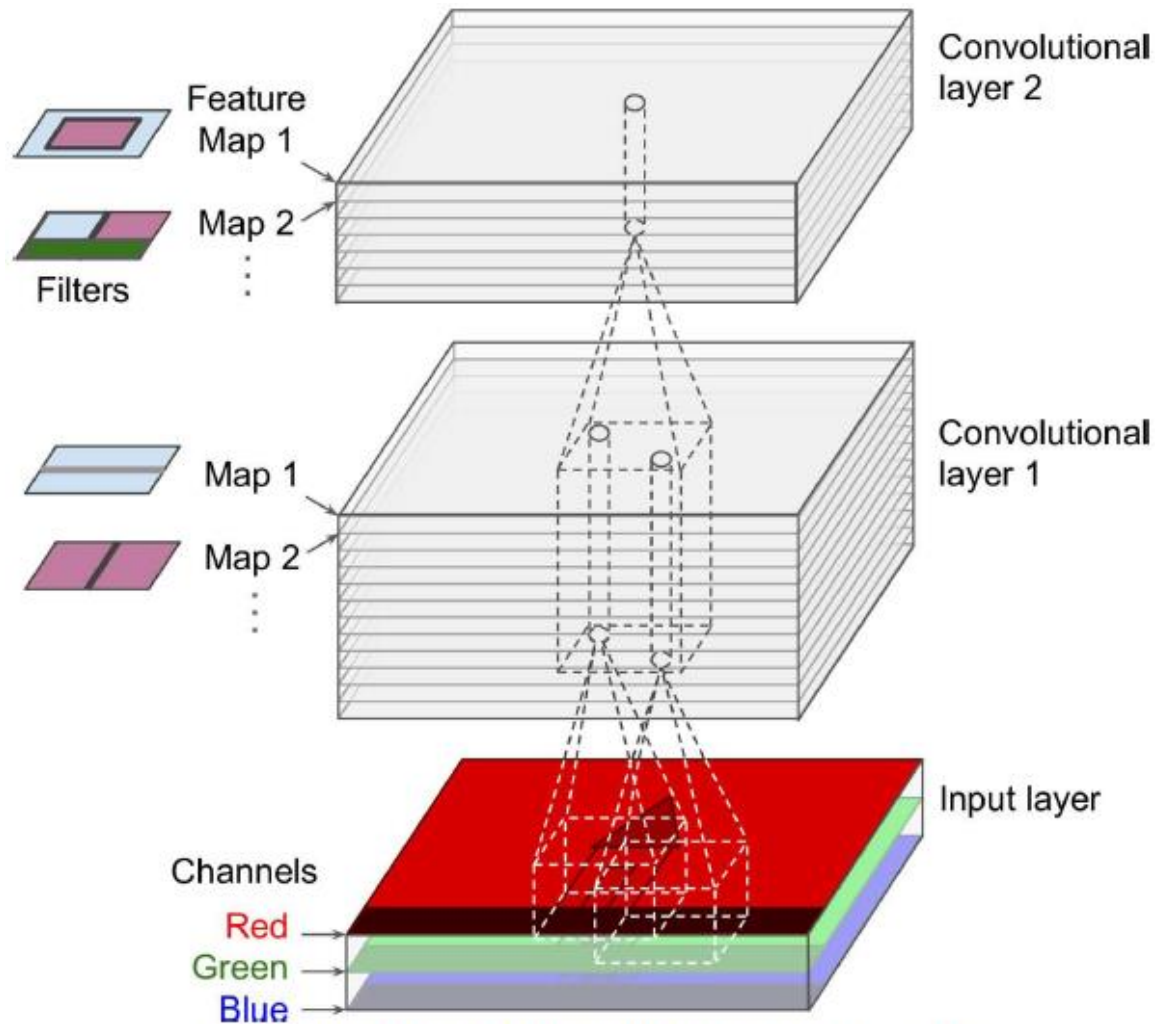
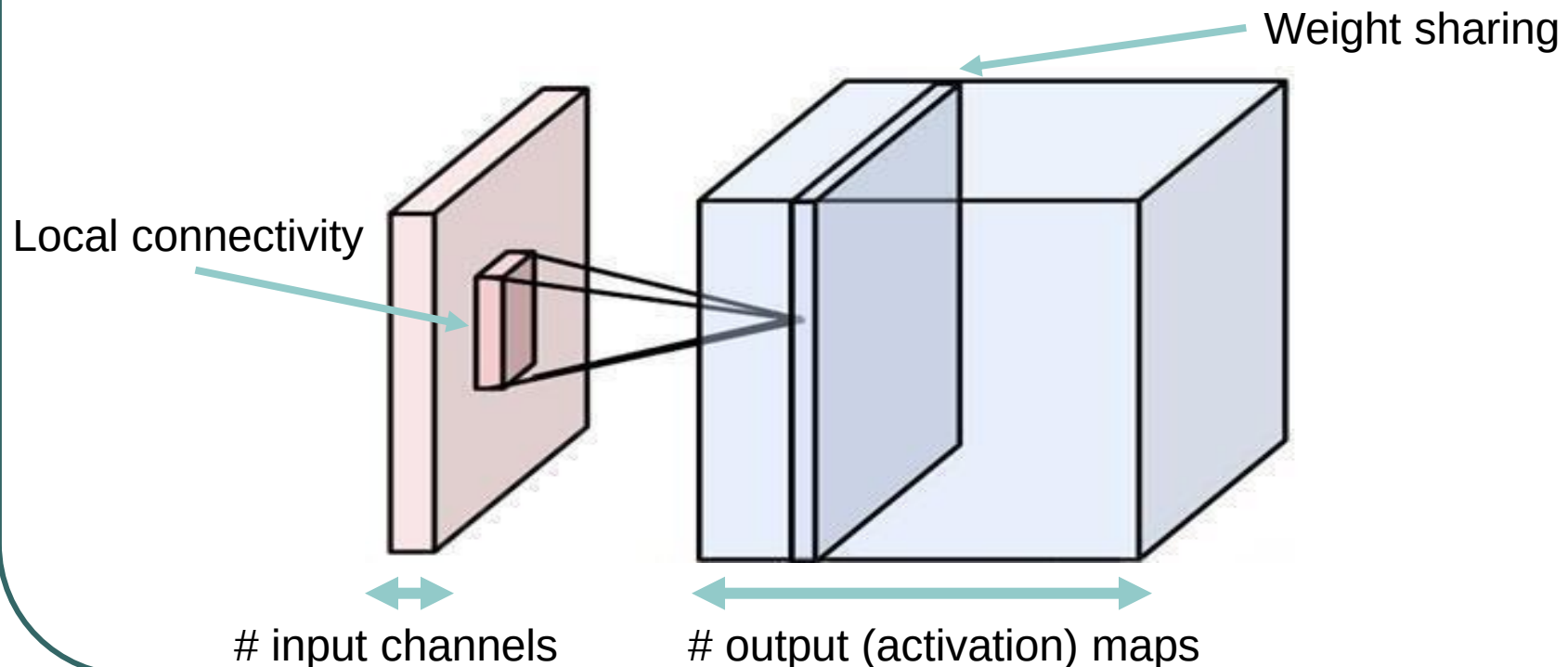


Figure 13-6. Convolution layers with multiple feature maps, and images with three channels

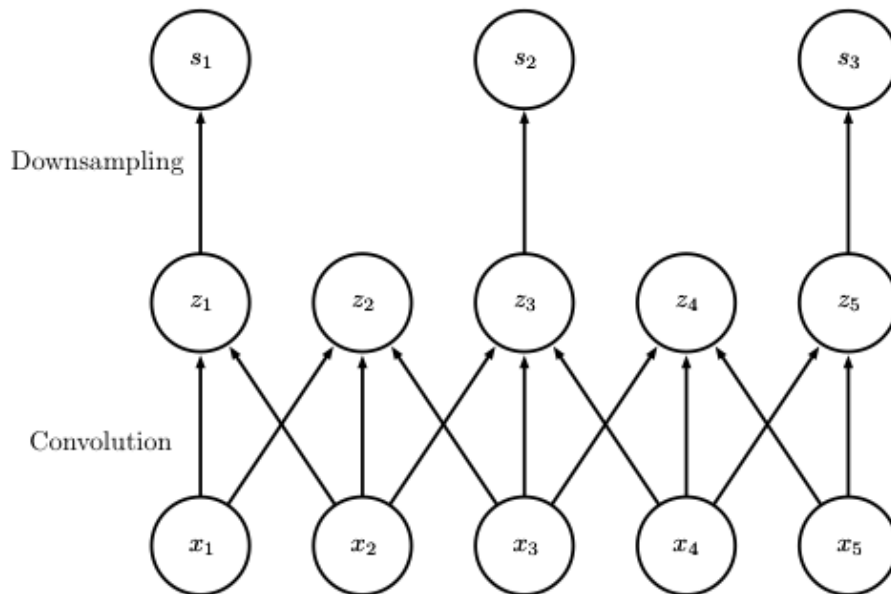
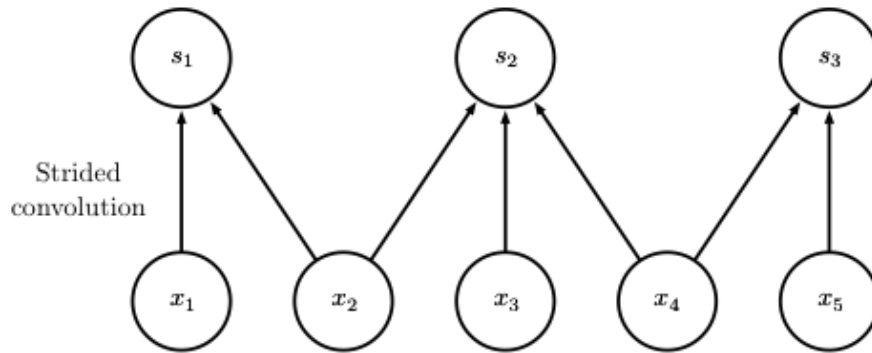
Multi-channel Convolution

- Local connectivity
- Weight sharing
- Handling multiple input channels
- Handling multiple output maps

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n}$$



Convolution with Stride



$$Z_{i,j,k} = c(\mathbf{K}, \mathbf{V}, s)_{i,j,k}$$

$$= \sum_{l,m,n} V_{l,(j-1)s+m,(k-1)s+n} K_{i,l,m,n}$$

s: stride

Zero-padding Controls Size

- No zero-padding

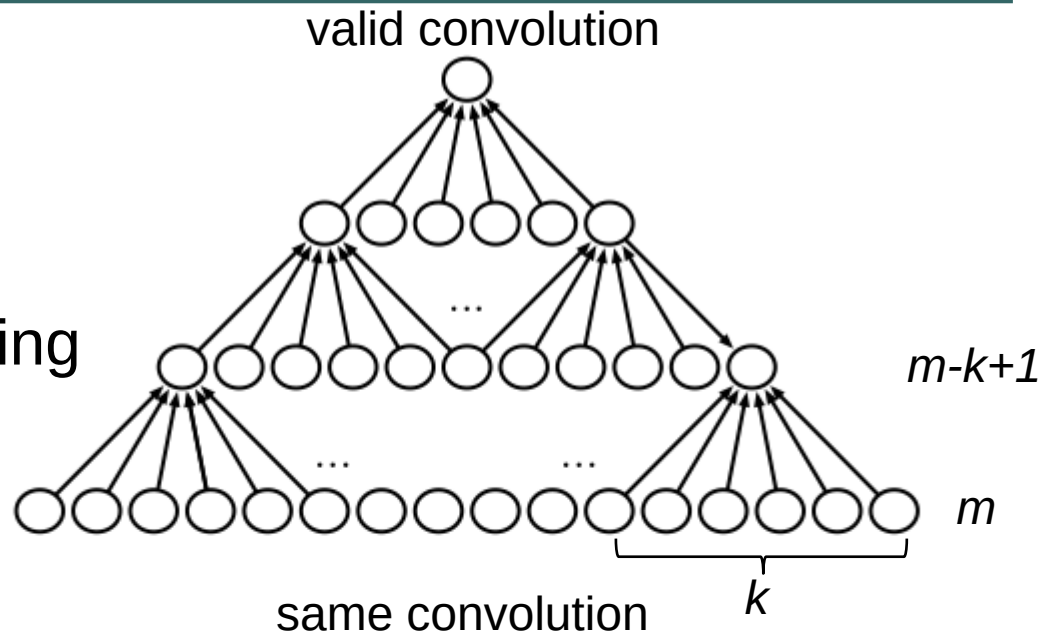
- Valid convolution
- Shrinking size

- Just enough zero-padding

- Same convolution
- Keep the size
- Padding with $k-1$ zeros

- Padding with enough zeros

- Full convolution
- Every pixel is visited k times.



Zero-padding Controls Size

- In order for a layer to have the **same** height and width as the previous layer, it is common to add zeros around the inputs

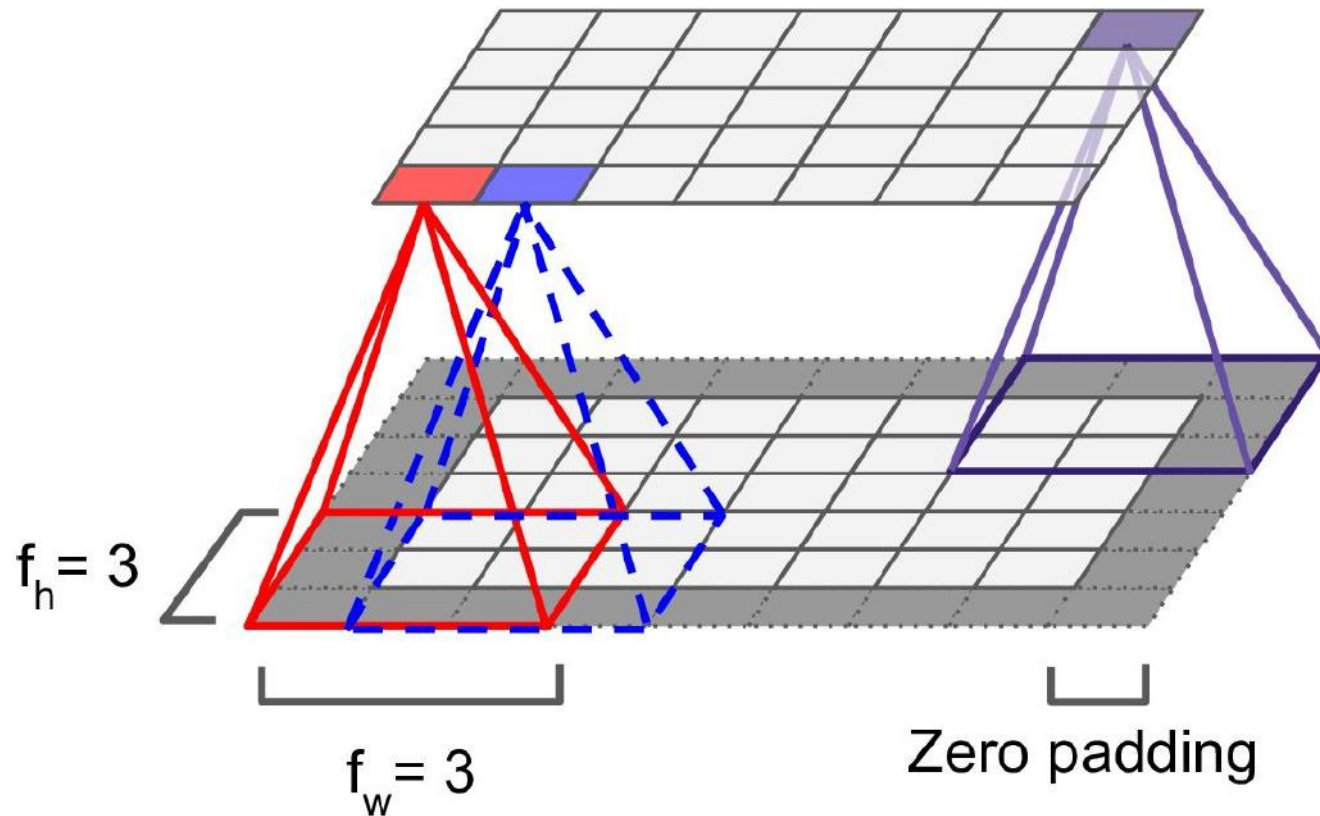


Figure 13-3. Connections between layers and zero padding

Reducing dimensionality using stride

- To connect a large input layer to a much **smaller** layer by spacing out the receptive fields

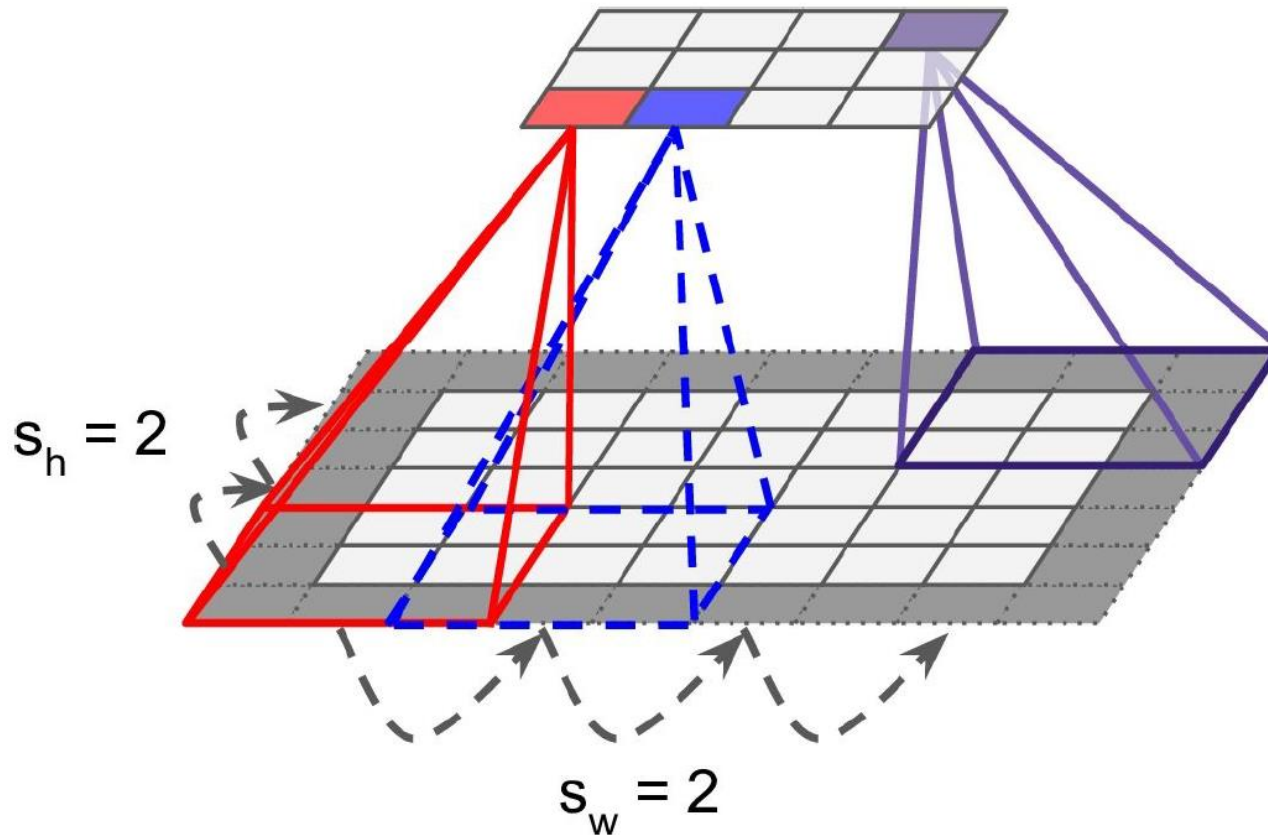
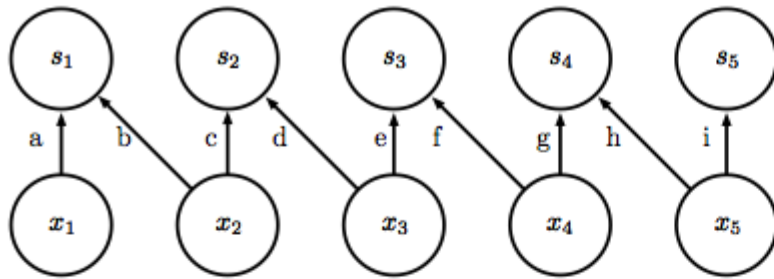


Figure 13-4. Reducing dimensionality using a stride

Local Connections

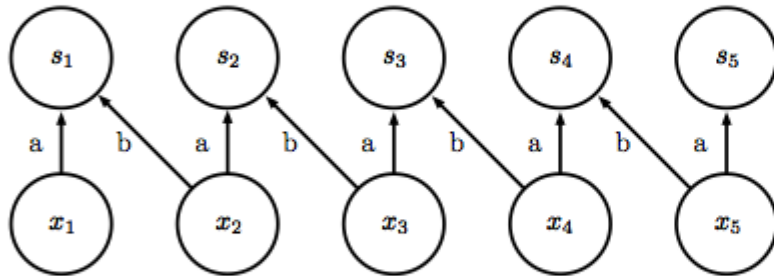


Local connection:
like convolution,
but no sharing

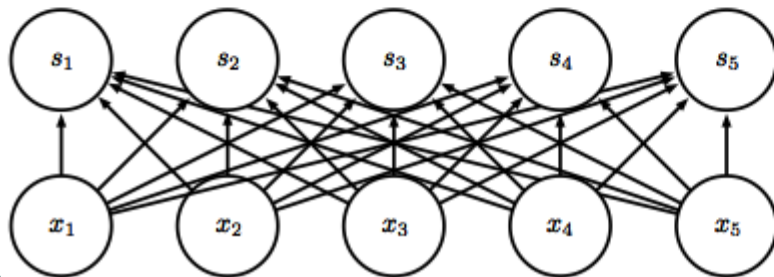
unshared convolution

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,j,k,l,m,n}$$

K: 6D tensor

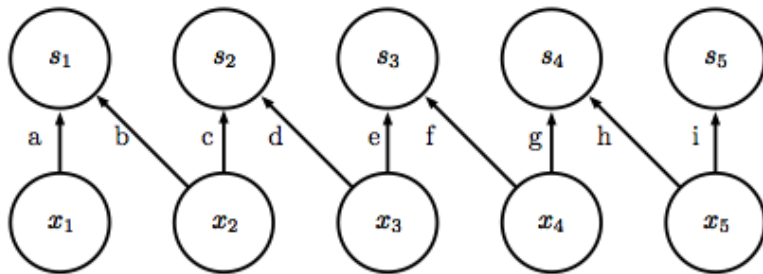


Convolution

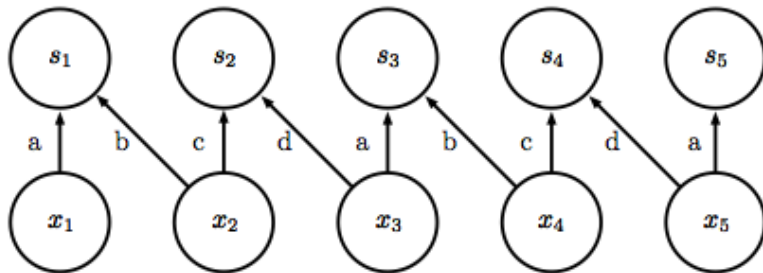


Fully connected

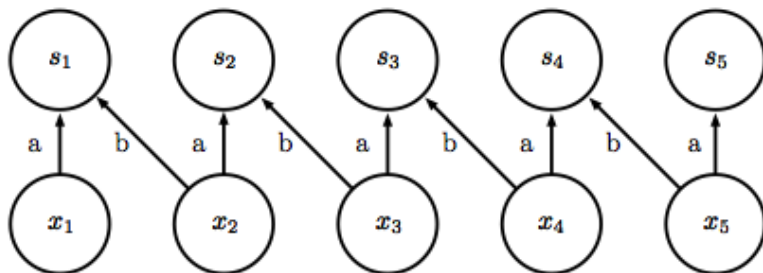
Tiled Convolution



Local connection
(no sharing)



Tiled convolution
(cycle between
groups of shared
parameters)

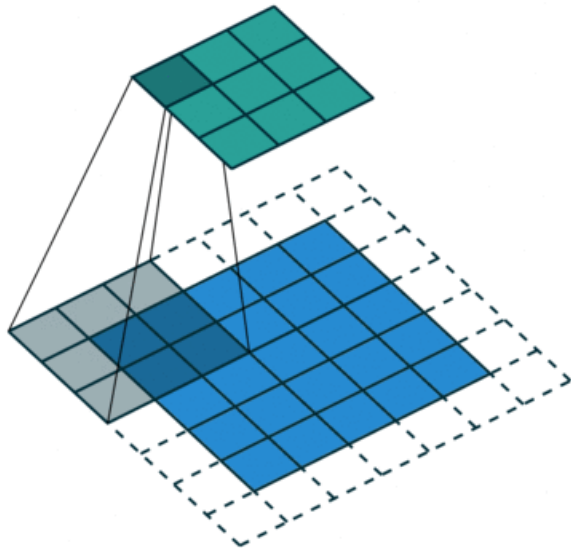


Convolution
(one group shared
everywhere)

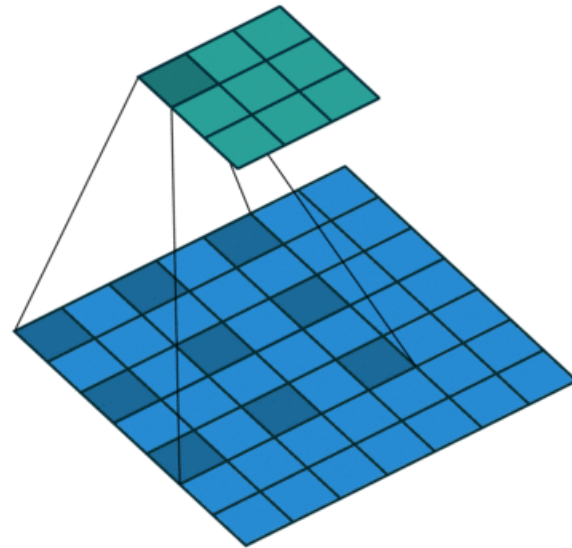
$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,j\%t,k\%t,l,m,n}$$

t : number of kernels

Dilation



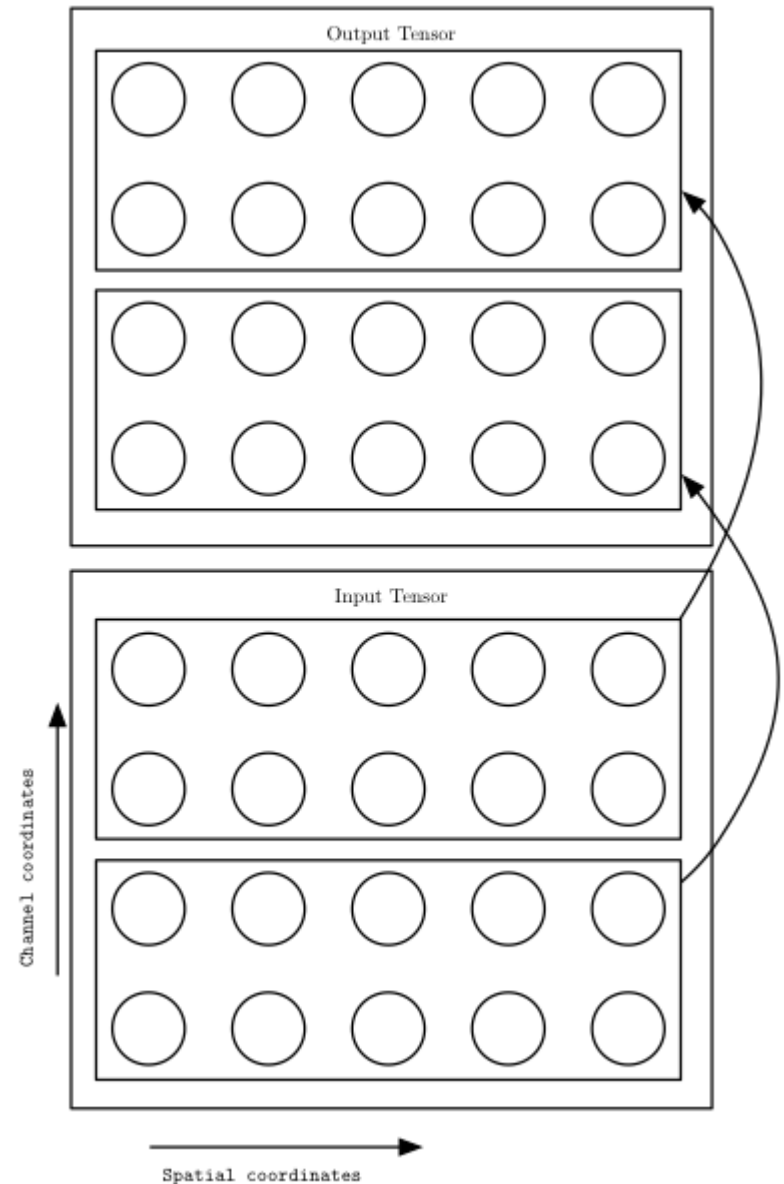
Shared Convolution
with a 3×3 kernel
(add padding)



Dilated Convolution
with a 3×3 kernel
and dilation rate 2

Partial Connectivity Between Channels

- Each output channel i is a function of only a subset of input channels.



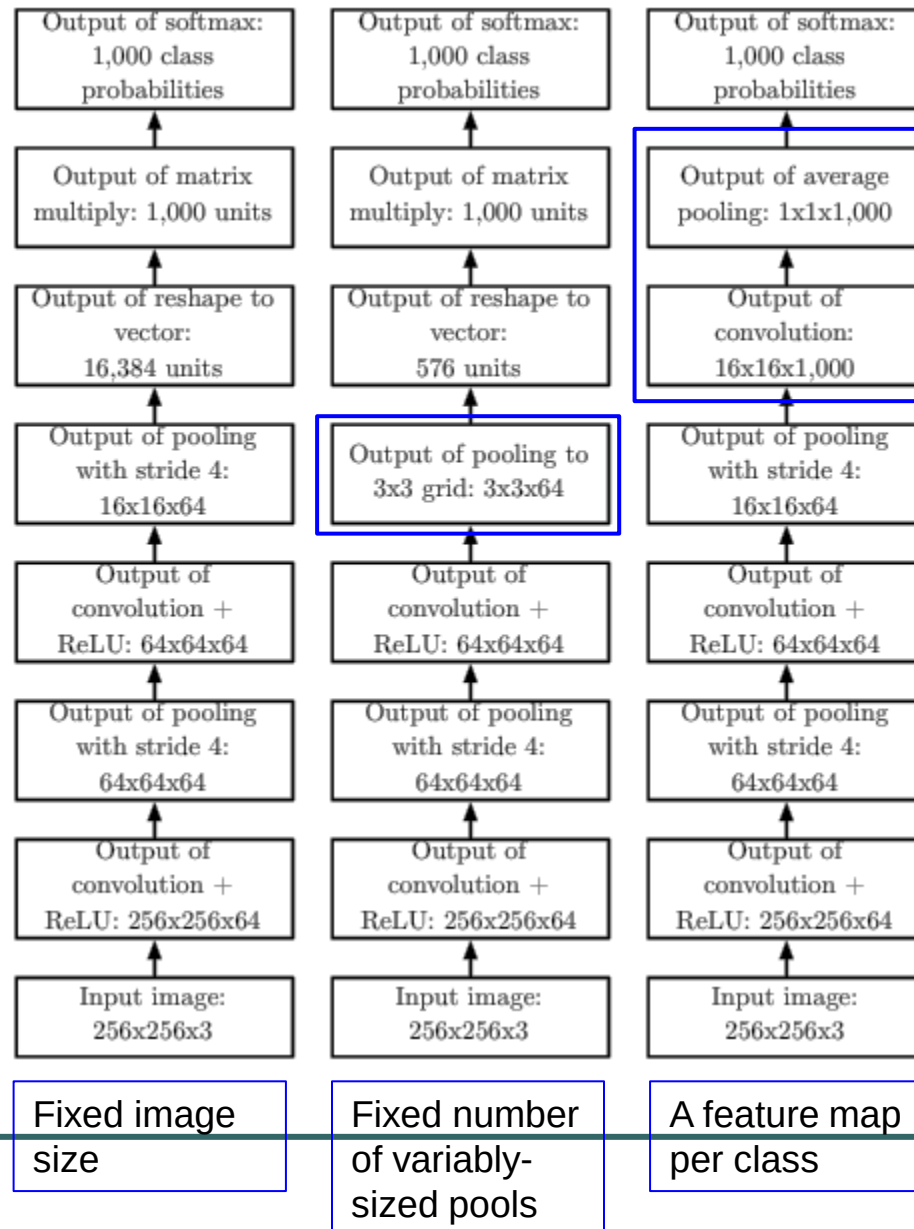
Efficient Convolution Algorithms

- A d -dimensional kernel is **separable** if it can be expressed as the outer product of d vectors.
- Convolution with a d -dimensional kernel can be decomposed into d one-dimensional convolutions.
- Complexity: $O(w^d) \rightarrow O(wd)$

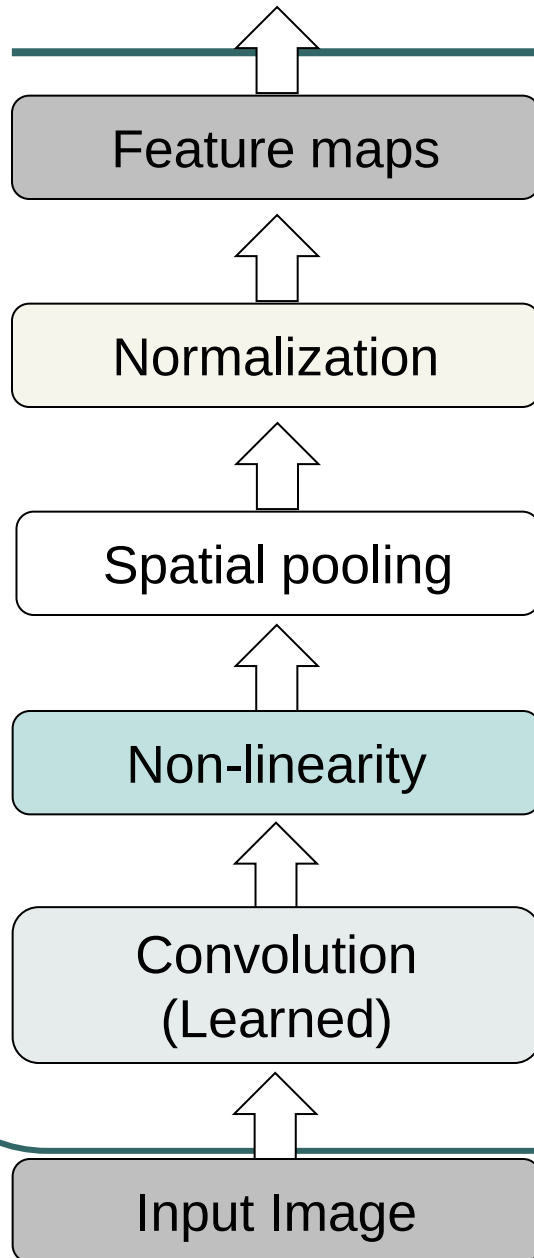
Initialization of Kernels

- Randomly initialized
 - Random filters often work surprisingly well.
- Designed by hand
 - Edge detection, Haar wavelet, etc.
- Learned with an unsupervised criterion
 - K-means clustering

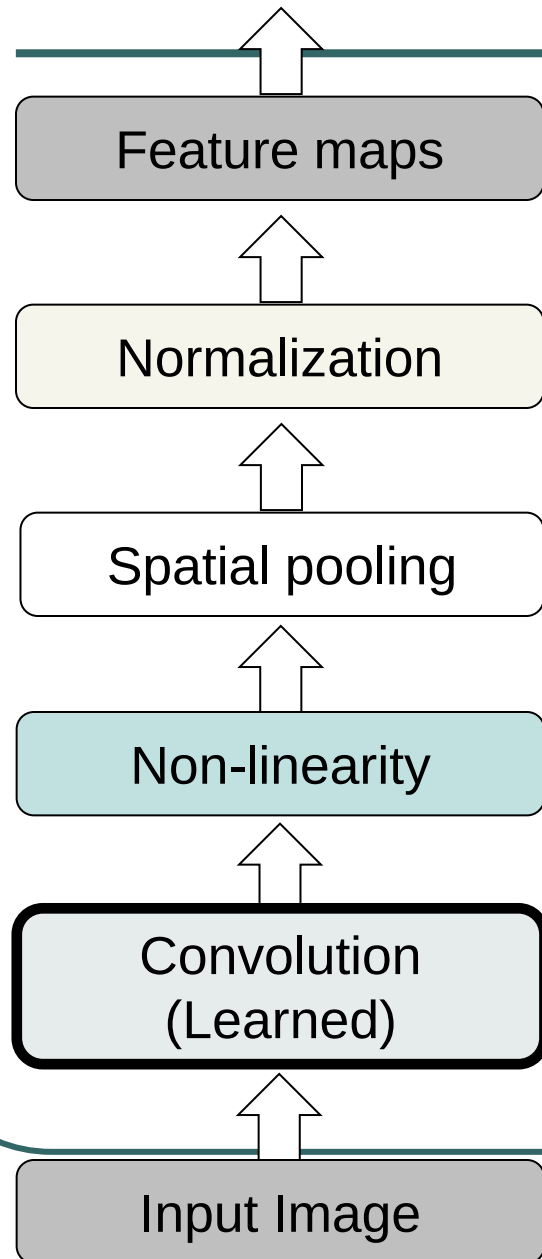
Example Classification Architectures



Convolutional Neural Networks



Convolutional Neural Networks

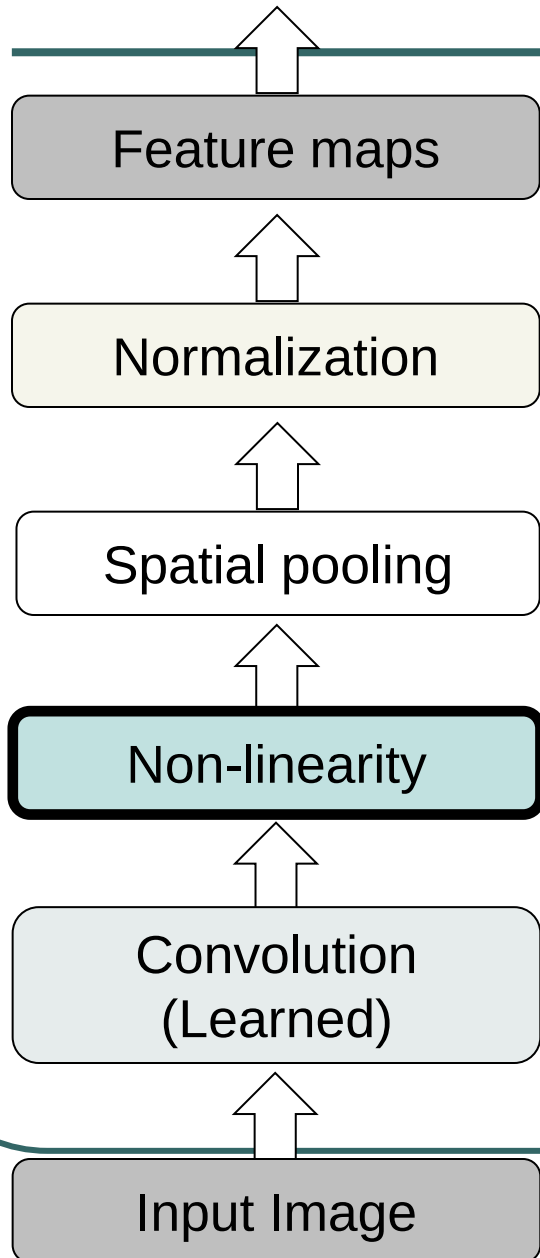


Input

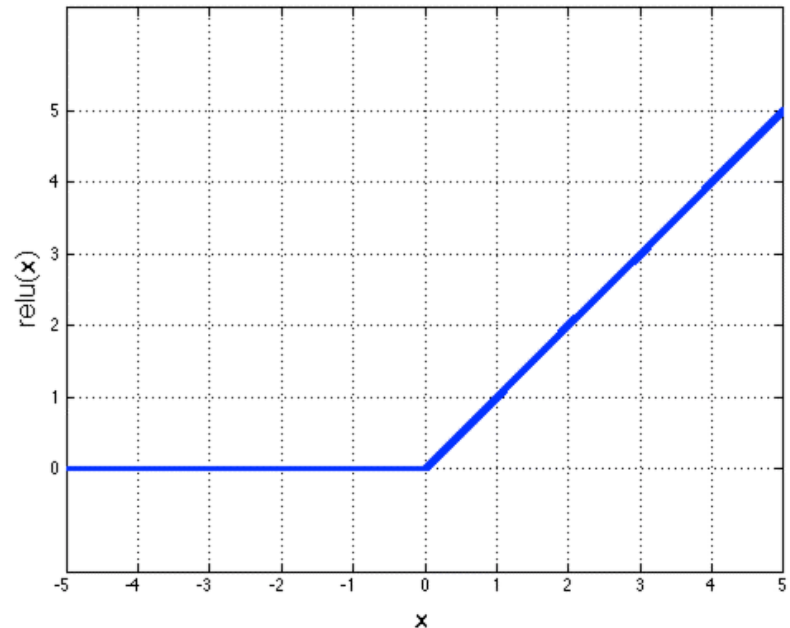


Feature Map

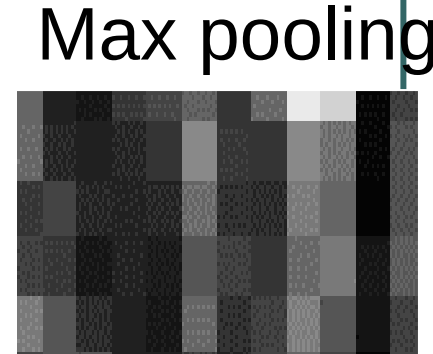
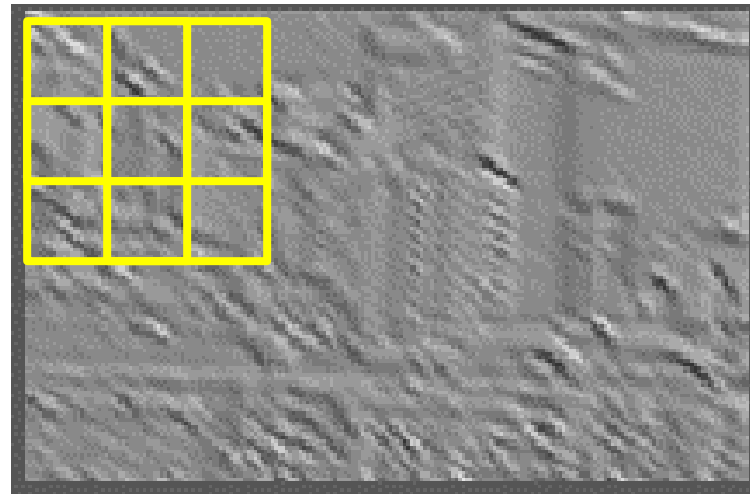
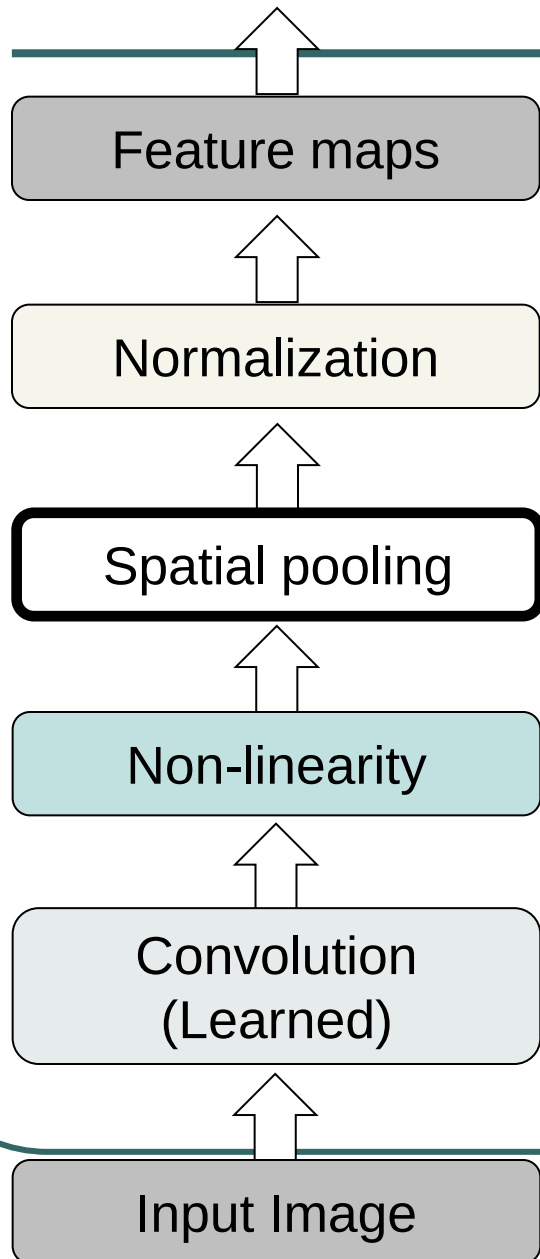
Convolutional Neural Networks



Rectified Linear Unit (ReLU)



Convolutional Neural Networks

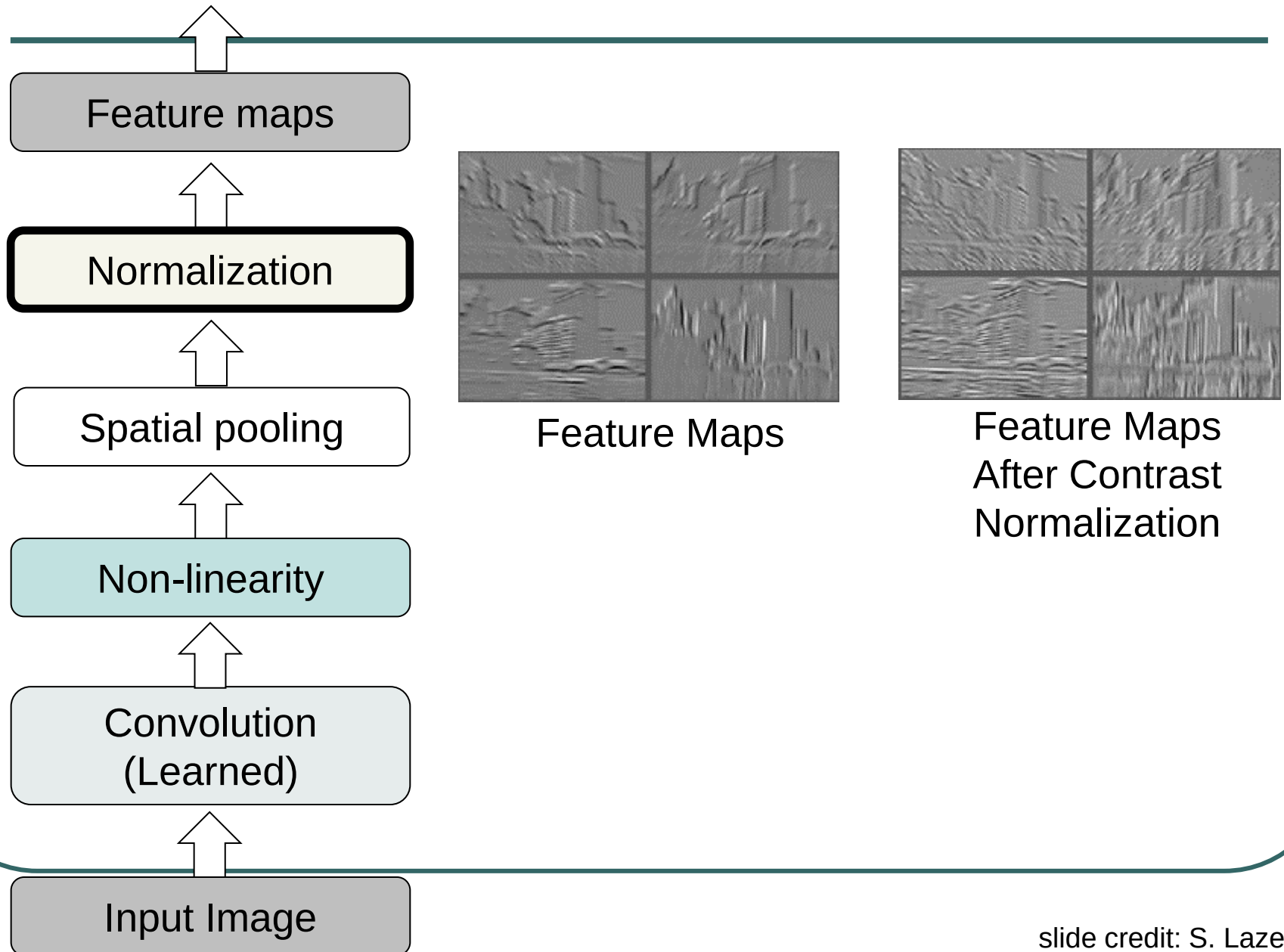


Max pooling

Max-pooling: a non-linear down-sampling

Provide *translation invariance*

Convolutional Neural Networks



Typical CNN Architectures

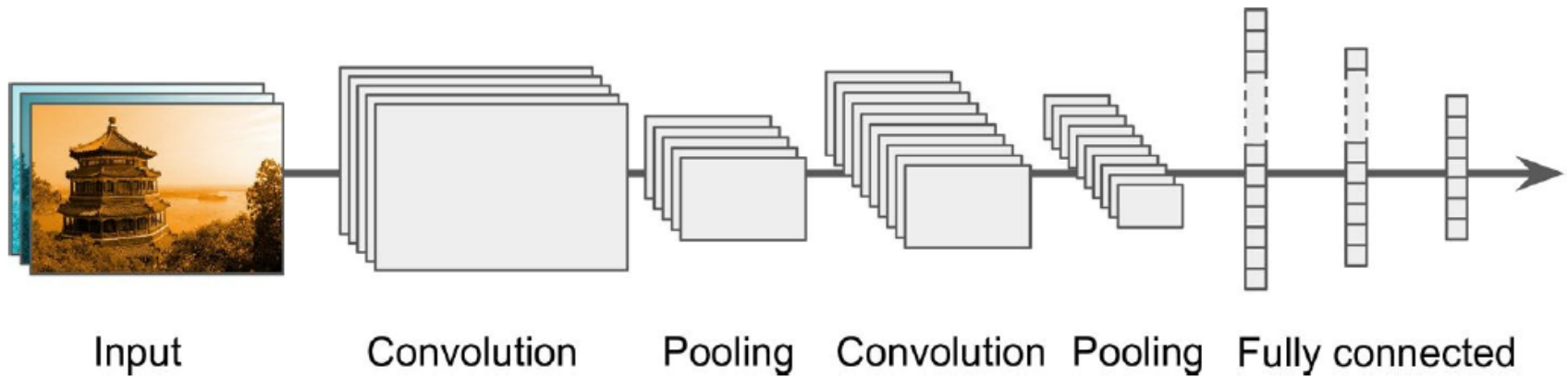
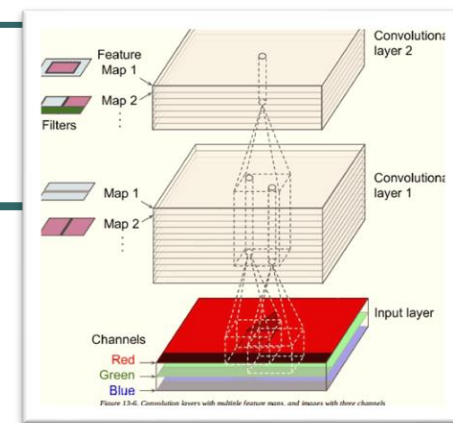


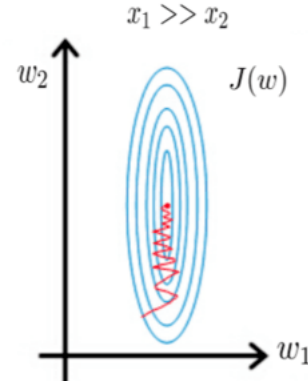
Figure 13-9. Typical CNN architecture

- A common **mistake** is to use convolution **kernels** that are **too large**. You can often get the same effect as a 9×9 kernel by stacking two 3×3 kernels on top of each other, for a lot less compute.

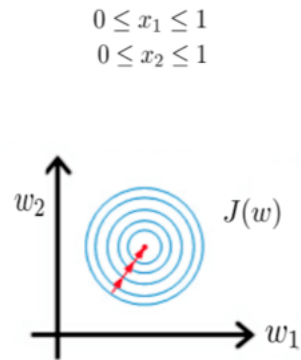
Normalization

- To improve the speed, performance, and stability of artificial neural networks.

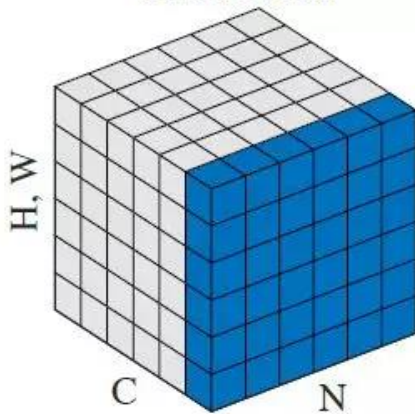
Gradient descent
without scaling



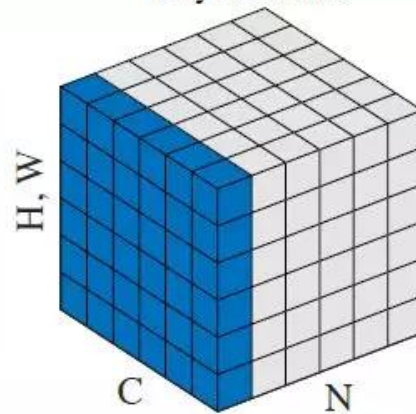
Gradient descent
after scaling variables



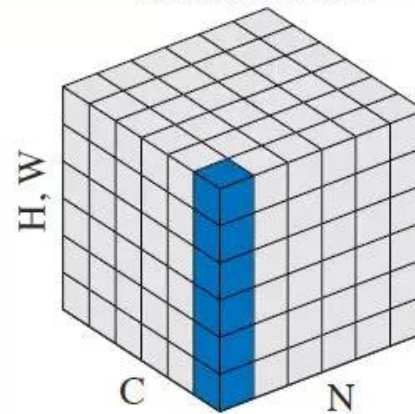
Batch Norm



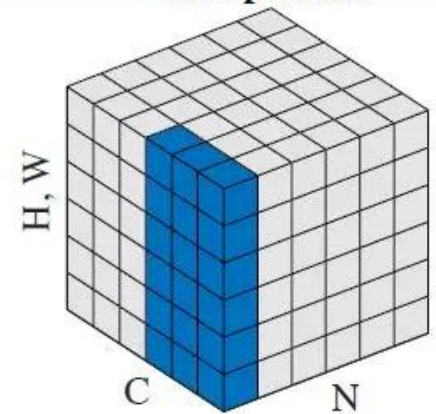
Layer Norm



Instance Norm



Group Norm

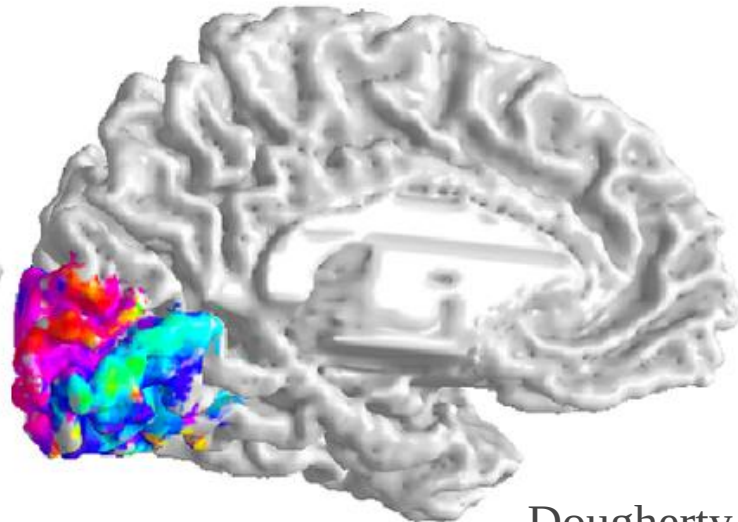
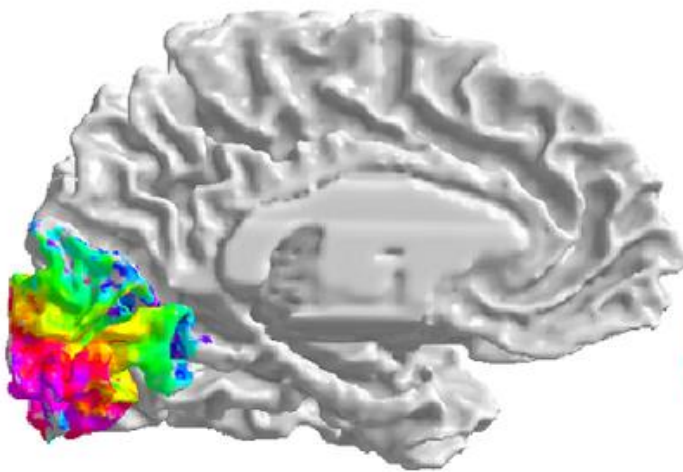
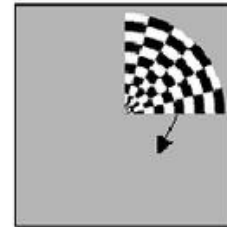
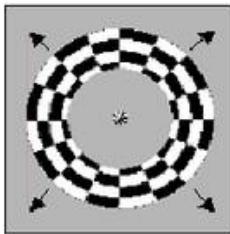


$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \text{ and } \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i^{(k)} = \frac{x_i^{(k)} - \mu_B^{(k)}}{\sqrt{\sigma_B^{(k)2} + \epsilon}}, \text{ where } k \in [1, d] \text{ and } i \in [1, m]$$

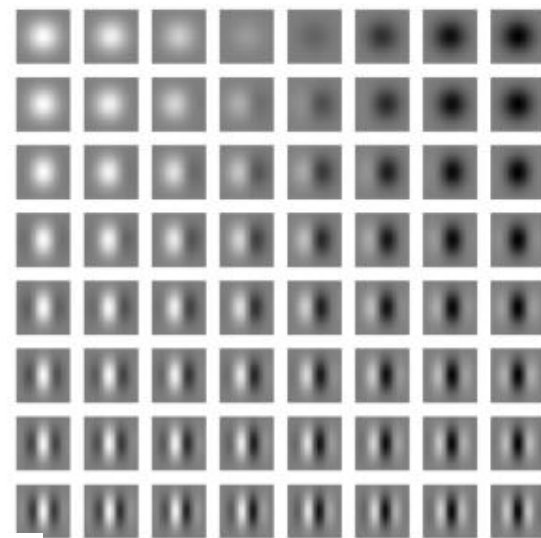
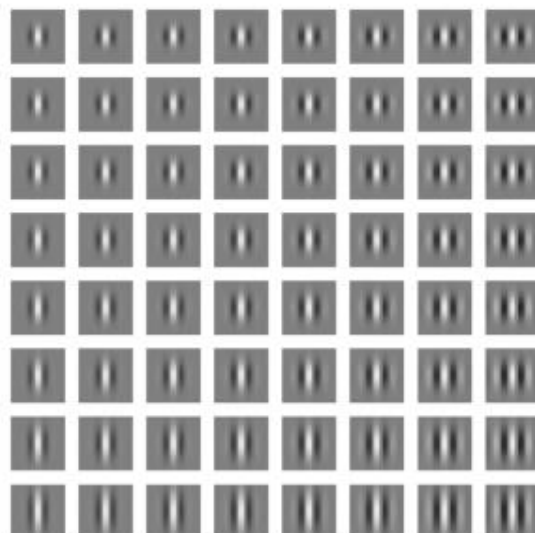
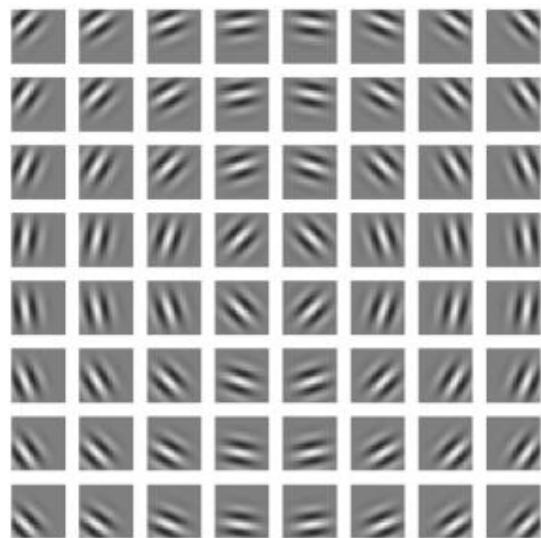
Retinotopy

- V1 is arranged in a spatial map called **retinotopy**.



Dougherty et al., 2003

Gabor Model for the Receptive Field of V1 Cell



$$s(I) = \sum_{x \in \mathbb{X}} \sum_{y \in \mathbb{Y}} w(x, y) I(x, y)$$

$$w(x, y; \alpha, \beta_x, \beta_y, f, \phi, x_0, y_0, \tau) = \alpha \exp(-\beta_x x'^2 - \beta_y y'^2) \cos(fx' + \phi),$$

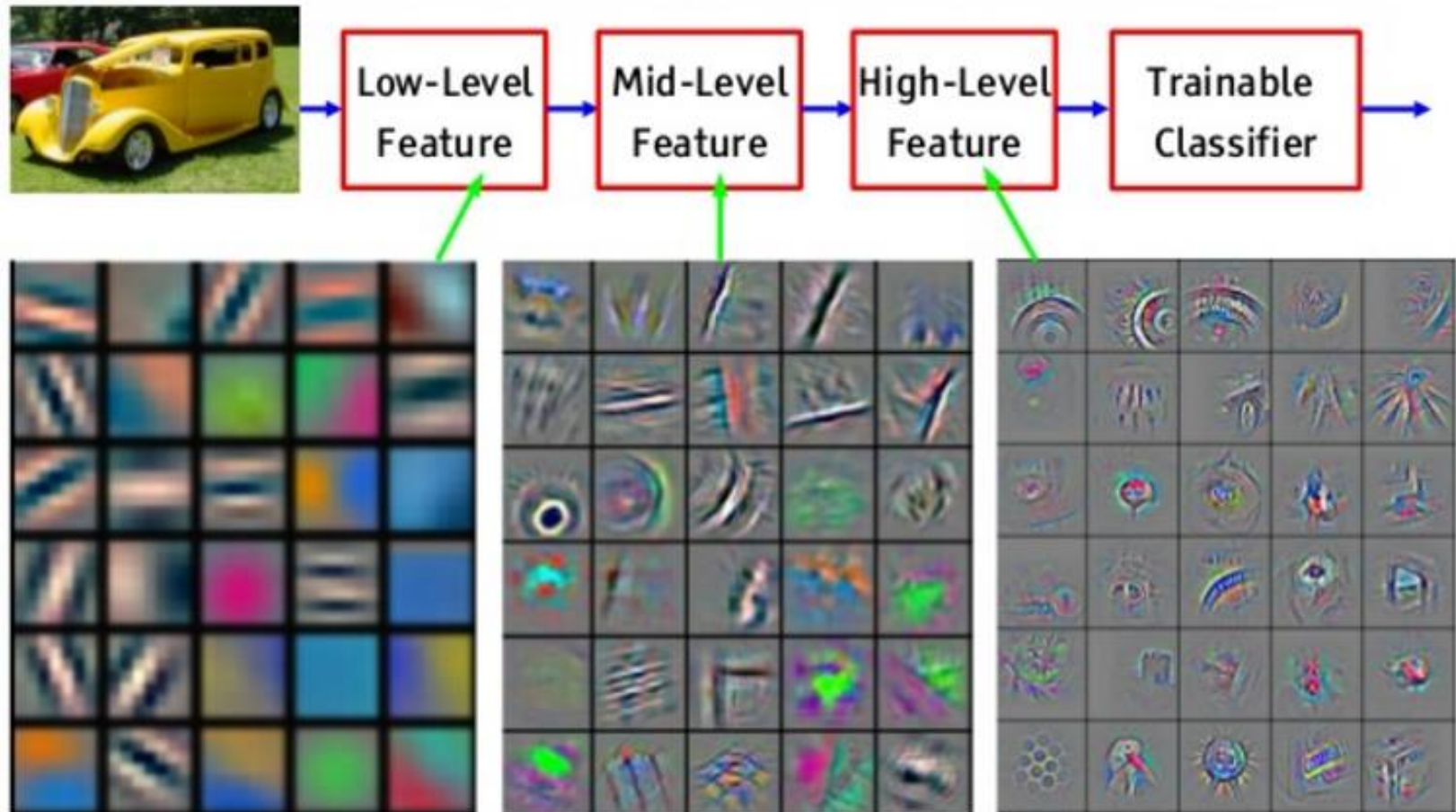
where

$$x' = (x - x_0) \cos(\tau) + (y - y_0) \sin(\tau)$$

$$y' = -(x - x_0) \sin(\tau) + (y - y_0) \cos(\tau)$$

Gabor-like Learned Kernels

- CNN:



adapted from Jeff Dean presentation 2016

Neocognitron [Fukushima, Biological Cybernetics 1980s]

V1 contains many simple cells and complex cells [Hubel & Wiesel, 1959].

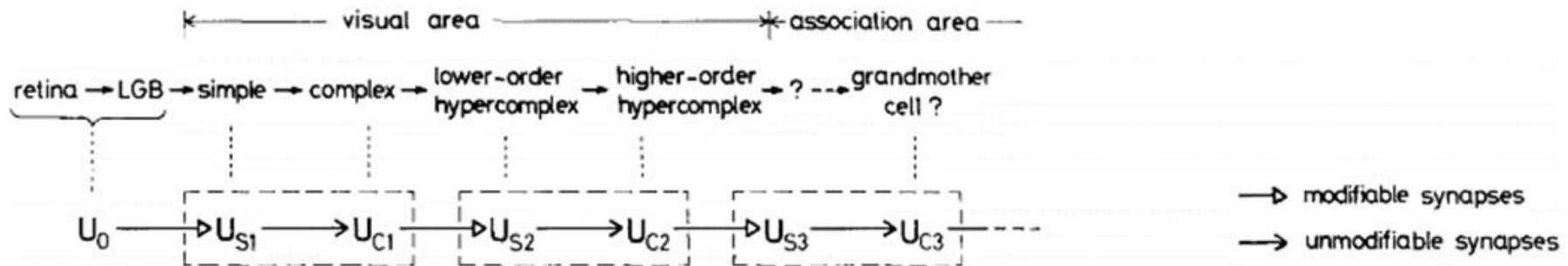
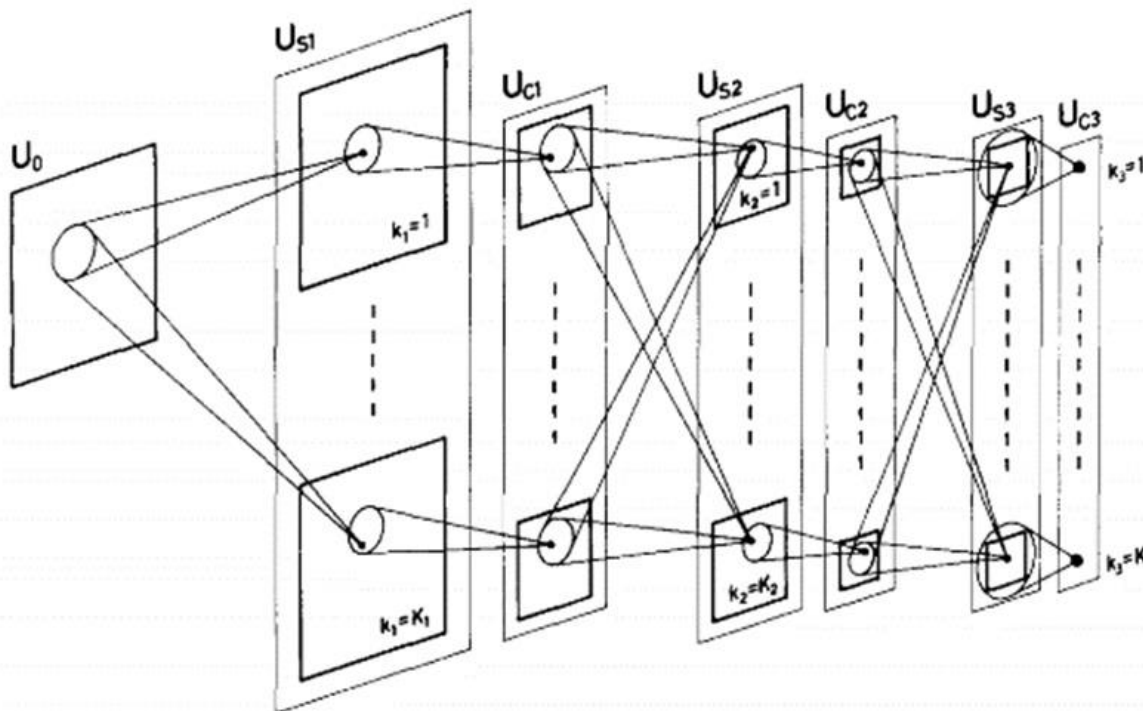


Fig. 1. Correspondence between the hierarchy model by Hubel and Wiesel, and the neural network of the neocognitron



Deformation-Resistant
Recognition

S-cells: (simple)
- extract local features

C-cells: (complex)
- allow for positional errors

LeNet-5 (LeCun et al. 1998]

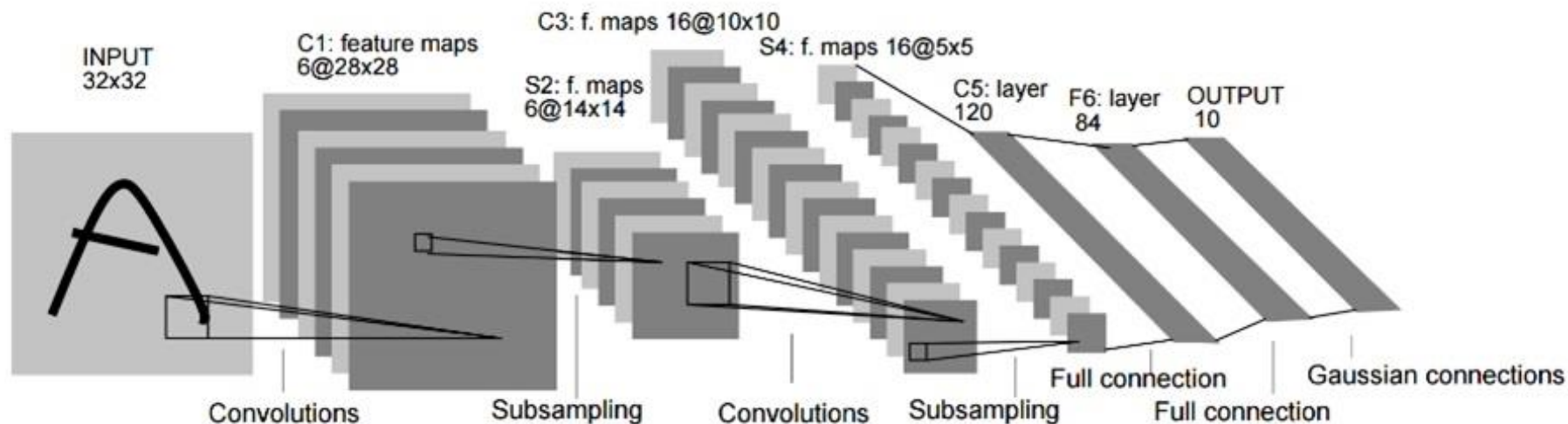


Table 14-1. LeNet-5 architecture

Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully Connected	—	10	—	—	RBF
F6	Fully Connected	—	84	—	—	tanh
C5	Convolution	120	1 × 1	5 × 5	1	tanh
S4	Avg Pooling	16	5 × 5	2 × 2	2	tanh
C3	Convolution	16	10 × 10	5 × 5	1	tanh
S2	Avg Pooling	6	14 × 14	2 × 2	2	tanh
C1	Convolution	6	28 × 28	5 × 5	1	tanh
In	Input	1	32 × 32	—	—	—



Gradient-based learning applied to document recognition
[LeCun, Bottou, Bengio, Haffner 1998]

LeNet-1 from 1993

IMAGENET

- Start in 2007 at Princeton
- Debut in CVPR'2009
 - Total classes : 21.8K
 - Total images : 14M
- ILSVR Challenge: 2010 - 2017



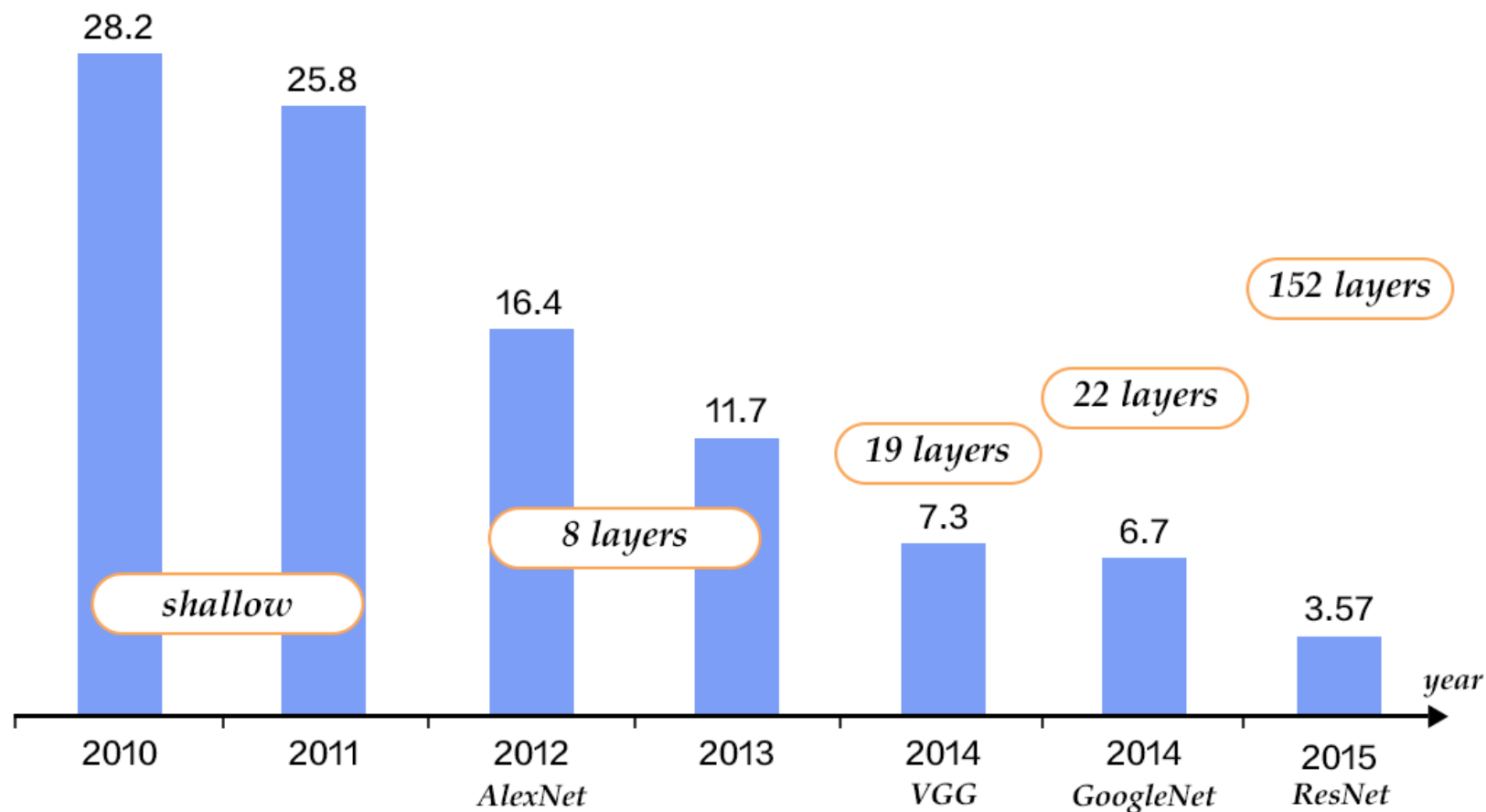
Jia Deng



Fei-Fei Li

ImageNet Challenge 2012-2014

Best non-convnet in 2012: 26.2%

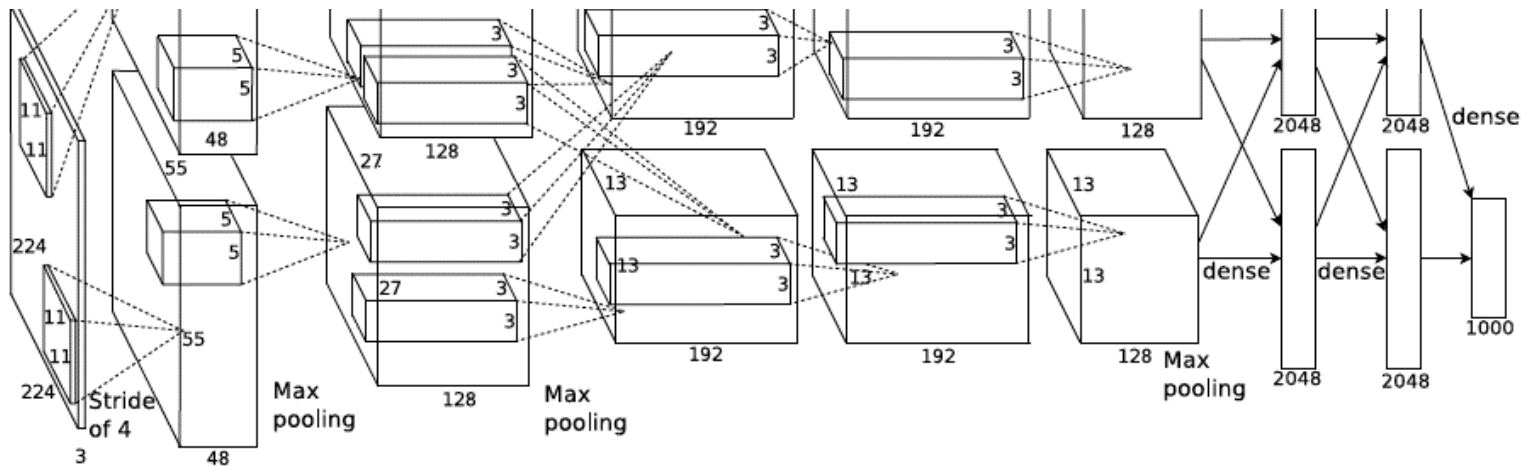


AlexNet

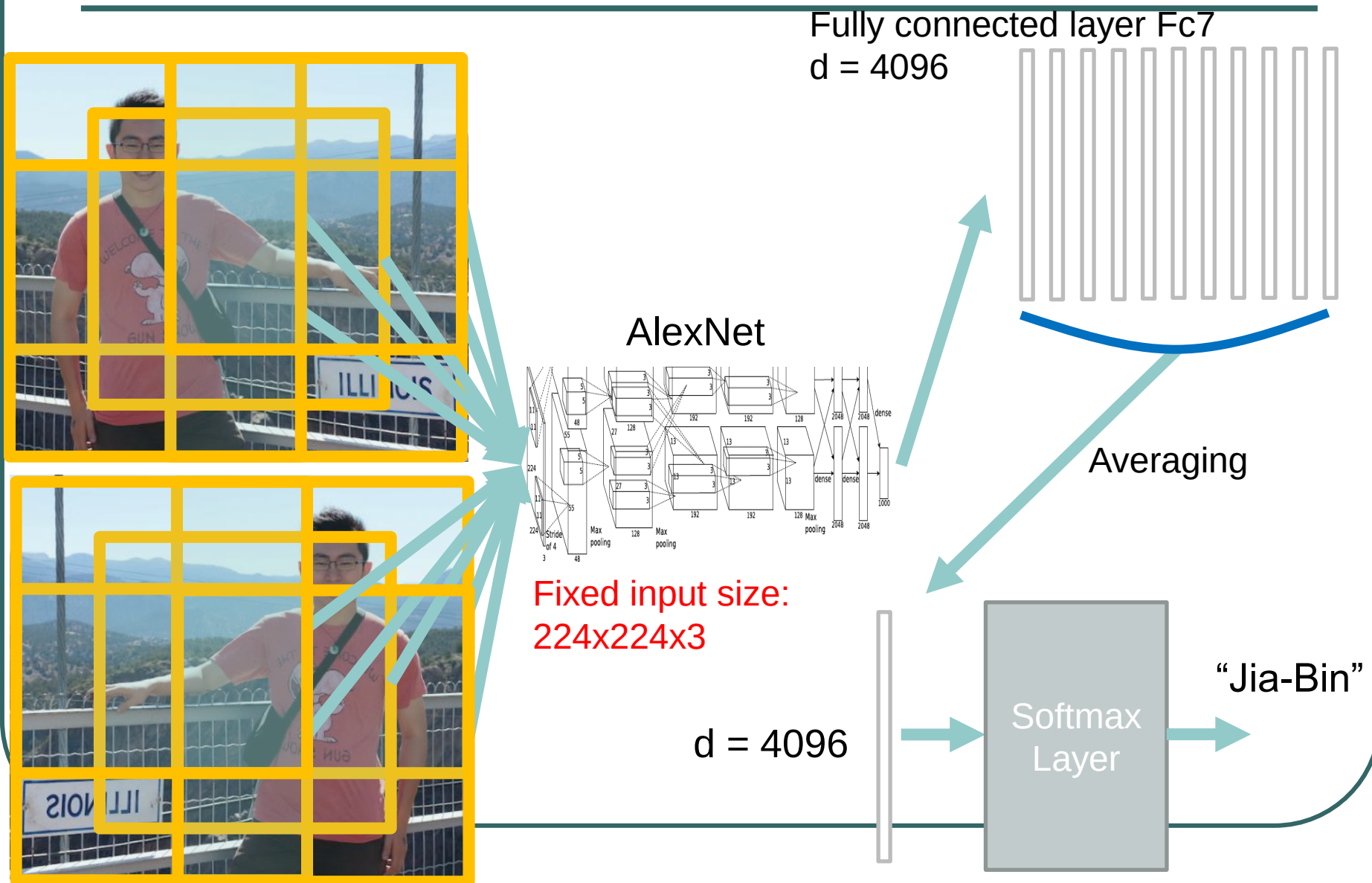
- Similar framework to LeCun'98 but:
 - Bigger model (7 hidden layers, 650,000 units, 60,000,000 params)
 - More data (10^6 vs. 10^3 images): data augmentation
 - GPU implementation (50x speedup over CPU)
 - Trained on two GPUs for a week

Table 14-2. AlexNet architecture

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully Connected	—	1,000	—	—	—	Softmax
F9	Fully Connected	—	4,096	—	—	—	ReLU
F8	Fully Connected	—	4,096	—	—	—	ReLU
C7	Convolution	256	13×13	3×3	1	SAME	ReLU
C6	Convolution	384	13×13	3×3	1	SAME	ReLU
C5	Convolution	384	13×13	3×3	1	SAME	ReLU
S4	Max Pooling	256	13×13	3×3	2	VALID	—
C3	Convolution	256	27×27	5×5	1	SAME	ReLU
S2	Max Pooling	96	27×27	3×3	2	VALID	—
C1	Convolution	96	55×55	11×11	4	VALID	ReLU
In	Input	3 (RGB)	227×227	—	—	—	—

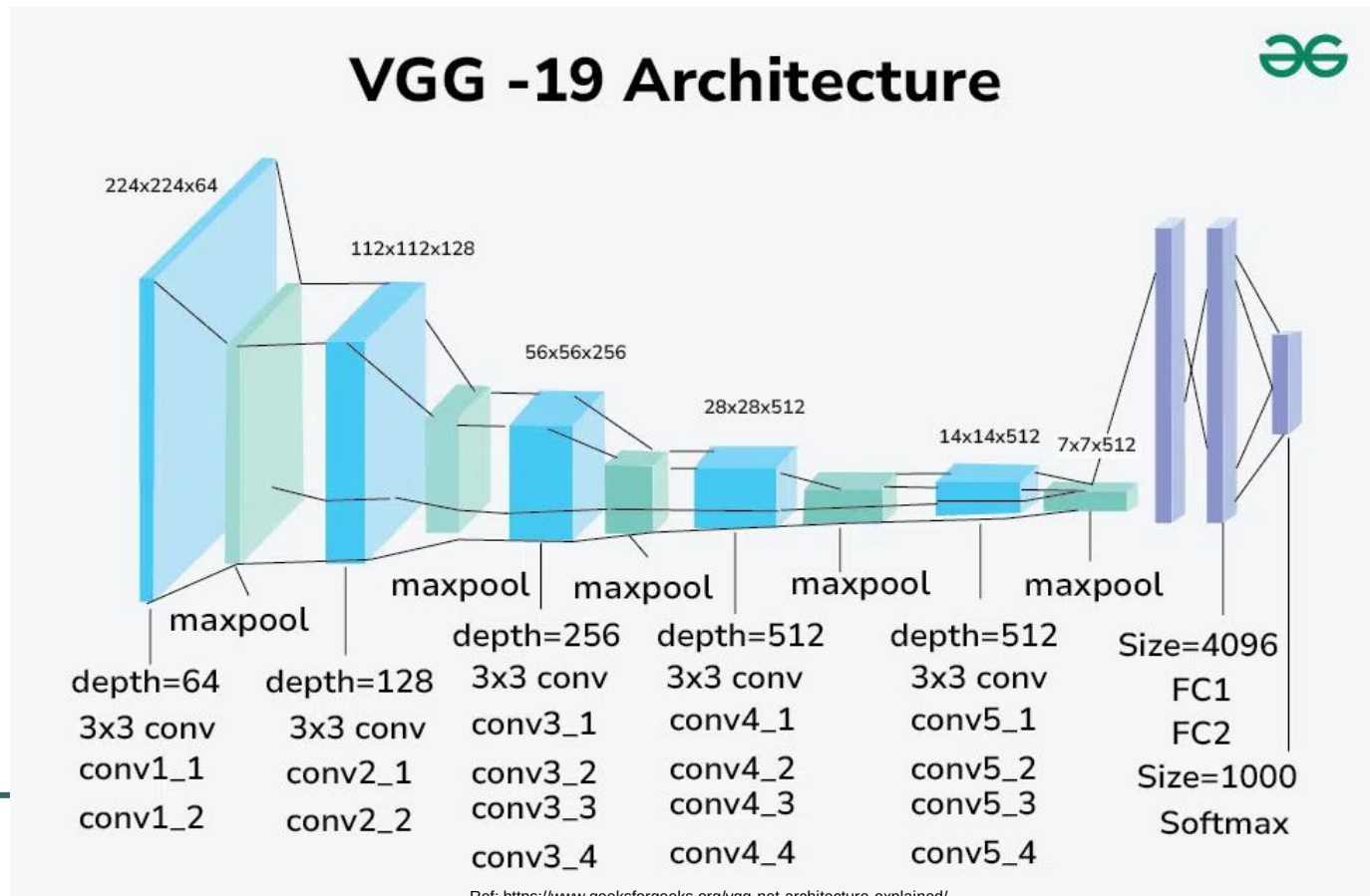


Using CNN for Image Classification



VGGNet

- Developed by the Visual Geometry Group at the University of Oxford
- Known for its deep architecture
- Using small 3x3 convolutional filters consistently throughout the network
 - helps in capturing fine details and improving performance compared to larger filters used in earlier architectures like AlexNet



GoogLeNet

- **Inception Module:** uses multiple filter sizes (1x1, 3x3, and 5x5) in parallel within the same layer, allowing the network to capture features at different scales and improves its ability to detect objects.
- **Dimensionality Reduction:** employs 1x1 convolutions to reduce the dimensionality of feature maps before applying larger convolutions. This reduces the computational cost and helps in managing the number of parameters.
- **Auxiliary Classifiers:** To combat the vanishing gradient problem, GoogLeNet includes auxiliary classifiers at intermediate layers. These classifiers provide additional gradient signals and help in stabilizing the training process.

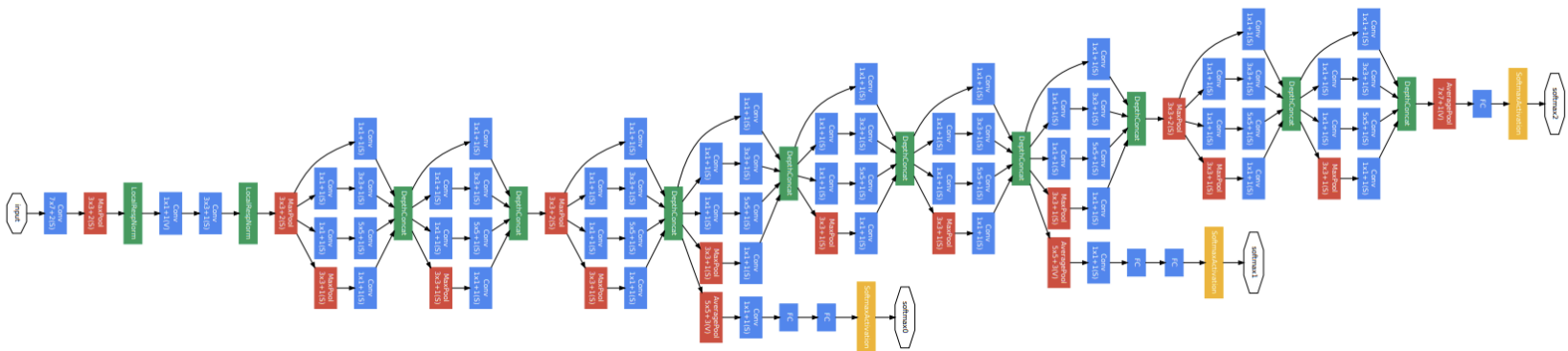
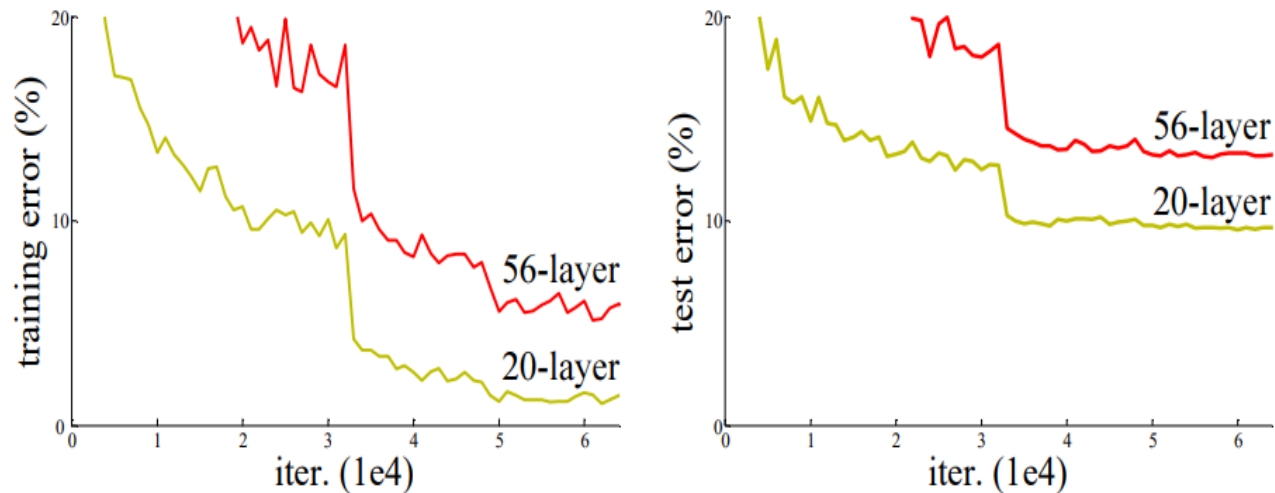


Figure: GoogLeNet with all the bells and whistles

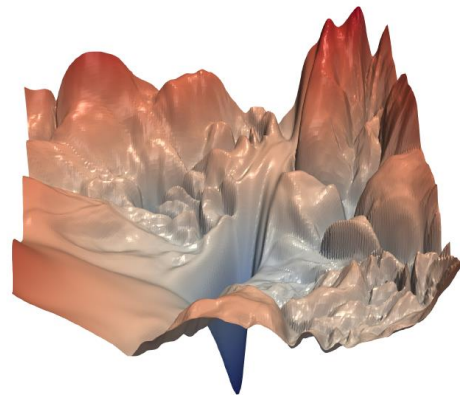
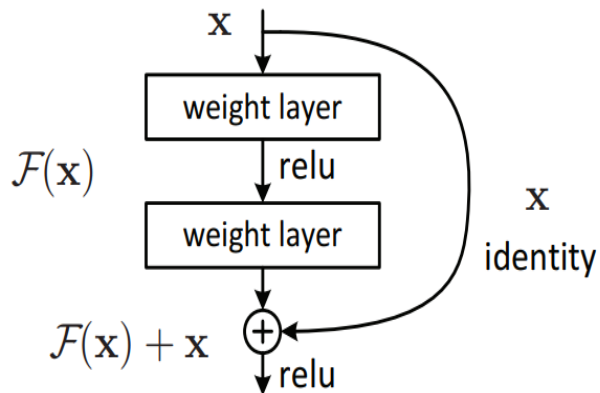
ResNet

- ResNet (Residual Network) is the Winner of ILSVRC 2015 in image classification, detection, and localization, as well as Winner of MS COCO 2015 detection, and segmentation

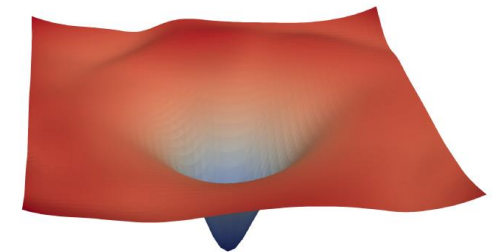


ResNet

- To solve the problem of vanishing/exploding gradients, a skip/shortcut connection is added to add the input x to the output after a few weight layers, as below



(a) without skip connections



(b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

Source: Li, Hao, et al. "Visualizing the loss landscape of neural nets." *Advances in Neural Information Processing Systems*. 2018.

ResNet

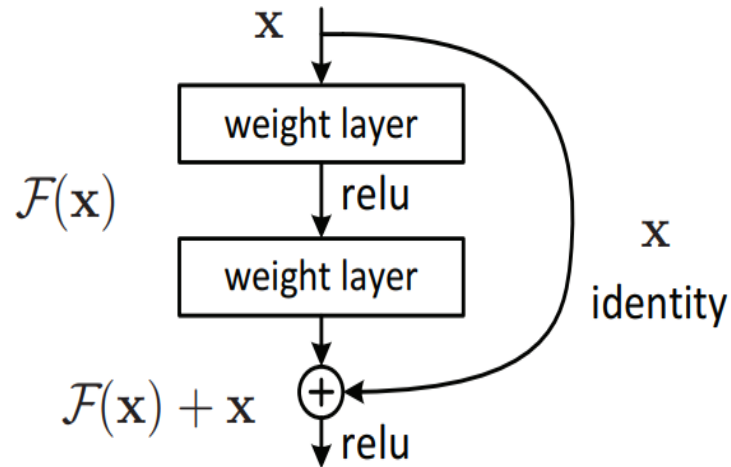
- ResNet can avoid vanishing gradient problem

$$\begin{array}{ccccccc} x & \rightarrow & w_1 & \rightarrow & w_2 & \rightarrow & w_3 & \rightarrow & w_4 & \rightarrow & Loss \\ & & y_1 & & y_2 & & y_3 & & y_4 & & \end{array}$$

$$\begin{aligned} \frac{\partial Loss}{\partial w_1} &= \frac{\partial Loss}{\partial y_4} \frac{\partial y_4}{\partial z_4} \frac{\partial z_4}{\partial y_3} \frac{\partial y_3}{\partial z_3} \frac{\partial z_3}{\partial y_2} \frac{\partial y_2}{\partial z_2} \frac{\partial z_2}{\partial y_1} \frac{\partial y_1}{\partial z_1} \frac{\partial z_1}{\partial w_1} \\ &= \frac{\partial Loss}{\partial y_4} \sigma'(z_4) w_4 \sigma'(z_3) w_3 \sigma'(z_2) w_2 \sigma'(z_1) x_1 \end{aligned}$$

ResNet

- ResNet can avoid vanishing gradient problem



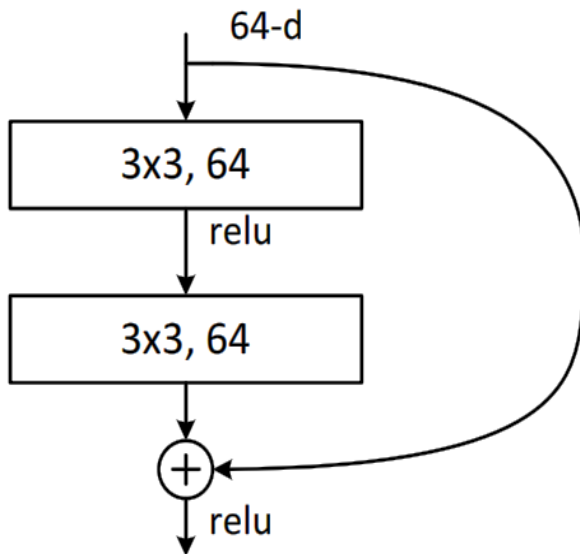
$$\mathbf{x}_L = \mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i),$$

$$\frac{\partial \mathcal{E}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{E}}{\partial \mathbf{x}_L} \frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{E}}{\partial \mathbf{x}_L} \left(1 + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i) \right).$$

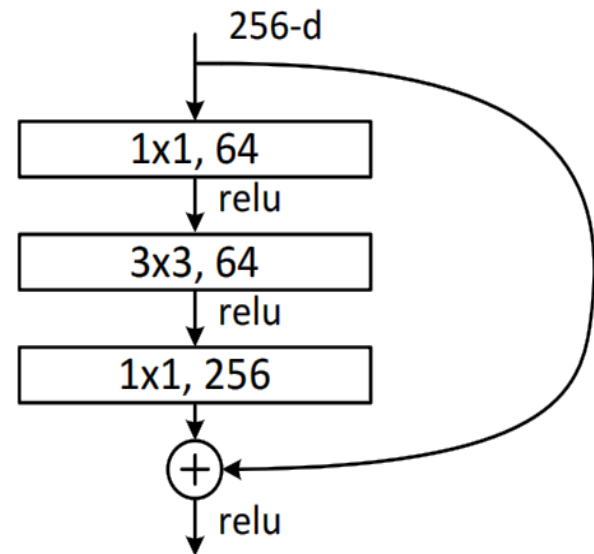
ResNet

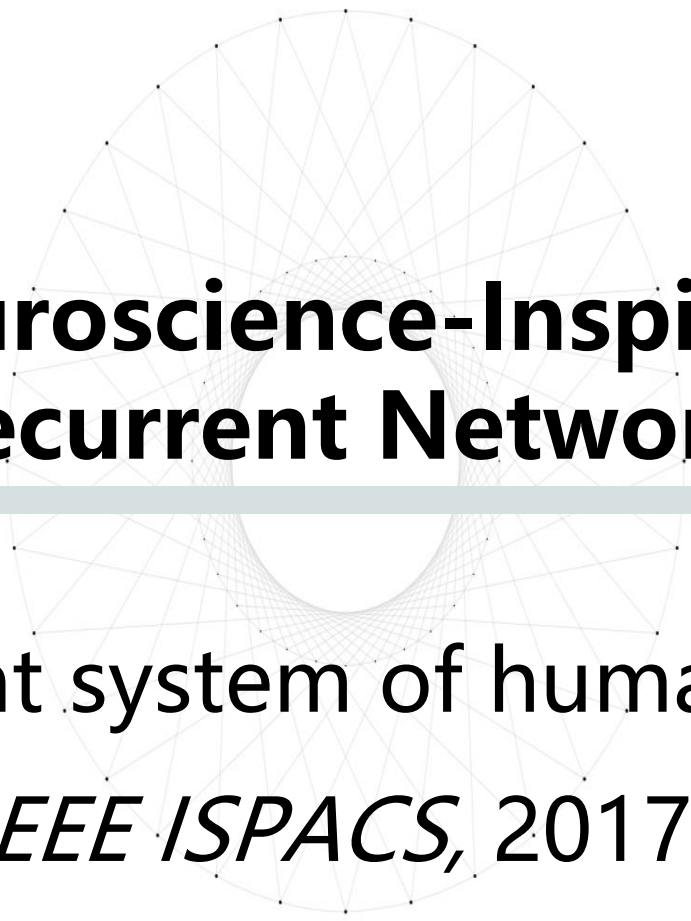
- ResNe18 (Basic block), ResNet50 & ResNet152 (Bottleneck block)

Basic block



Bottleneck block





Neuroscience-Inspired Recurrent Network

Recurrent system of human brain
(*IEEE ISPACS, 2017*)



Perception of
an ambiguous
figure is
affected by
own-age social
biases,
*Scientific
Reports*, 2018



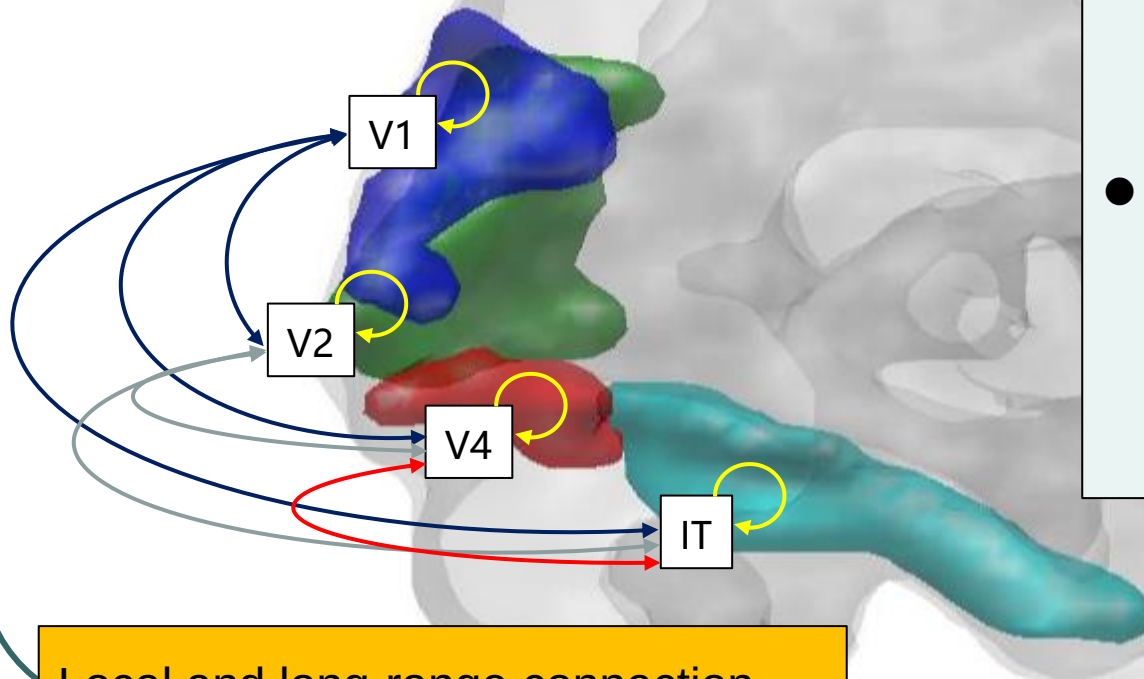
Human brain is a recurrent system

V1 (primary visual cortex): orientation

V2 (secondary visual cortex): corner detection

V4: color processing

IT (Inferior temporal lobe): object recognition



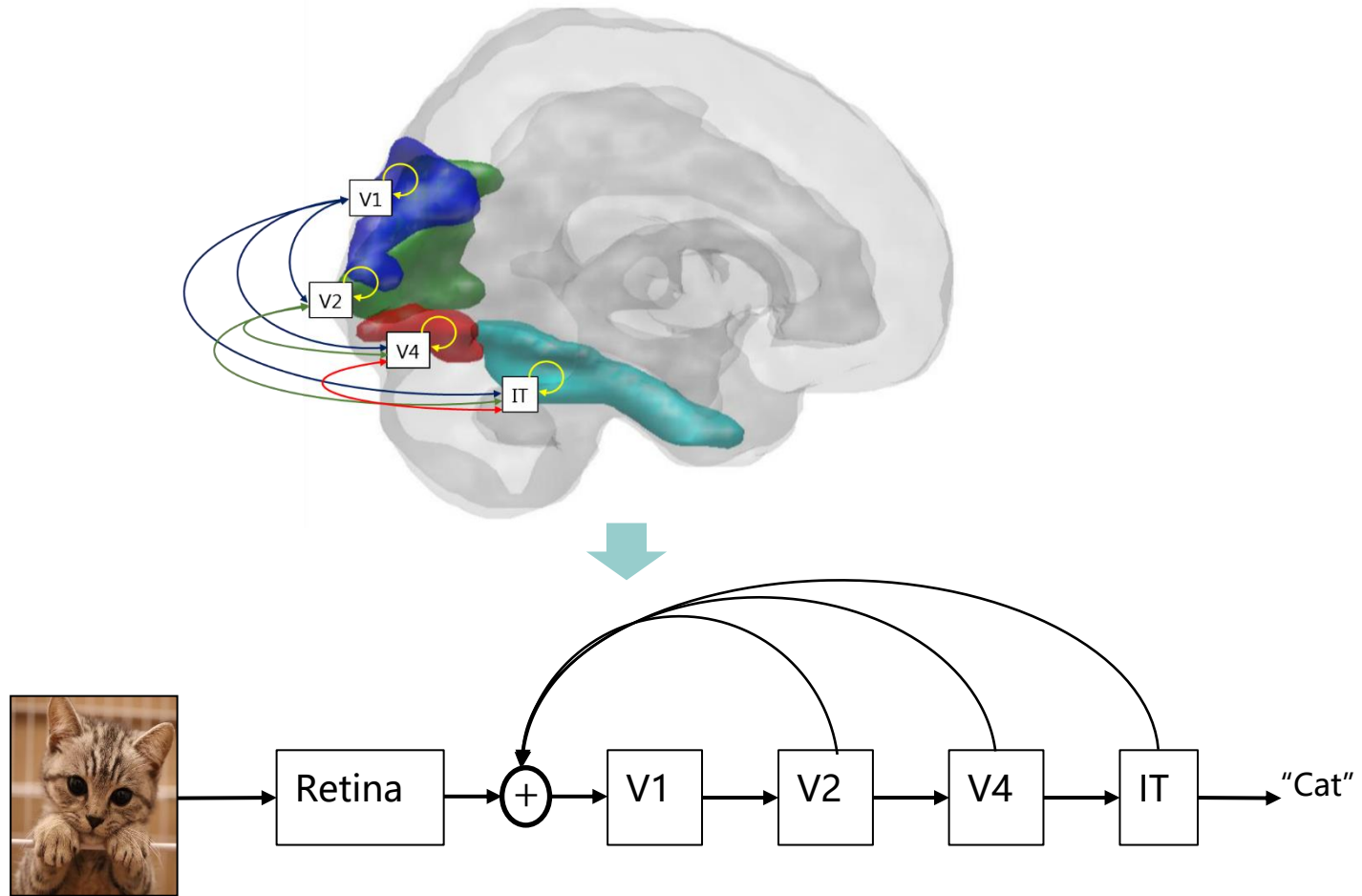
- Human can recognize image in a single forward
 - Potter et al. *AP&P* 2014
 - Takes about **13 ms**
 - 63% accuracy
- In normal, human takes about **350 ms** to recognize an object
 - Almost 100% accuracy
 - Lots of recurrences

Local and long-range connection

- Top-down modulation

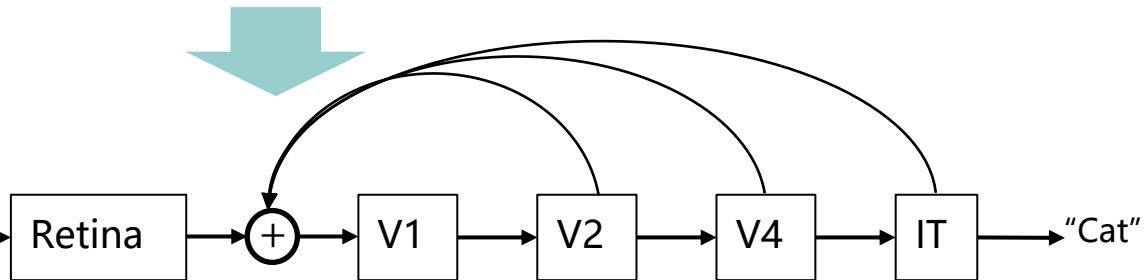
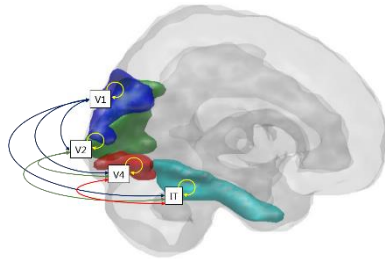
Ventral Stream: Object Recognition

Proposed recurrent model

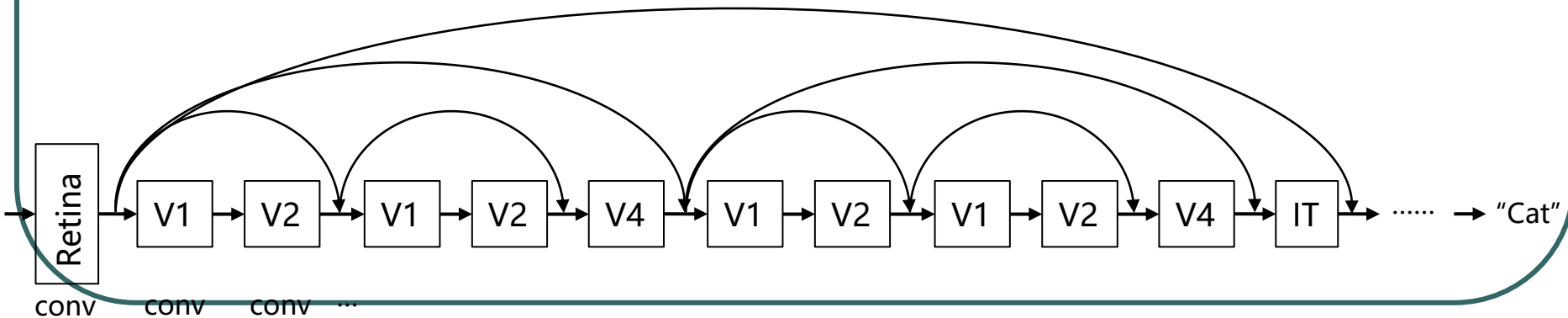


A recurrent network has long-range connection

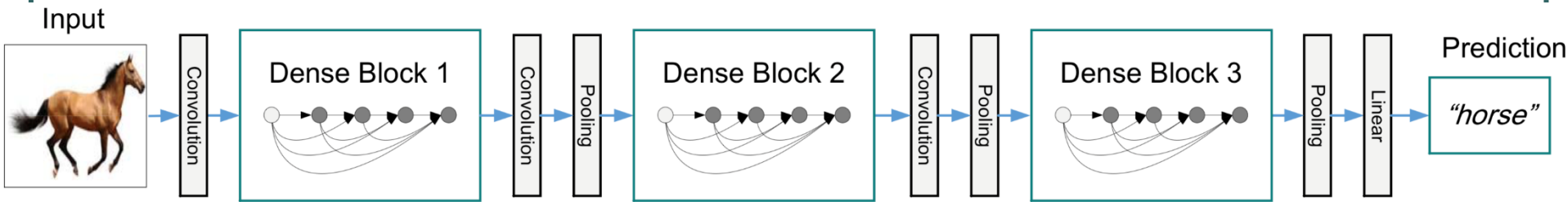
Unfold Recurrent Model to A Forward Network



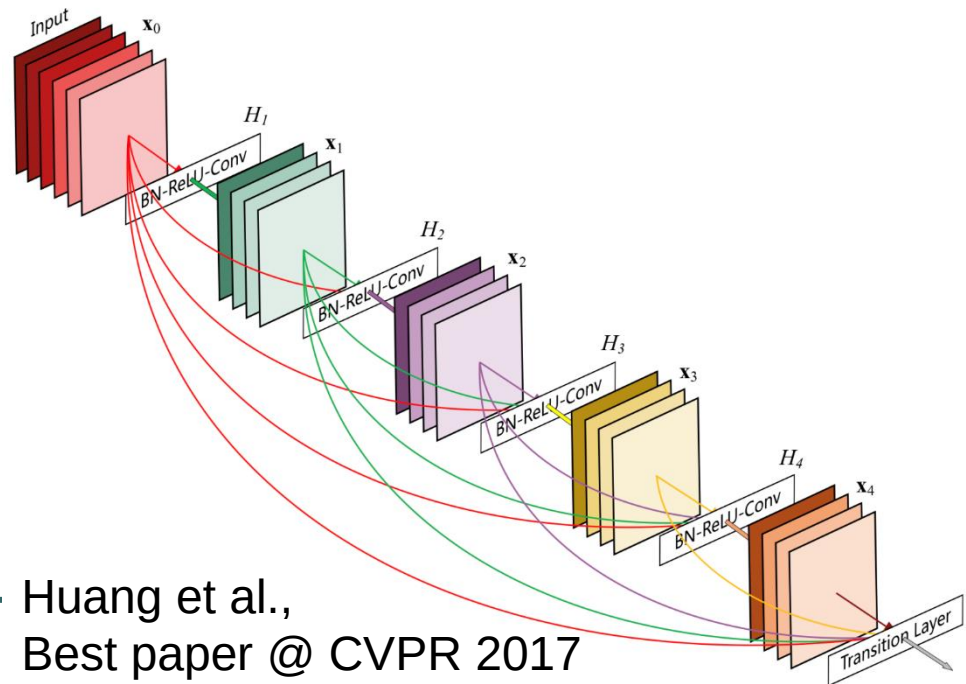
Recurrence 2



DenseNet



- Feature reuse
- High parameter efficiency



Huang et al.,
Best paper @ CVPR 2017

Beyond Classification

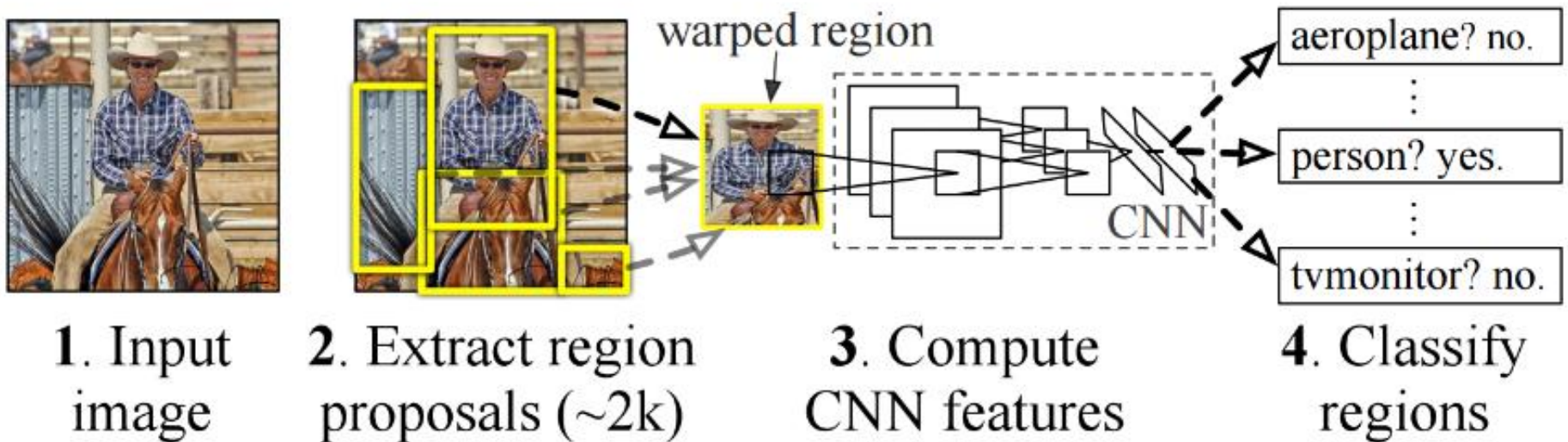
- Detection
- Segmentation
- Regression
- Pose estimation
- Matching patches
- Synthesis
- Style transfer

and many more...

R-CNN: Regions with CNN features

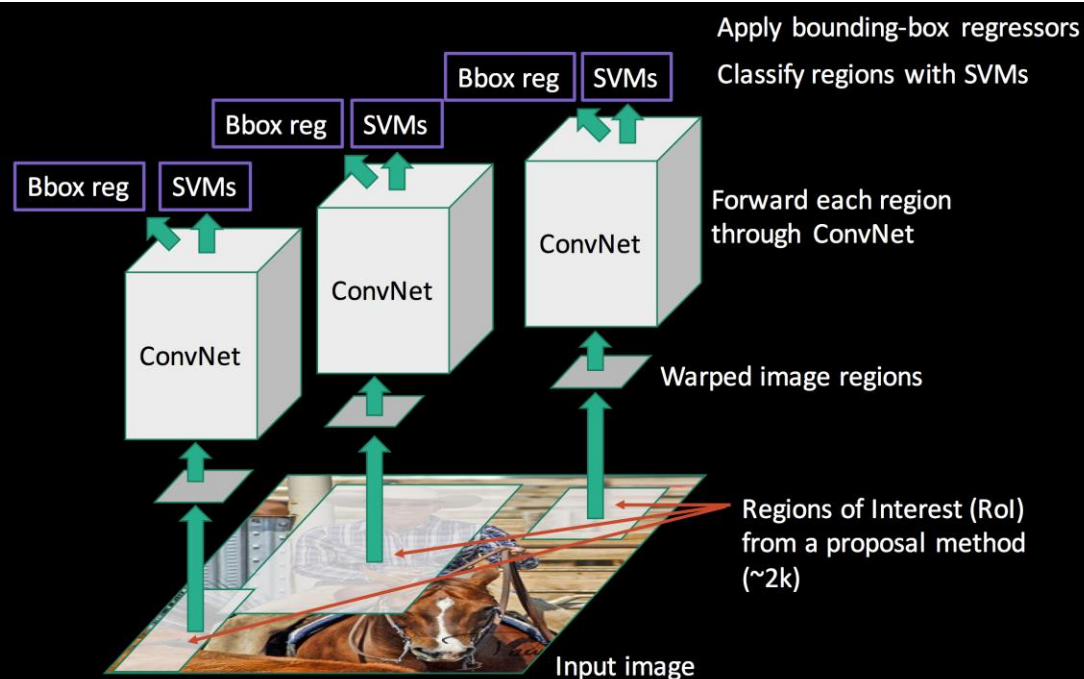
- Trained on ImageNet classification
- Finetune CNN on PASCAL (data sets for object classification)

R-CNN: *Regions with CNN features*



R-CNN: Regions with CNN features

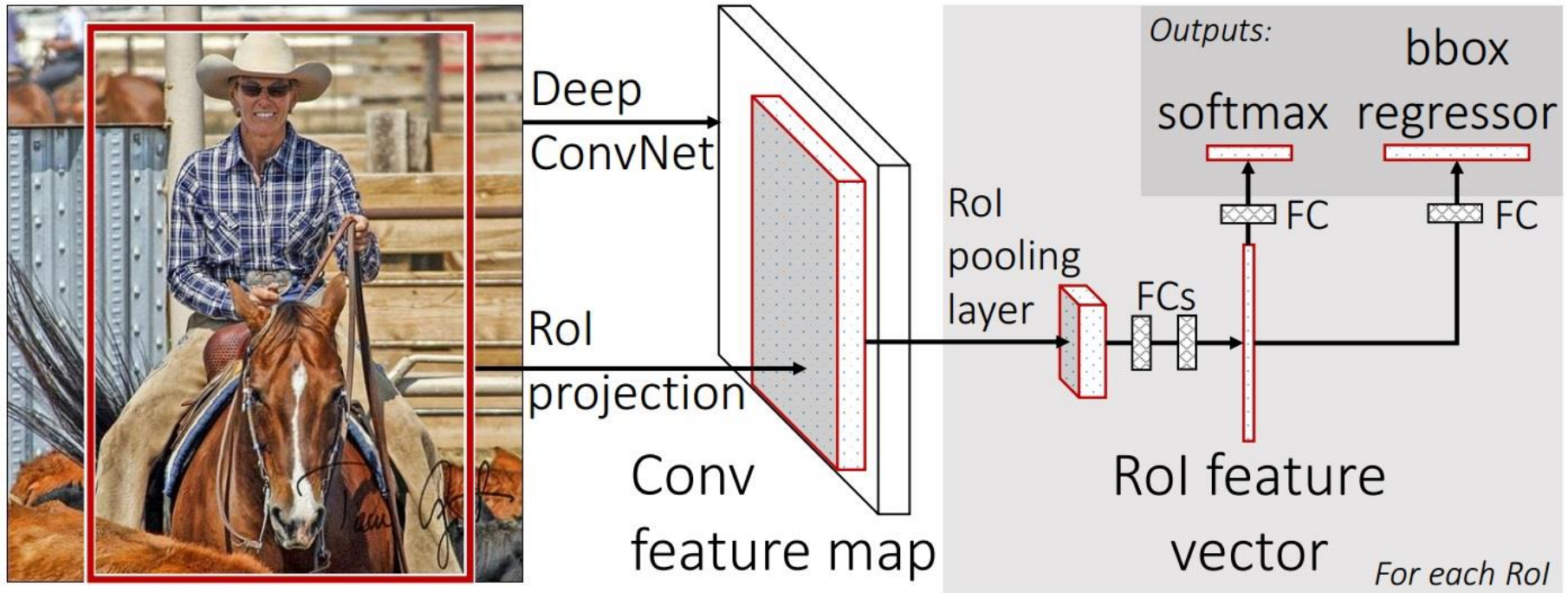
- Selective search for region proposals



van de Sande et al. ICCV'11)

RCNN [Girshick et al. CVPR 2014]

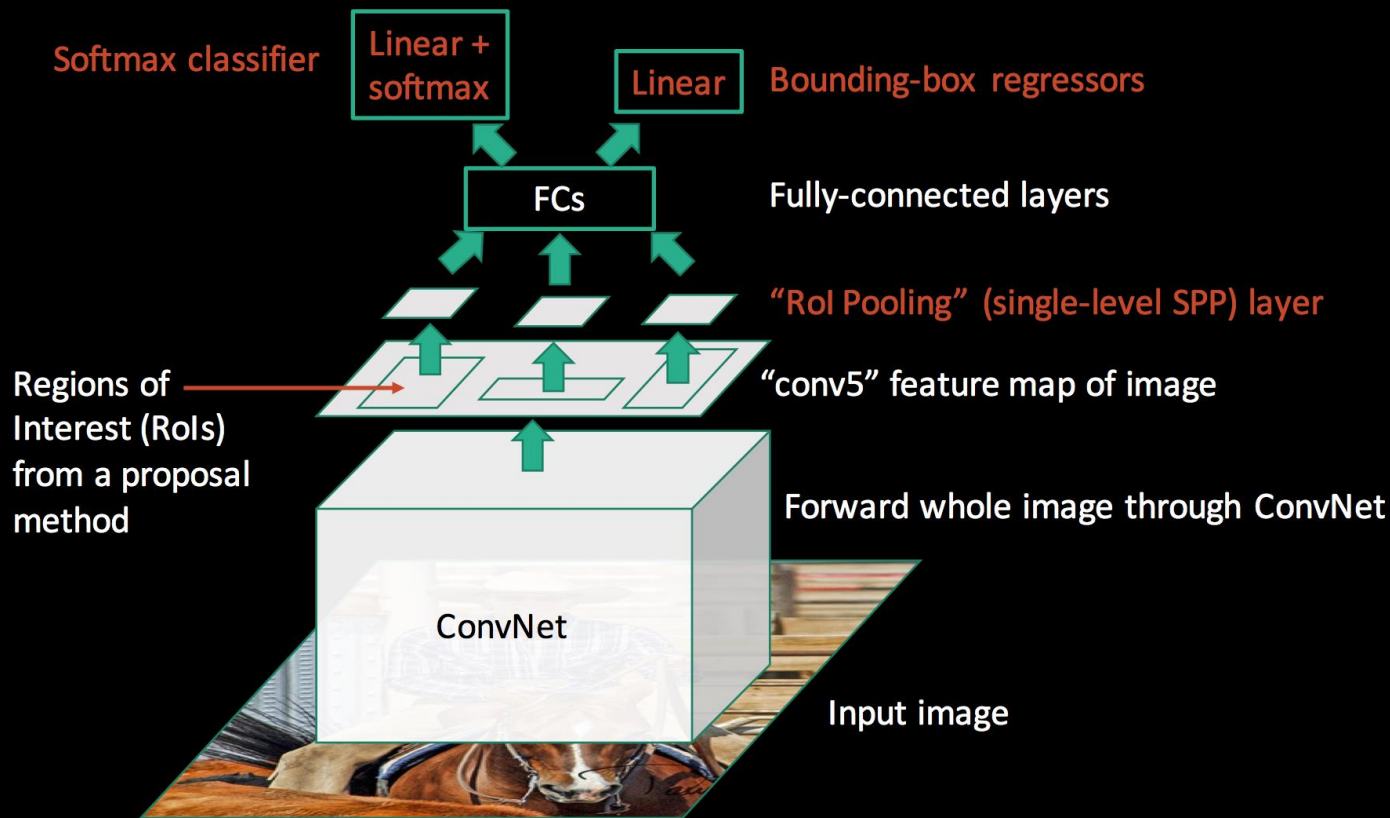
Fast R-CNN



Fast RCNN [[Girshick, R 2015](#)]
<https://github.com/rbgirshick/fast-rcnn>

Fast R-CNN

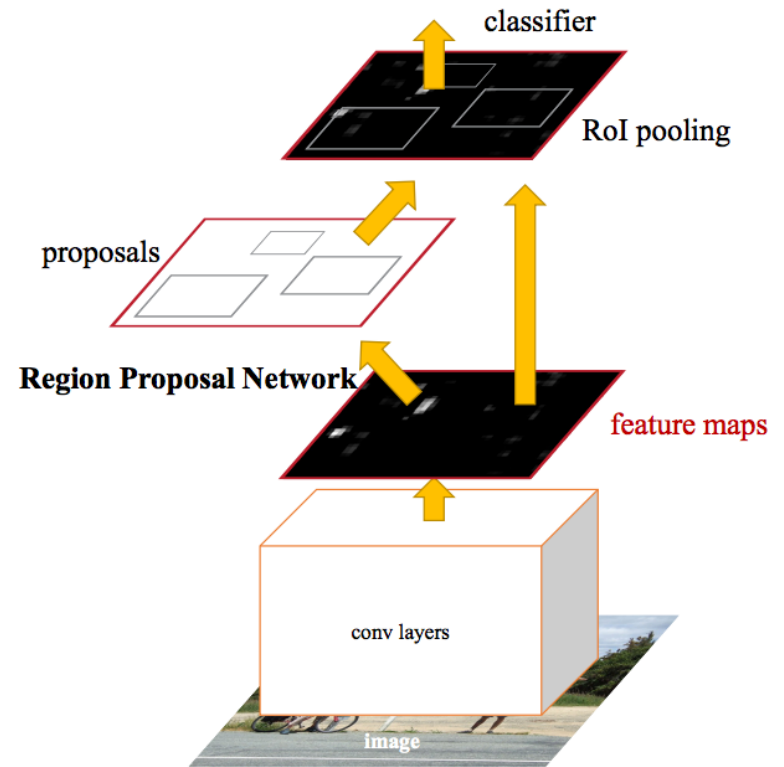
- Fast (avoid duplicate calculation of CNN features)



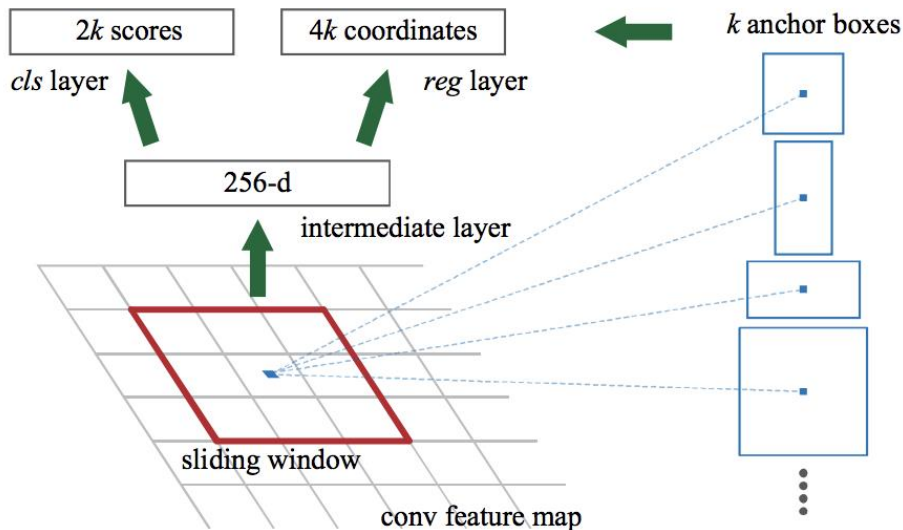
Fast RCNN
[[Girshick, R](#)
[2015](#)]

Faster R-CNN

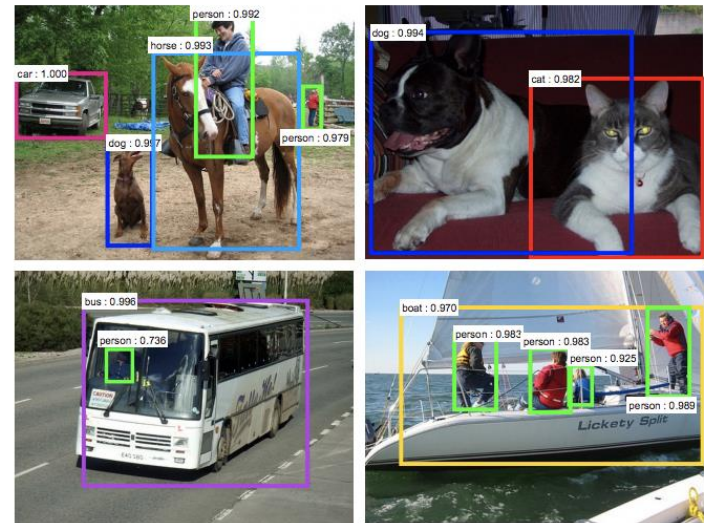
- Determine region proposals on feature map, instead of image.



Region Proposal Network



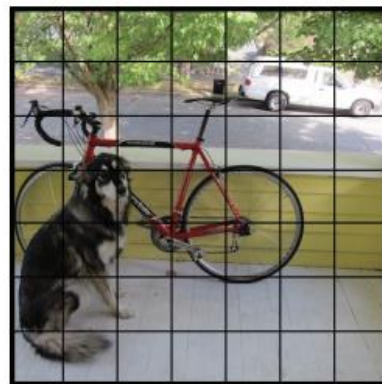
Backbone, eg, VGG16: 7x7x256



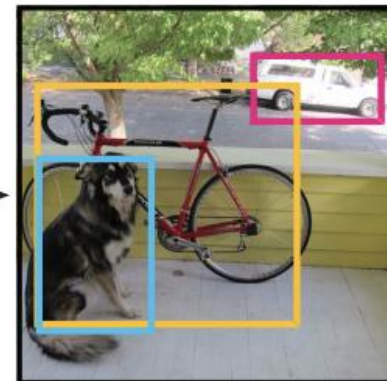
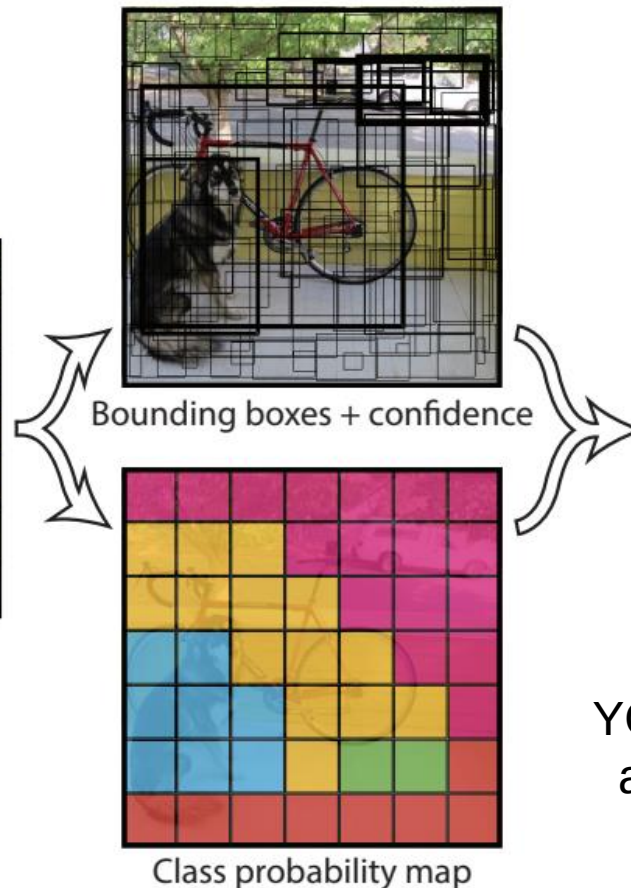
Faster RCNN [Ren, S, et al., 2015]

YOLO (You Only Look Once)

- YOLO prediction vector:
 $S \times S \times (B \times 5 + C)$ E.g., $7 \times 7 \times (2 \times 5 + 20)$
B: #bounding boxes
C: #classes
5: x,y,w,h,confidence



$S \times S$ grid on input



Final detections

YOLO [Redmon et al., CVPR 2016]

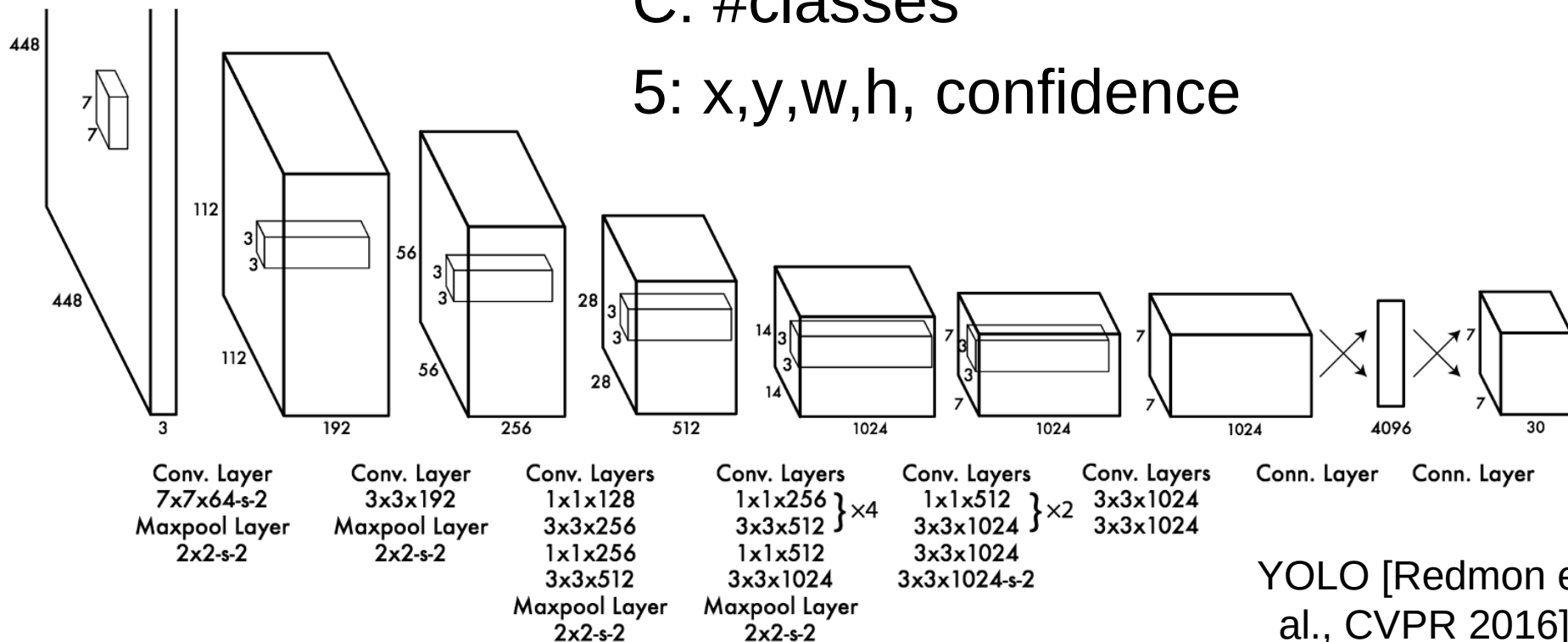
YOU
ONLY
LIVE
ONCE

YOU ONLY LOVE ONCE



YOLO (You Only Look Once)

- YOLO prediction vector:
 $S \times S \times (B \times 5 + C)$ E.g., $7 \times 7 \times (2 \times 5 + 20)$
 B : #bounding boxes
 C : #classes
 5 : x, y, w, h , confidence

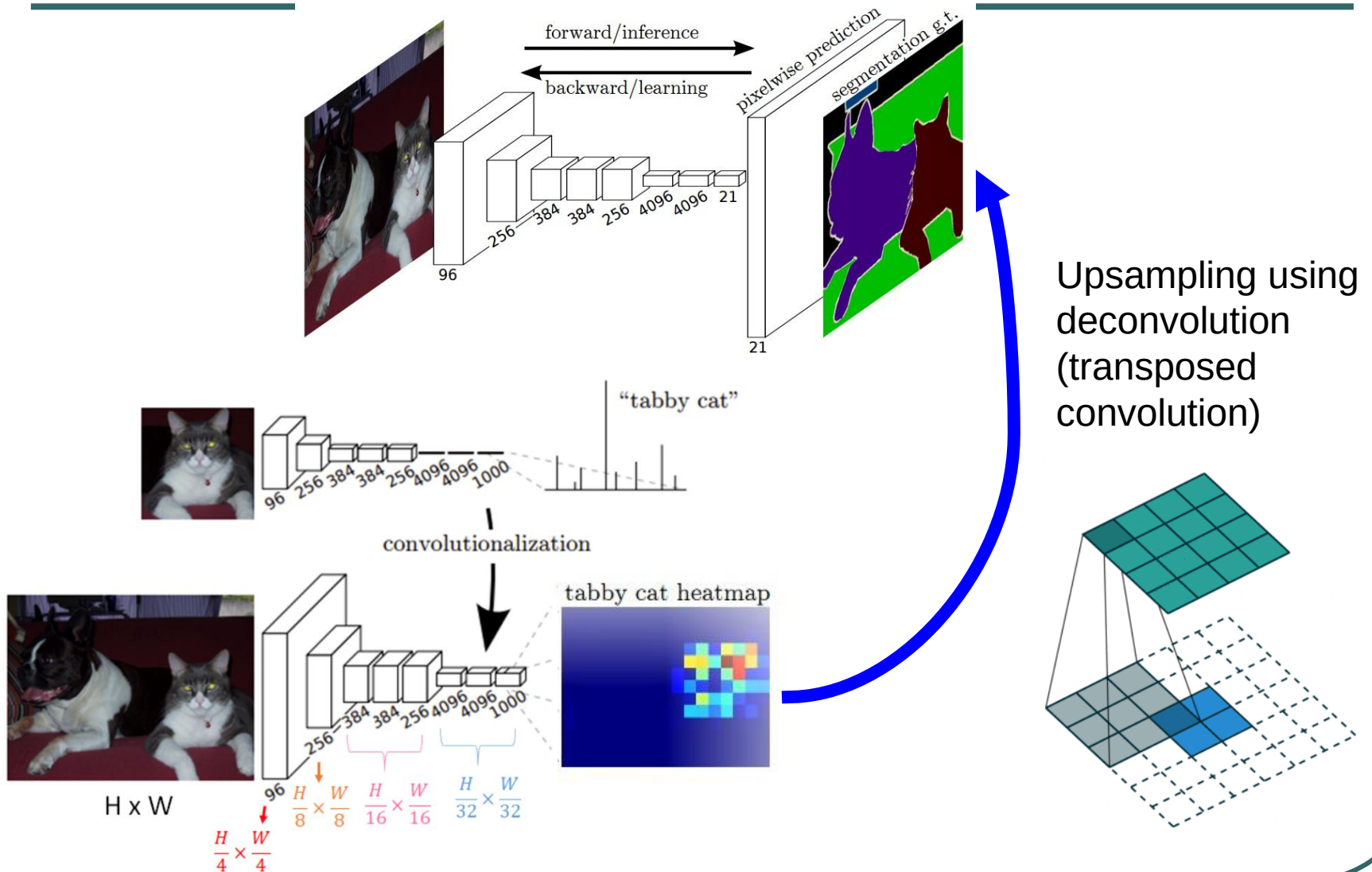


YOLO [Redmon et al., CVPR 2016]

YOLO (You Only Look Once)

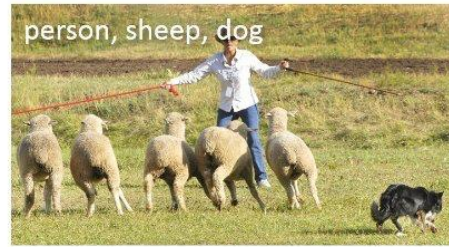
Version	Year	Contributions
v1	2015	One-shot object detector
v2	2016	Multi-scale training, dimensional clustering
v3	2018	SPP block, Darknet-53
v4	2020	Mish-based activation, CSPDarknet-53 backbone
v5	2020	Anchor-free detection, SWISH-based activation, PANet
v6	2022	Self-attention, anchor-free object detection
v7	2022	Transformers, E-ELAN reparameterization
v8	2023	GANs, anchor-free detections
v9	2024	Programmable Gradient Information (PGI), Generalized Efficient Layer Aggregation Network (GELAN)
v10	2024	NMS-free training approach, dual label assignments, holistic model design for enhanced accuracy and efficiency

Labeling Pixels: Semantic Labels

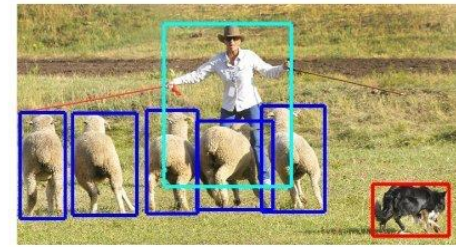


Mask R-CNN

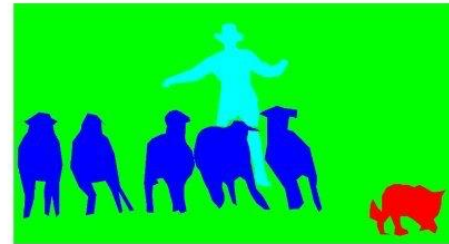
- Instance segmentation with Mask R-CNN



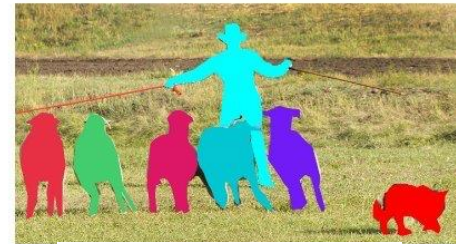
(a) Image classification



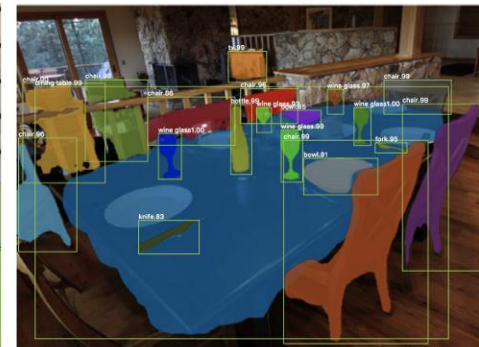
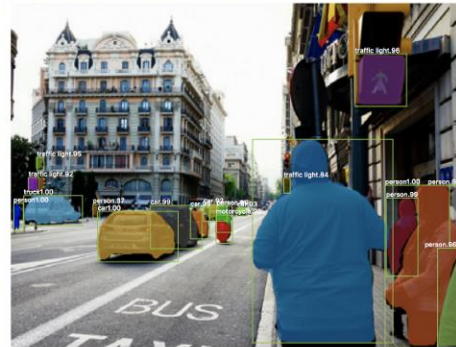
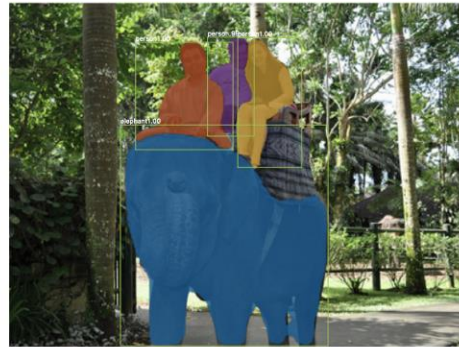
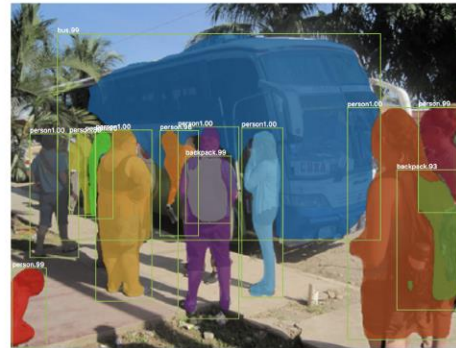
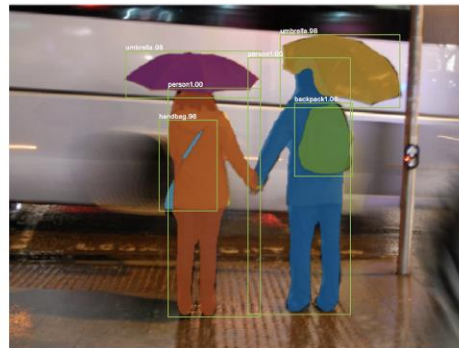
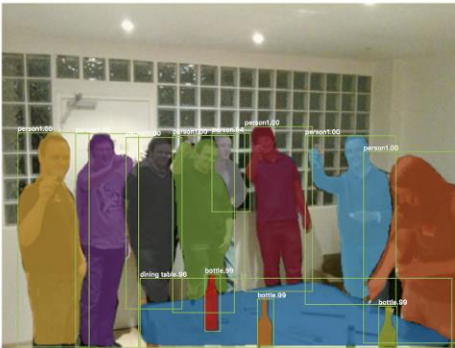
(b) Object localization



(c) Semantic segmentation



(d) Instance segmentation



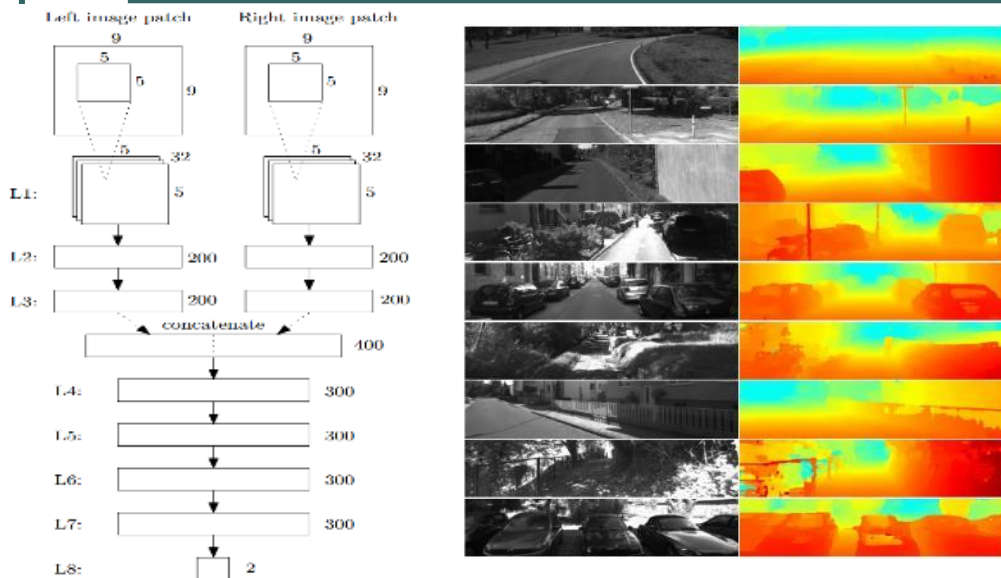
Mask RCNN [Kaiming He, 2017]

CNN for Regression

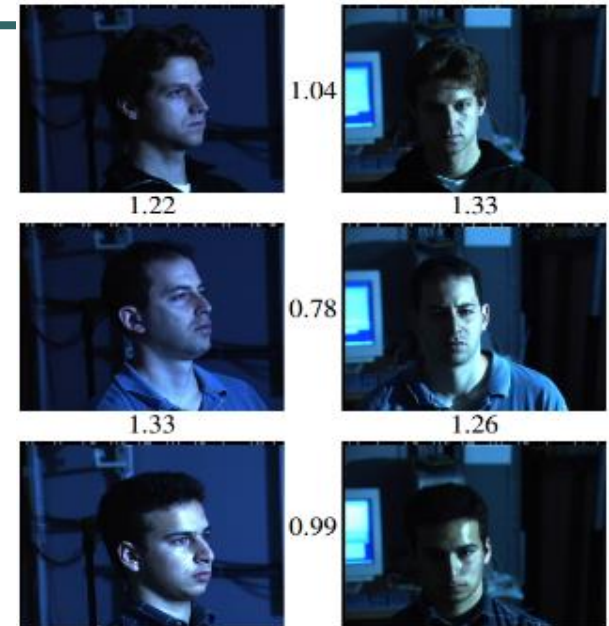


DeepPose [[Toshev and Szegedy CVPR 2014](#)]

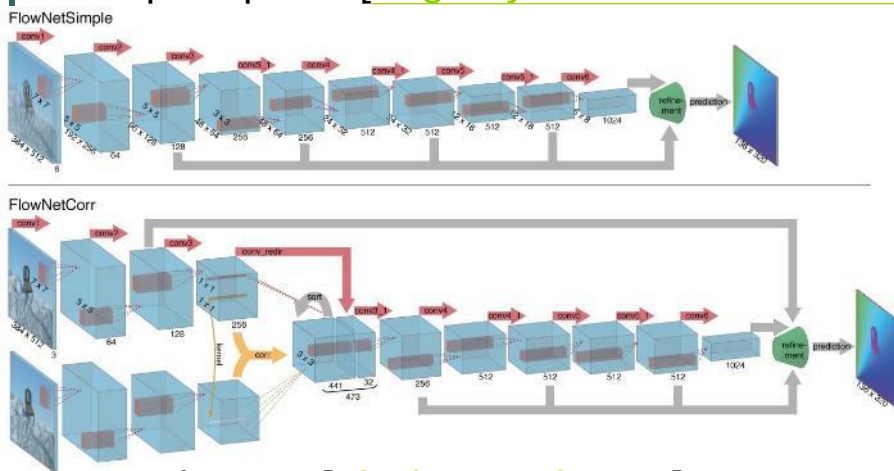
CNN as a Similarity Measure for Matching



Stereo matching [Zbontar and LeCun CVPR 2015]
 Compare patch [Zagoruyko and Komodakis 2015]



FaceNet [Schroff et al. 2015]

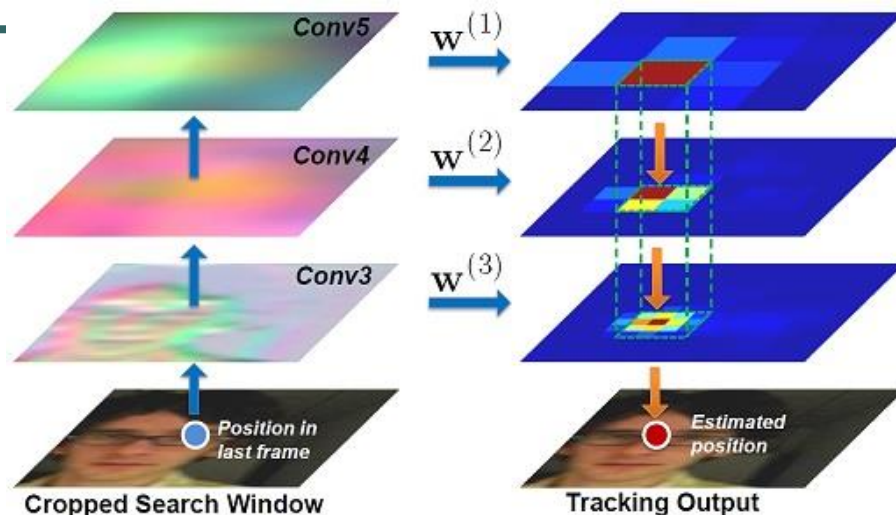


FlowNet [Fischer et al 2015]

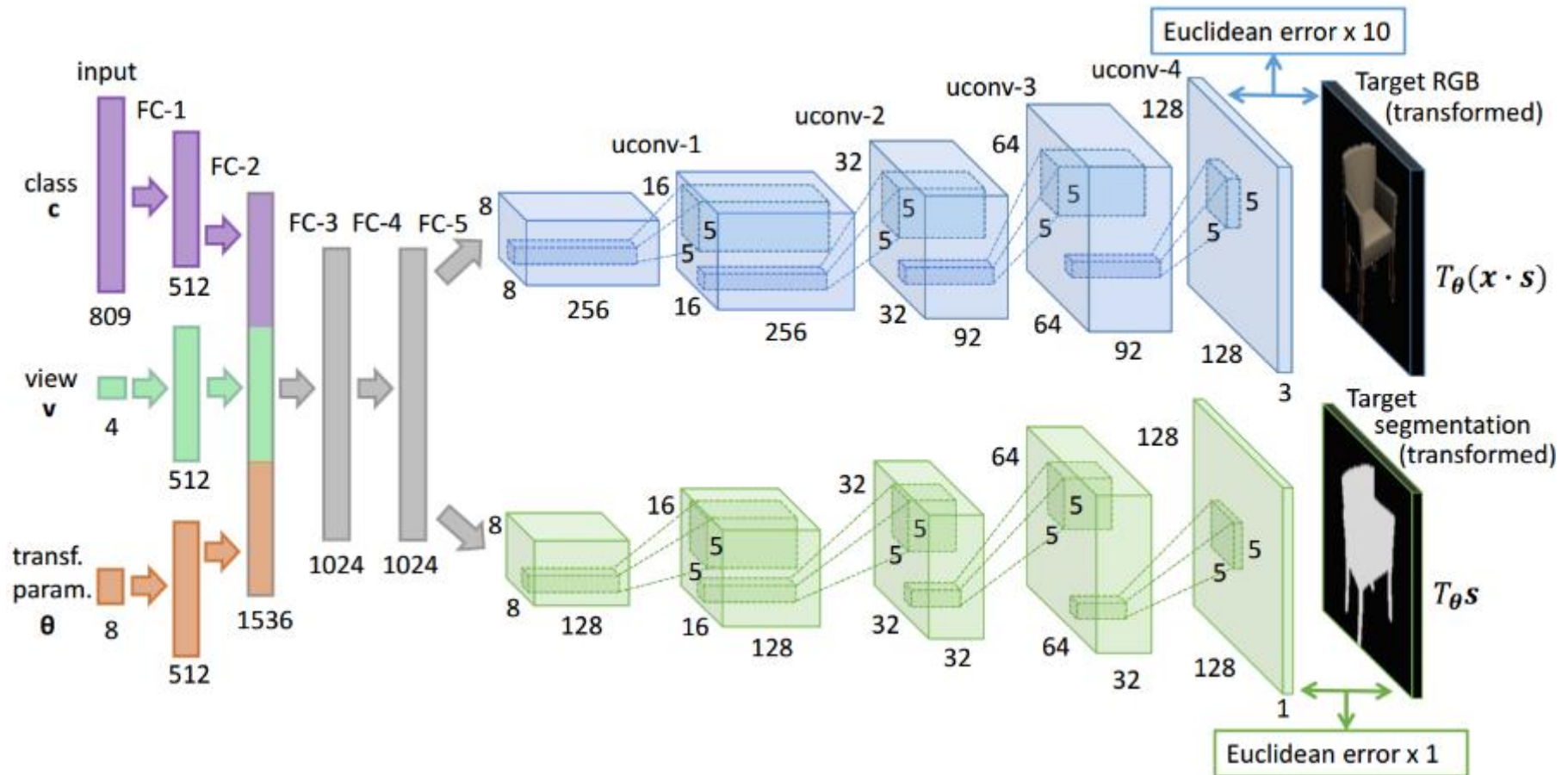


Match ground and aerial images
 [Lin et al. CVPR 2015]

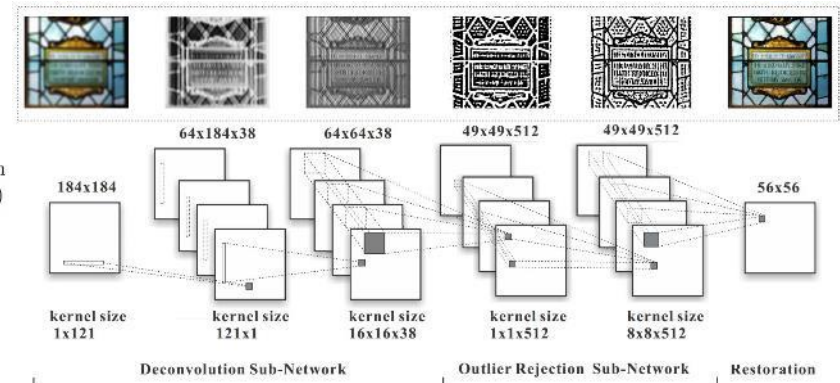
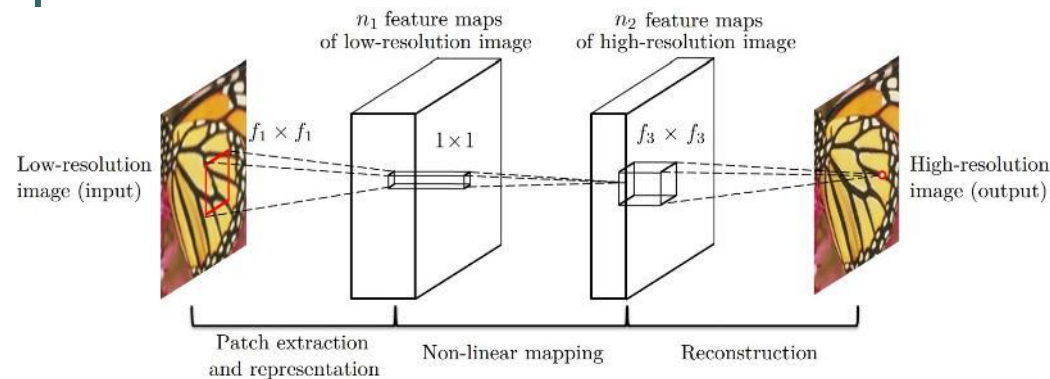
CNN for Online Visual Tracking



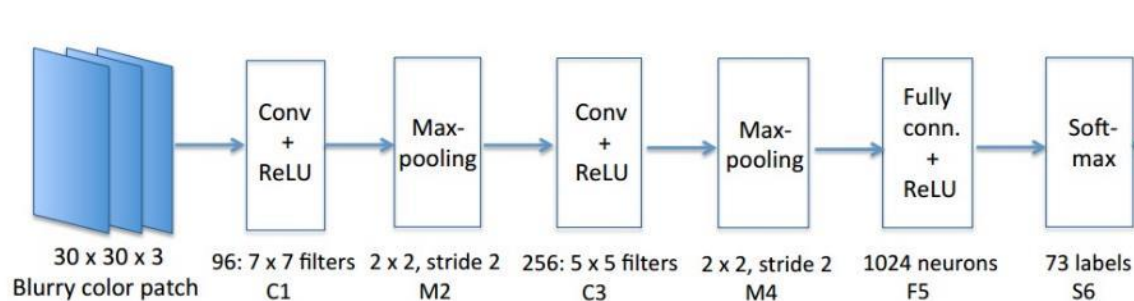
CNN for Image Generation



CNN for Image Restoration/Enhancement



Super-resolution
[Dong et al. ECCV 2014]



Non-blind deconvolution
[Xu et al. NIPS 2014]

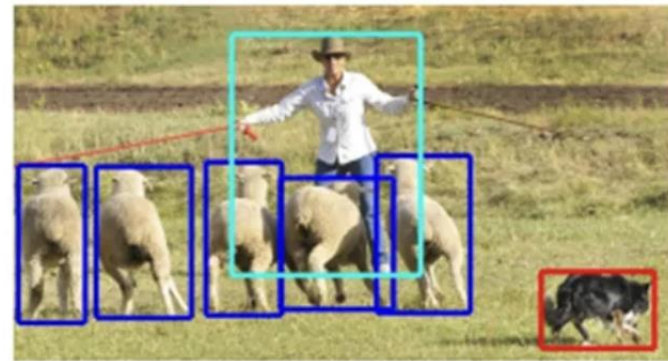


Non-uniform blur estimation
[Sun et al. CVPR 2015]

Segmentation



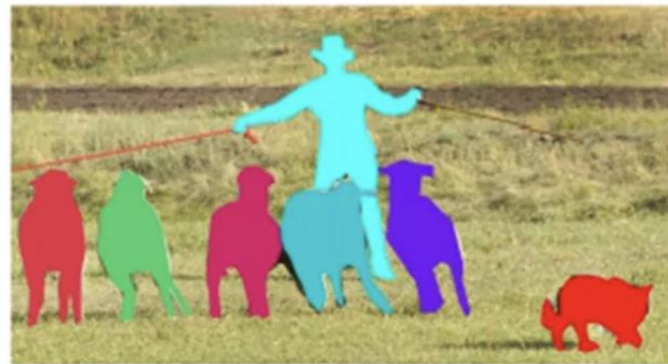
(a) Image classification



(b) Object localization



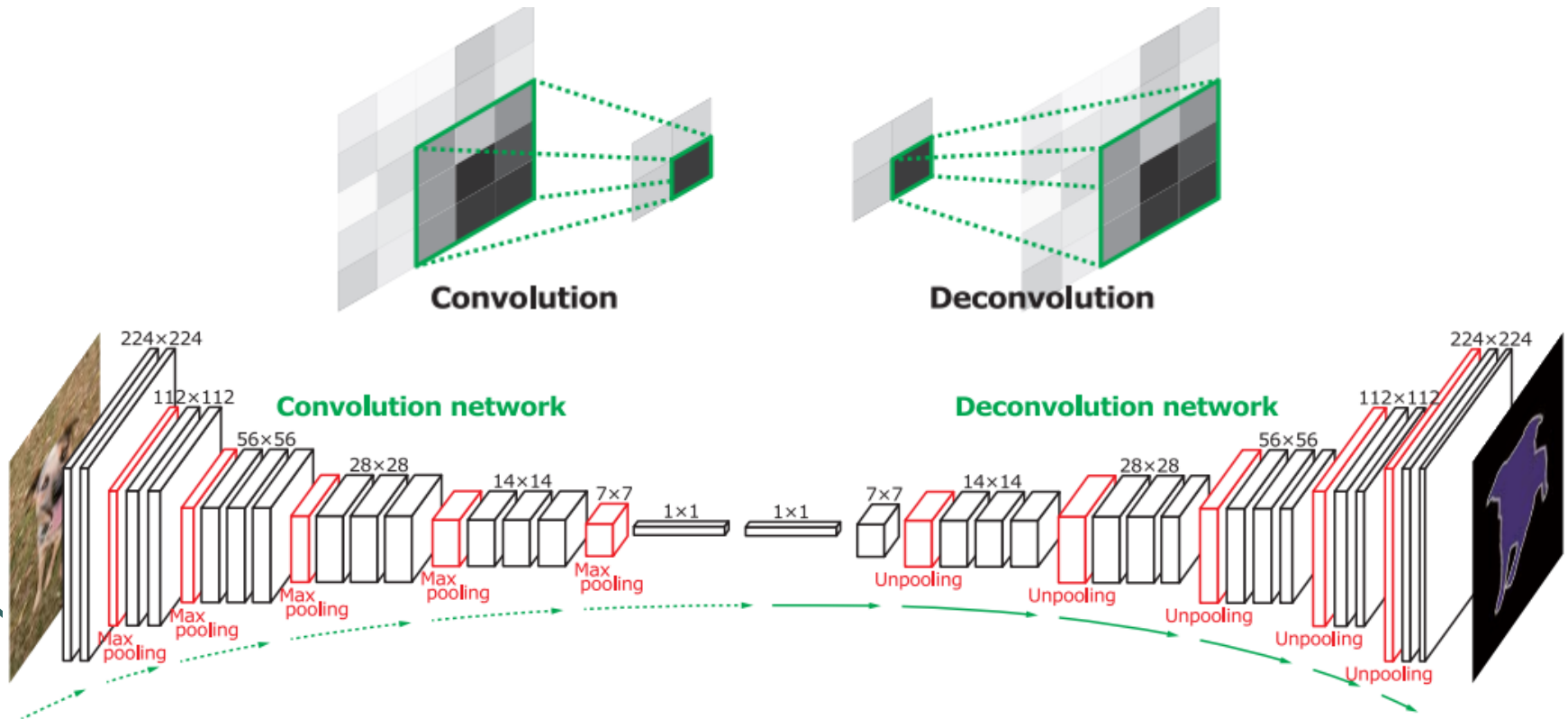
(c) Semantic segmentation



(d) Instance Segmentation

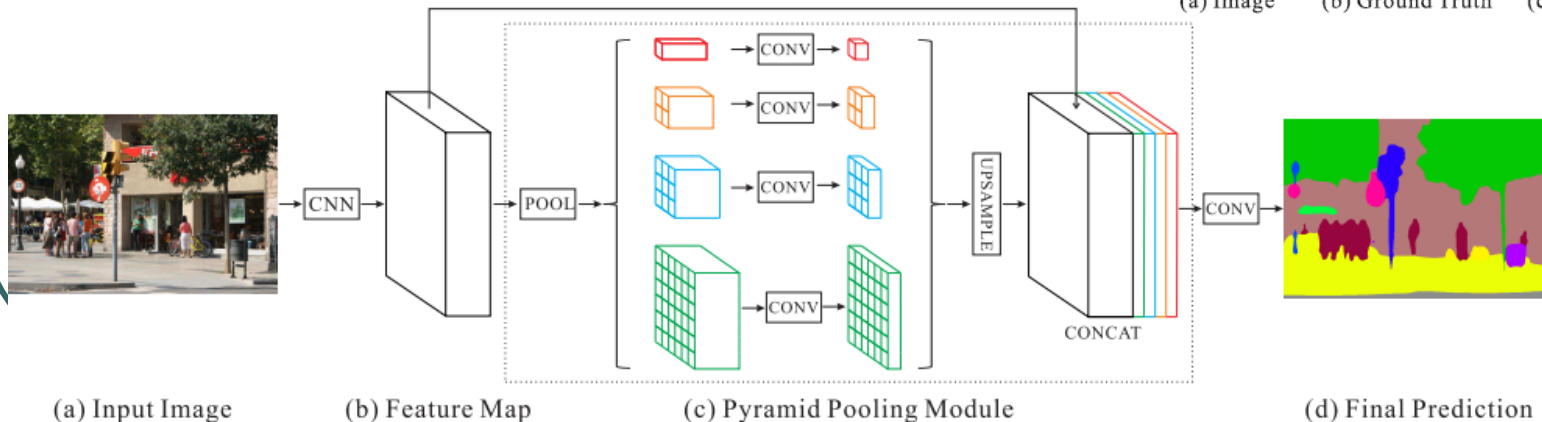
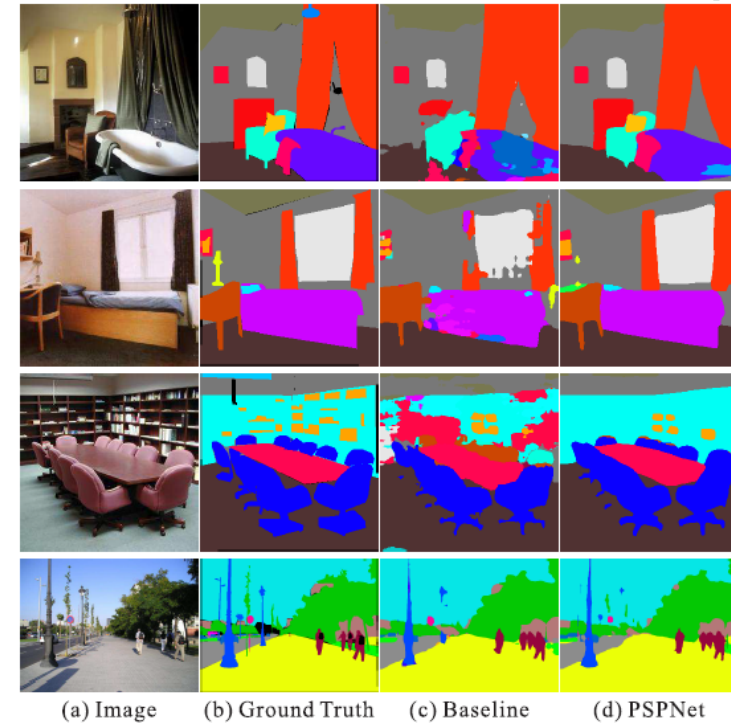
Semantic segmentation

- DeconvNet (ICCV 2015)
 - VGGNet as encoder
 - Design deconvolution to reconstruct the input



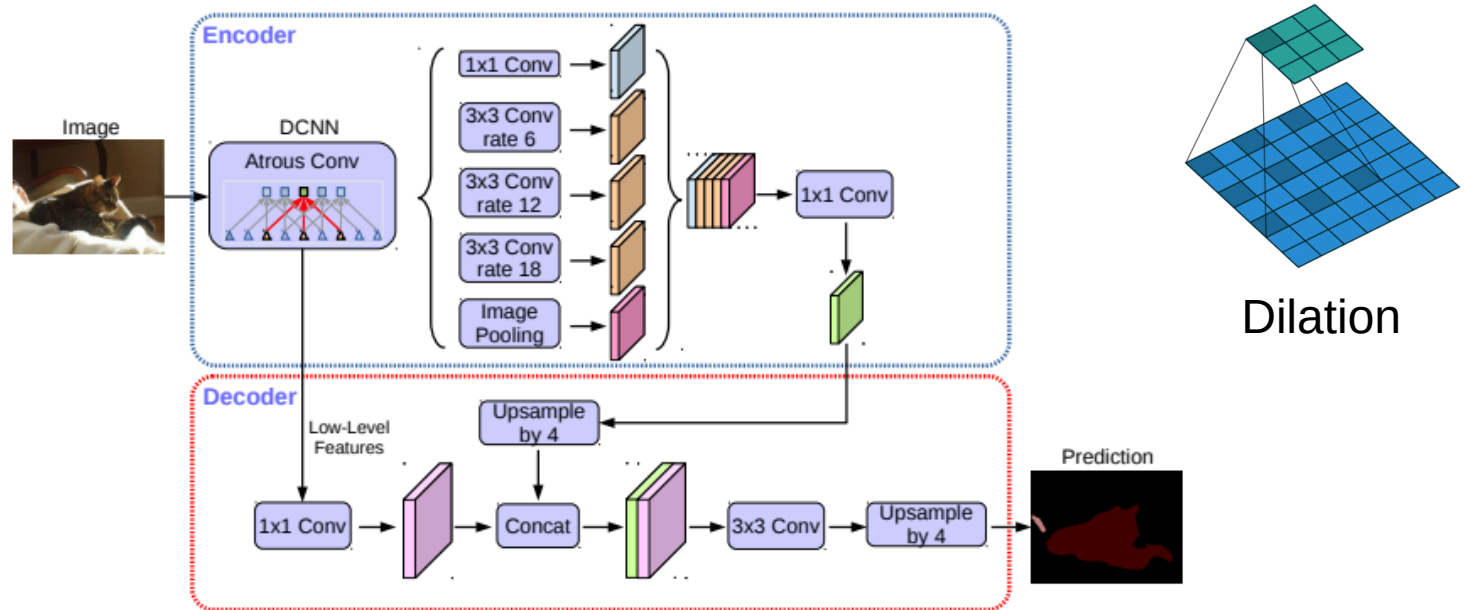
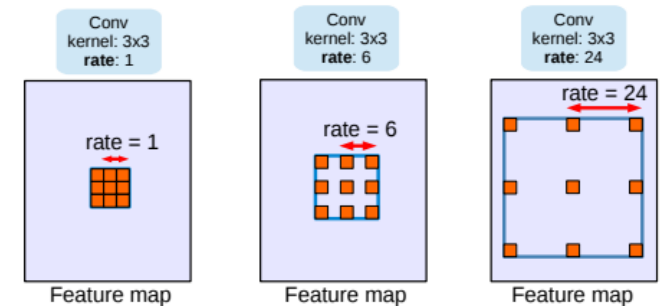
Semantic segmentation

- Pyramid Scene Parsing, PSPNet (CVPR 2017)



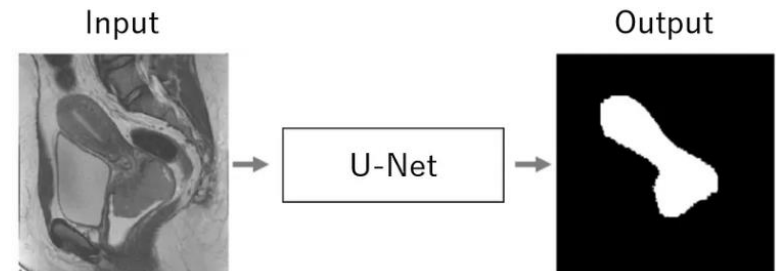
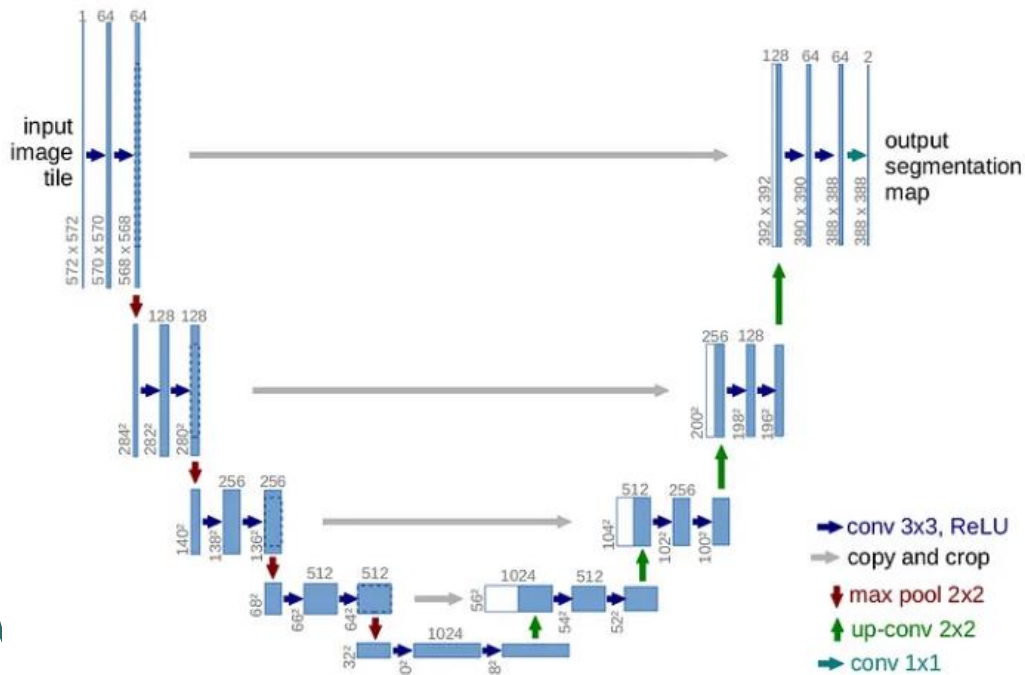
Semantic segmentation

- **DeepLabv3+ (ECCV 2018)**
 - Atrous Spatial Pyramid Pooling



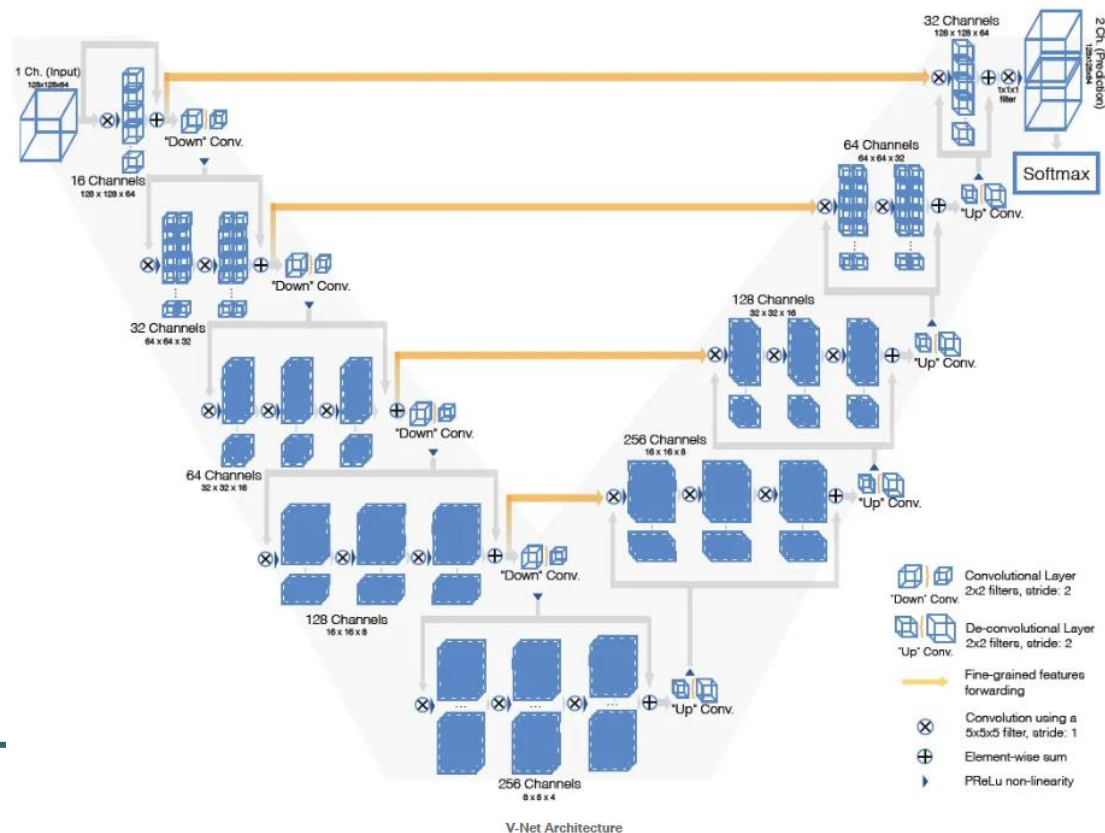
Semantic segmentation in Medical

- Unet (MICCAI 2015)
 - A U-shape **encoded-decoder** architecture
 - Skip connection design



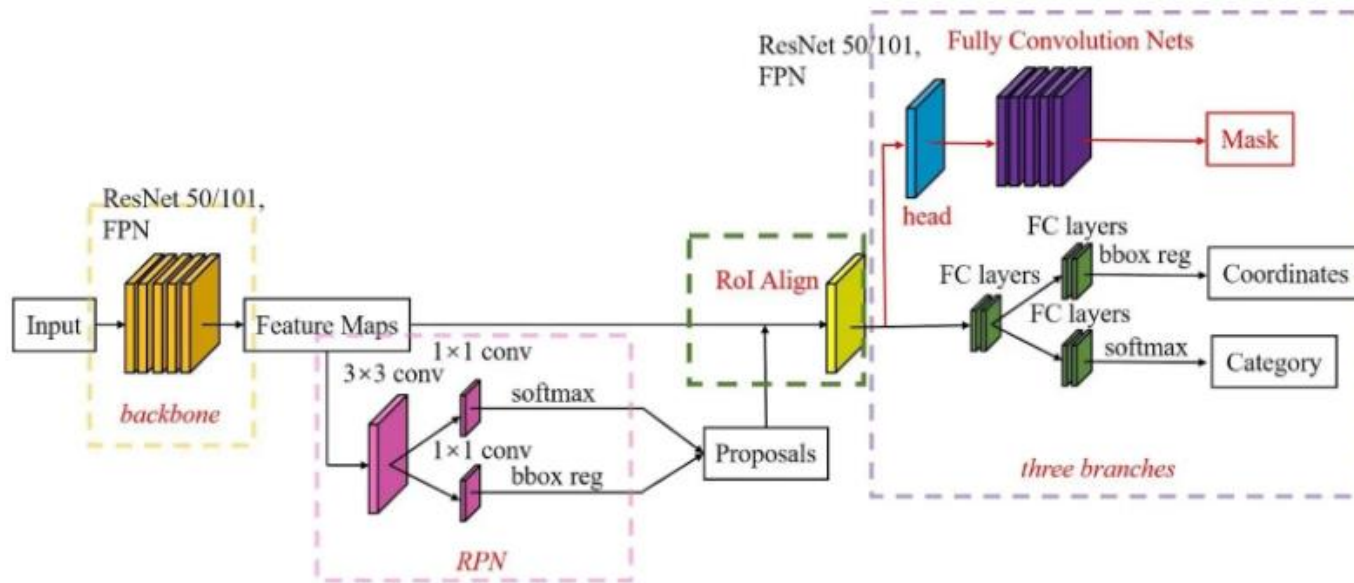
Semantic segmentation in Medical

- Vnet (3DV-2016)
 - 3D architecture
 - Change pooling operation into down conv. operation



Instance segmentation

- Mask Region-based Convolutional Neural Network (Mask R-CNN) – ICCV 2017
 - Feature Pyramid Network (FPN)
 - Region Proposal Network (RPN)



Instance segmentation

- Other models like YOLOv8, Segment Anything (SAM) ...

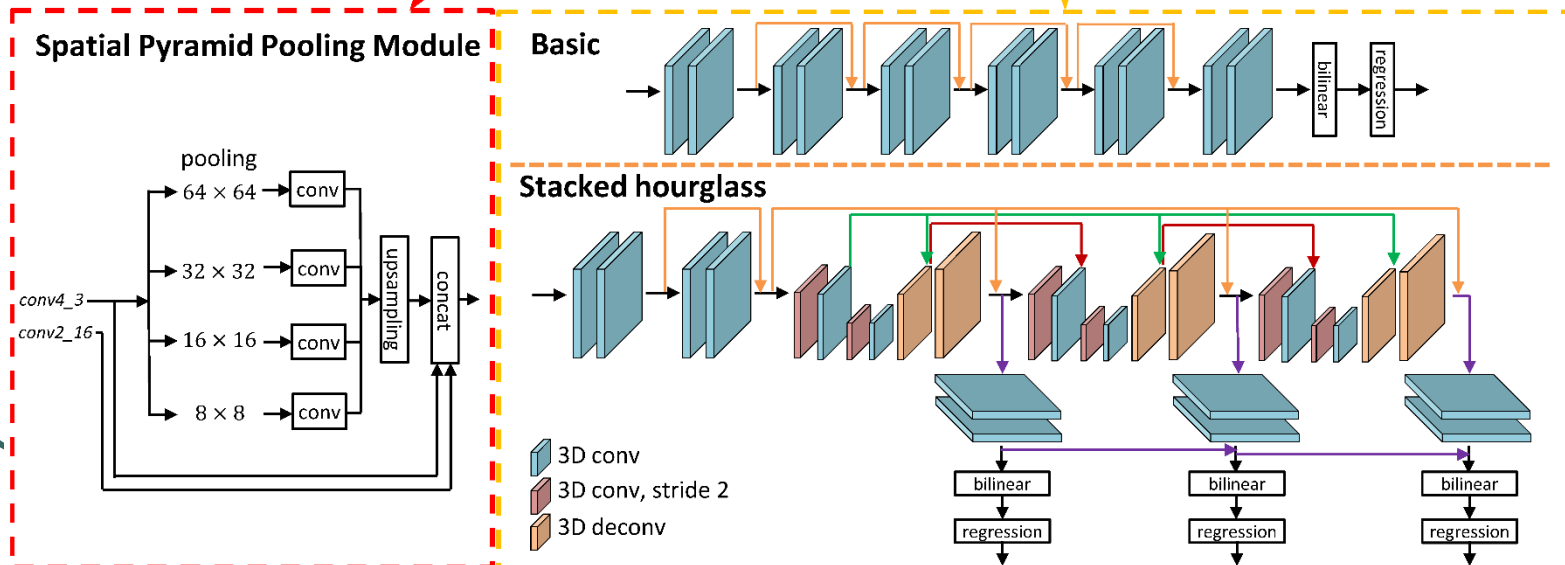
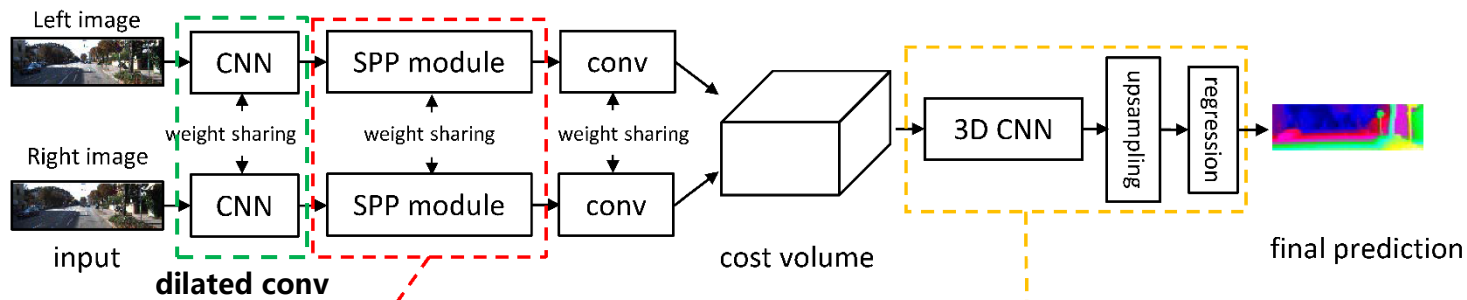


Figure 2. **Mask R-CNN** results on the COCO test set. These results are based on ResNet-101 [15], achieving a *mask AP* of 35.7 and running at 5 fps. Masks are shown in color, and bounding box, category, and confidences are also shown.

Pyramid Stereo Matching Network

Our Solution: Exploiting Global Context

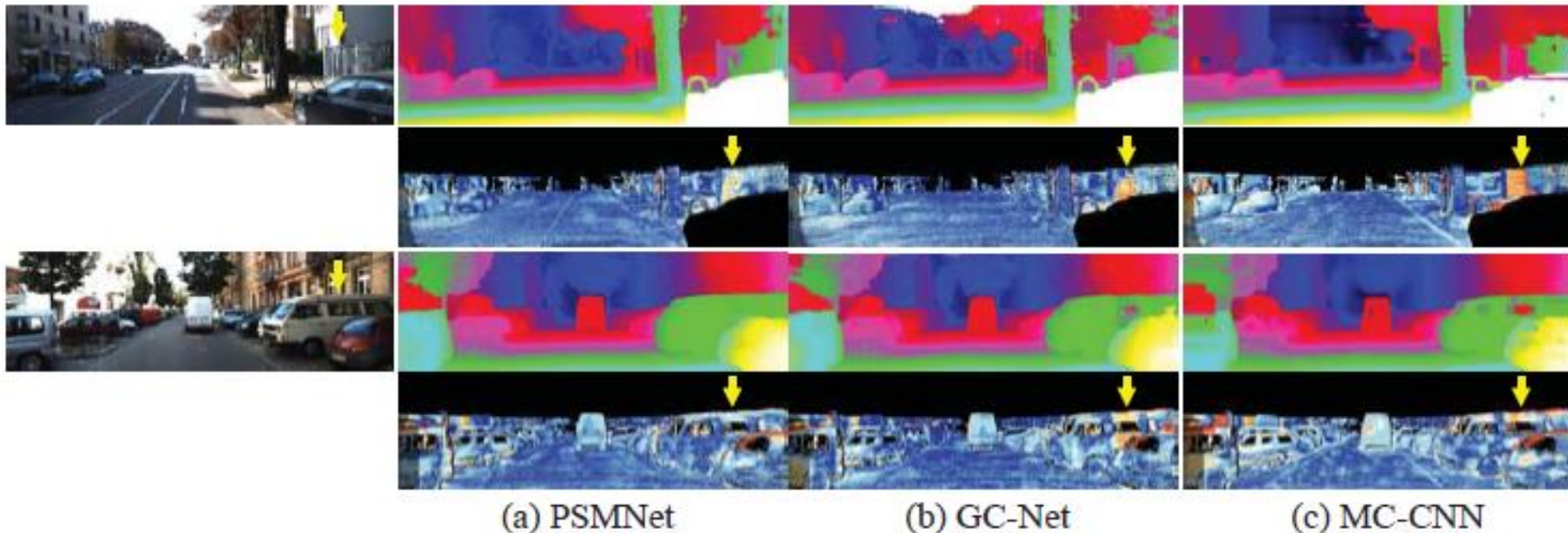
- Pyramid Stereo Matching Network (PSMNet)
- Multi-scale features for matching: **dilated convolution**, **spatial pyramid pooling**
- **3D CNN** for processing cost volume



Stereo matching

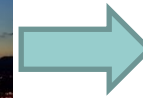
- PSMnet (CVPR 2018)

Rank	Method	All (%)			Noc (%)			Runtime (s)
		D1-bg	D1-fg	D1-all	D1-bg	D1-fg	D1-all	
1	PSMNet (ours)	1.86	4.62	2.32	1.71	4.31	2.14	0.41
3	iResNet-i2e2 [14]	2.14	3.45	2.36	1.94	3.20	2.15	0.22
6	iResNet [14]	2.35	3.23	2.50	2.15	2.55	2.22	0.12
8	CRL [21]	2.48	3.59	2.67	2.32	3.12	2.45	0.47
11	GC-Net [13]	2.21	6.16	2.87	2.02	5.58	2.61	0.90



Style Transfer

- Find an output image with
 - similar activations of early layers (low-level) of content image
 - similar activations of later layers (high-level) of style image



Style image

Content image

Output

Exploiting Spatial Relation for Reducing Distortion in Style Transfer [Chang and Chen, WACV 2021]

Style Transfer

